



PRESIDENCY COLLEGE

(AUTONOMOUS)

AFFILIATED TO BENGALURU CITY UNIVERSITY, APPROVED BY AICTE, DELHI & RECOGNISED BY THE GOVT. OF KARNATAKA
RE-ACCREDITED BY NAAC WITH 'A+' GRADE

A Project Report on

“CAREERAI HUB: CAMPUS PLACEMENT MANAGEMENT SYSTEM”

Submitted in partial fulfillment of the requirements for the award of the Degree.

Master of Computer Applications

From

Presidency College (Autonomous)

Affiliated to Bengaluru City University

Submitted By

Thakasila Tarun Kumar

24P01267

Department of Computer Applications

Presidency College (Autonomous)

Re-accredited by NAAC with 'A+'

Kempapura, Hebbal, Bengaluru – 24



Certificate

*This is to certify that **Thakasila Tarun kumar** with Register No. **24P01267** has satisfactorily completed the Fourth Semester MCA Project titled “**CareerAI Hub Campus Placement Management system**”, as a partial fulfilment of the requirements for the award of the Degree in **Master of Computer Applications**, awarded by Presidency College (Autonomous), during the Academic Year **2025-2026**.*

Project Guide :

Head of Department
(Department of computer Application)

Examiners

1. _____
2. _____

Reg No: _____

Examination Center: _____

Date of the exam: _____



Declaration

The project titled “**CareerAI Hub Campus Placement Management System**” developed by me in the partial fulfillment of Presidency College (Autonomous) affiliated to Bengaluru City University. It is a systematic work carried by us under the guidance of **Ms.Anu.D**, Assistant professor, Department of Computer Applications.

I declare that this project has not been submitted for awarding any degree of any other University.

Name of the student: Thakasila Tarun Kumar

Date:

Signature



Acknowledgement

The development of software is generally a bit complex and time-consuming task. The goal of developing the project “**CareerAI Hub Campus Placement Management System**” could not be achieved without the encouragements of kindly helpful and supportive people. Here by we convey our sincere thanks for all of them.

I take this opportunity to express my gratitude to people who had been instrumental in the successful completion of this project.

I am thankful to our management trustee for providing us an opportunity to work and complete the project successfully.

I wish to express my thanks to our principal **Dr. Pradeep Kumar Shinde** for his support to the project work. I would like to acknowledge my gratitude to our HOD of Master of Computer Applications. **Dr. Alli.A** for her encouragement and support.

The guidance and support received from our Internal Guide **Ms. Anu.D**, who contributed to this project, was vital for the success of the project. We are grateful for their constant support and help.



Sl.no	Description	Page no
01.	Introduction	4
	1.1 Modules	5-6
02.	Requirement Analysis	7-12
	2.1 Existing System	11
	2.2 Proposed System	11
	2.3 Feasibility Study	12
03.	Literature Survey	13-14
04.	System Specifications	15
05.	H/w and s/w Configurations	16
06.	Overview of Tools and Technologies used	17-30
	HTML5	17-20
	CSS3	21-22
	JAVASCRIPT	23
	MySQL	23-24
	Node.js	24
	PHP8.2	25
	Express.js	26
	Visual Studio Code	26-27
	XAMPP	27-28
	php MyAdmin	28-29
	Google Chrome	29-30
07.	System Design	31-40
	7.1 Data Flow Diagram	31-33
	7.2 E-R Diagram	34-35
	7.3 Structure of Table	36-40
08.	System Testing	41-46
	8.1 Level of Testing	47-51
09.	Output Screen Pages	52-58
10.	System Implementation – Coding	59-86
11.	Conclusion and Future Enhancements	87
12.	Bibliography	88



ABSTRACT:

The project titled "CareerAI Hub: Campus Placement Management System" is a comprehensive web-based application developed to automate and streamline the campus recruitment and placement process. Traditional placement management systems largely depend on manual record keeping, spreadsheets, and paper-based documentation, which often result in data redundancy, processing delays, communication gaps, and human errors. CareerAI Hub addresses these challenges by providing a centralized platform that efficiently manages student profiles, recruiter information, placement drives, job applications, and placement analytics.

The system offers role-based access control for three primary stakeholders: Students, Recruiters, and Placement Administrators. Students can create and manage their profiles, upload resumes, view eligibility criteria, apply for placement drives, and track application status. Recruiters can register their organizations, publish job opportunities, define eligibility requirements, review candidate applications, and update recruitment outcomes. Placement administrators are provided with advanced tools to manage student records, verify eligibility, schedule placement drives, monitor recruitment activities, generate reports, and communicate important notifications.

The application is developed using modern web technologies including HTML5, CSS3, JavaScript (ES6), Tailwind CSS, PHP 8.2, MySQL, Node.js, and Express.js. MySQL serves as the relational database management system, ensuring secure and efficient storage of placement-related information. The system follows a modular architecture and employs a normalized database design to maintain data integrity, minimize redundancy, and improve retrieval efficiency.

CareerAI Hub incorporates automated eligibility verification mechanisms based on academic criteria such as CGPA, branch, and backlog count. This feature significantly reduces manual effort and ensures that only eligible students are allowed to apply for specific placement drives. The system also provides real-time application tracking, notification management, and dashboard-based analytics that assist placement officers in monitoring recruitment performance and placement statistics.

The software development process follows a structured methodology involving requirement analysis, feasibility study, system design, database design, implementation, testing, and



deployment. Design artifacts such as Data Flow Diagrams (DFD), Entity Relationship Diagrams (ERD), database schemas, and system flowcharts are used to model system functionality and data interactions. Comprehensive testing techniques including Unit Testing, Integration Testing, System Testing, Black Box Testing, White Box Testing, Validation Testing, and Performance Testing were conducted to ensure reliability and accuracy.

The implementation of CareerAI Hub significantly improves operational efficiency, reduces administrative workload, enhances transparency in recruitment activities, and eliminates common errors associated with manual placement management. The system serves as an effective bridge between students, recruiters, and educational institutions, creating a more organized and efficient placement ecosystem.

Future enhancements may include Artificial Intelligence-based resume screening, machine learning-driven placement predictions, cloud deployment, mobile application support, online assessment modules, interview scheduling systems, and advanced analytics dashboards. These enhancements will further strengthen the platform and transform it into a complete placement intelligence and career development solution.



1.INTRODUCTION:

The CareerAI Hub: Campus Placement Management System is an advanced web-based platform developed to streamline and automate the entire campus recruitment process. Traditional placement management systems often rely on manual documentation, spreadsheets, and email-based communication, which can lead to inefficiencies, data redundancy, delays, and human errors. As educational institutions handle increasing numbers of students and recruiting organizations, there is a growing need for a centralized and intelligent placement management solution.

CareerAI Hub provides a comprehensive digital platform that connects students, recruiters, and placement administrators in a single ecosystem. Students can create and manage professional profiles, upload resumes, track application status, receive placement notifications, and apply for eligible job opportunities. Recruiters can register their organizations, publish job openings, define eligibility criteria, review applications, and shortlist candidates efficiently. Placement administrators can manage student records, company details, placement drives, and recruitment schedules through a secure administrative dashboard.

One of the key features of the system is its automated eligibility verification mechanism. The platform evaluates students based on predefined criteria such as CGPA, academic branch, backlog status, and graduation year, ensuring that only qualified candidates can apply for specific recruitment drives. This automation reduces administrative workload and eliminates manual screening errors.

The system is developed using PHP for the backend and MySQL for database management, ensuring reliable performance, secure data storage, and efficient information retrieval. Additional features include user authentication, role-based access control, placement analytics, application tracking, notification management, and report generation.

CareerAI Hub significantly improves transparency, communication, and operational efficiency throughout the placement process. By digitizing recruitment activities and providing real-time access to information, the system reduces processing time, enhances decision-making, and improves the overall placement experience for students, recruiters, and educational institutions



1.1 MAJOR MODULES:

- STUDENT PORTAL MODULE
- PLACEMENT ADMINISTRATION MODULE
- RECRUITER MANAGEMENT MODULE
- DATABASE MANAGEMENT MODULE

FUNCTIONALITIES OF MODULES:

1.Student Portal Module

The Student Portal Module provides students with a centralized platform to manage their placement activities efficiently. Students can register, create and update their profiles, upload resumes and academic documents, view available job opportunities, and apply for recruitment drives. The module also provides real-time notifications regarding placement drives, interview schedules, company announcements, and selection results. Students can track the status of their applications and access detailed information about recruiting companies and job roles. This module enhances student participation and ensures smooth communication between students and the placement cell.

2.Placement Administration Module

The Placement Administration Module enables placement officers to efficiently manage all recruitment-related activities through a secure administrative interface. It allows administrators to maintain student records, verify eligibility criteria, schedule placement drives, manage company information, and monitor recruitment progress. The module supports the publication of notifications, generation of placement reports, and analysis of placement statistics. By automating routine administrative tasks, it reduces manual effort, improves operational efficiency, and ensures transparency throughout the placement process.



3. Recruiter Management Module

The Recruiter Management Module provides organizations with a platform to conduct recruitment activities effectively. Recruiters can create company profiles, post job openings, define eligibility requirements, and manage recruitment drives. The module allows recruiters to view eligible candidates, access student resumes, shortlist applicants, schedule interviews, and update candidate selection status. It facilitates seamless interaction between recruiters and the institution while reducing the time and effort required for candidate screening and recruitment management.

4. Database Management Module

The Database Management Module serves as the core component of the system by securely storing and managing all placement-related information. It maintains student profiles, recruiter details, job postings, application records, interview schedules, and placement results in a structured manner. The module ensures data accuracy, integrity, security, and efficient retrieval of information. It also supports user authentication, data backup, report generation, and smooth communication among different modules. By providing a reliable data management framework, it ensures the overall stability and performance of the placement management system.



2.REQUIREMENT ANALYSIS:

2.1 SOFTWARE REQUIREMENTS:

The CareerAI Hub: Campus Placement Management System requires a combination of modern software technologies and development tools to ensure efficient development, testing, deployment, and maintenance. These software components work together to provide a secure, scalable, and user-friendly platform for managing campus placement activities.

Frontend Technologies

The frontend of the system is developed using HTML5, CSS3, JavaScript (ES6), and Tailwind CSS. HTML5 is used to structure web pages, while CSS3 provides styling and visual presentation. JavaScript enables dynamic and interactive functionalities such as form validation, real-time updates, and user interactions. Tailwind CSS helps create responsive and modern user interfaces that adapt seamlessly to different screen sizes and devices, ensuring an enhanced user experience.

Backend Technology

PHP 8.2 serves as the backend programming language responsible for implementing the business logic of the application. It handles user authentication, session management, form processing, data validation, and communication between the frontend and the database. PHP provides a secure and efficient environment for processing placement-related operations and managing user requests.

Database Management System

MySQL is used as the relational database management system for storing and managing all application data. It maintains records related to students, recruiters, placement drives, job applications, interview schedules, and placement results. MySQL ensures data consistency, efficient query processing, and reliable storage, making it suitable for handling large volumes of placement-related information.



Database Administration Tool

phpMyAdmin is used for database administration and management. It provides a graphical user interface for creating databases, managing tables, executing SQL queries, importing and exporting data, and monitoring database performance. This tool simplifies database maintenance and reduces the complexity of manual database operations.

Application Development and Deployment Stack

Node.js, Express.js, and Vite are utilized to support modern web application development and deployment. Node.js provides a runtime environment for executing JavaScript outside the browser. Express.js facilitates server-side application development and API management, while Vite enhances frontend development through faster build processes, hot module replacement, and optimized production builds. Together, these technologies improve application performance, scalability, and maintainability.

Development Environment

Visual Studio Code (VS Code) and PyCharm Professional are used as integrated development environments (IDEs) for coding, debugging, testing, and project management. These tools offer advanced features such as syntax highlighting, code completion, version control integration, debugging support, and extension ecosystems that improve developer productivity and code quality.

Web Server

XAMPP and WAMP Server are used to create a local server environment for application development and testing. These platforms include Apache Web Server, PHP, and MySQL, allowing developers to run and test the application on local machines before deployment. They provide an easy-to-configure environment for web application development.

Operating System Support

The system is designed to be platform-independent and can operate on multiple operating systems, including Windows 10, Windows 11, macOS Sequoia, and various Linux distributions such as Ubuntu and Fedora. This flexibility enables developers and administrators to choose the operating environment that best suits their requirements.



2.2 HARDWARE REQUIREMENTS

The successful development, testing, deployment, and operation of the CareerAI Hub: Campus Placement Management System require adequate hardware resources to ensure reliable performance and efficient execution of application processes. The following hardware components are recommended for supporting the system and its development environment.

System Configuration

A desktop computer or laptop with a modern hardware configuration is required to develop, test, and run the application. The system should be capable of supporting web development tools, database servers, browsers, and other software components simultaneously without performance issues. A dedicated system with updated hardware specifications improves development efficiency and application responsiveness.

Processor

The application requires an Intel Core i3, Intel Core i5, Intel Core i7, AMD Ryzen 3, Ryzen 5, or equivalent processor. A multi-core processor is recommended to handle database operations, server-side processing, and multitasking efficiently. Higher-performance processors improve system responsiveness during testing, deployment, and concurrent user operations.

Memory (RAM)

A minimum of 4 GB RAM is required to run the application and supporting software tools. However, 8 GB RAM or higher is strongly recommended for smoother performance, faster application execution, efficient database management, and simultaneous use of development environments, browsers, and server applications. Systems with 16 GB RAM provide better performance for large-scale testing and future scalability.

Storage Device

The system requires at least 50 GB of free storage space for installing the operating system, development tools, database software, project files, backups, and documentation. Solid State Drives (SSD) are recommended instead of traditional Hard Disk Drives (HDD) because they offer faster data access, reduced loading times, and improved overall system performance.

Monitor and Display

A monitor with a minimum screen resolution of 1366×768 pixels is required for effective application development and testing. A Full HD (1920×1080) display is recommended to provide better visibility, enhanced user interface design capabilities, and improved productivity during coding and debugging activities.

Input Devices

Standard input devices such as a keyboard and mouse are necessary for data entry, application navigation, system configuration, and software development tasks. Ergonomic input devices may be used to improve user comfort during extended development sessions.



Network Connectivity

A stable internet connection is recommended for downloading software packages, receiving updates, communicating with recruiters and students, accessing cloud services, using version control systems such as GitHub, and deploying the application to web servers. Broadband or high-speed internet connectivity ensures uninterrupted access to online resources and services.

Backup and Storage Devices

External storage devices such as USB drives, external hard disks, or cloud storage services may be used for maintaining project backups, database exports, and important documentation. Regular backups help prevent data loss and support disaster recovery.

Server Hardware (Optional for Deployment)

For institutional deployment, a dedicated server with higher hardware specifications may be used to support multiple concurrent users. Recommended server specifications include:

- Intel Xeon or equivalent processor
- 16 GB RAM or higher
- 250 GB SSD or greater storage capacity
- High-speed network connectivity
- Backup power supply and security mechanisms

Power Supply

A reliable power source is necessary to ensure uninterrupted operation of the system. The use of an Uninterruptible Power Supply (UPS) is recommended to prevent data loss and system shutdown during power failures.

Hardware Compatibility

The hardware should be compatible with modern operating systems such as Windows 10/11, Linux distributions, and macOS. Compatibility ensures smooth installation and operation of the software components required for the placement management system.



2.3 EXISTING SYSTEM:

The existing placement management process mainly relies on spreadsheets, paper records, emails, and Google Forms. This manual approach creates several challenges in managing student and recruiter information.

Limitations of the Existing System

- Student records are stored in multiple files, leading to data duplication and inconsistency.
- Eligibility verification is performed manually, making the process time-consuming and error-prone.
- There is no real-time tracking of application and interview status.
- Important placement notifications may be missed due to reliance on emails and notice boards.
- Generating placement reports and statistics requires significant manual effort.

2.4 PROPOSED SYSTEM:

CareerAI Hub is a centralized web-based placement management system designed to automate and streamline placement activities.

Advantages of the Proposed System

- Maintains all student, recruiter, and placement data in a centralized database.
- Automatically verifies student eligibility based on predefined criteria such as CGPA, branch, and backlogs.
- Provides separate dashboards for administrators, students, and recruiters.
- Enables real-time tracking of applications, interviews, and placement status.
- Generates placement reports, analytics, and statistics instantly.
- Improves accuracy, transparency, and efficiency in the placement process



2.5 FEASIBILITY STUDY:

A feasibility study is conducted to evaluate whether the proposed system can be developed and implemented successfully within the available resources, time, and operational environment. For the CareerAI Hub: Campus Placement Management System, the feasibility study includes technical, economic, operational, and schedule feasibility.

Technical Feasibility

The proposed system is developed using HTML5, CSS3, JavaScript, PHP, MySQL, Node.js, and related web technologies. The required software tools, development environment, and hardware resources are readily available. The technologies used are reliable, widely supported, and suitable for developing a secure web-based placement management system. Therefore, the project is technically feasible.

Economic Feasibility

The project utilizes open-source technologies such as PHP, MySQL, Node.js, and XAMPP, which significantly reduce software licensing costs. Since the system automates manual placement activities, it reduces administrative effort, saves time, and improves productivity. Hence, the project is economically feasible.

Operational Feasibility

The system is designed with a user-friendly interface that can be easily used by students, recruiters, and placement administrators. It provides centralized data management, automated eligibility verification, real-time notifications, and placement tracking. These features improve efficiency and transparency in placement operations, making the system operationally feasible.

Schedule Feasibility

The project can be completed within the timeframe allocated for the MCA academic project. The availability of development tools, technologies, and project resources ensures smooth implementation, testing, and deployment within the planned schedule. Therefore, the project is schedule feasible.



3.LITERATURE SURVEY:

The placement process is one of the most important activities in higher educational institutions, as it serves as a bridge between students and potential employers. With the increasing number of students and recruiting organizations, managing placement activities through traditional methods has become challenging. Educational institutions require efficient systems to handle student information, company details, recruitment drives, eligibility verification, and placement statistics. As a result, researchers and software developers have focused on designing automated placement management systems to improve the efficiency of campus recruitment processes.

Traditionally, placement cells maintained student records, company information, and recruitment details using spreadsheets, paper documents, notice boards, and email communications. Although these methods were effective for smaller student populations, they became difficult to manage as the volume of data increased. Manual processes often resulted in data redundancy, record inconsistencies, delayed communication, and errors in eligibility verification. These limitations highlighted the need for computerized placement management solutions.

Several web-based recruitment and placement systems have been developed to automate various aspects of the recruitment process. These systems allow students to create profiles, upload resumes, and apply for available job opportunities online. Recruiters can post job openings, define eligibility criteria, and review candidate applications. Such systems have significantly reduced paperwork and improved communication between institutions and companies.

Research studies have shown that centralized database systems improve data consistency and accessibility. By storing student records, recruiter details, placement drives, and application information in a single database, institutions can reduce duplication and ensure accurate information management. Database-driven systems also provide better security, faster data retrieval, and efficient record maintenance compared to manual methods.

Modern placement management systems incorporate automated eligibility verification mechanisms. These systems compare student academic information, such as CGPA, branch, and backlog status, against company requirements before allowing applications. This automation reduces the workload of placement

officers and minimizes the chances of human error. It also ensures that only eligible candidates participate in recruitment drives.

Recent advancements in web technologies have further enhanced placement management systems by introducing real-time notifications, dashboard analytics, and application tracking features. Students can instantly receive updates regarding recruitment drives, interview schedules, and selection results. Placement administrators can monitor placement performance



through graphical reports and statistical summaries, enabling better decision-making and planning.

Many existing recruitment platforms focus primarily on job matching and online applications. However, they often lack features specifically required by educational institutions, such as academic eligibility checks, placement drive management, student verification, and institutional reporting. Consequently, there is a growing demand for a dedicated campus placement management solution that addresses the unique requirements of colleges and universities.

The proposed CareerAI Hub: Campus Placement Management System is designed to overcome these limitations by providing a centralized and automated platform for managing placement activities. The system integrates student profile management, recruiter management, eligibility verification, application tracking, drive scheduling, notifications, and placement analytics within a single environment. This approach improves efficiency, transparency, and accuracy throughout the placement process.

In conclusion, the literature survey indicates that automated placement management systems play a vital role in modern educational institutions. Existing studies and systems emphasize the importance of centralized data management, automation, and real-time communication. Building upon these concepts, CareerAI Hub provides a comprehensive solution that enhances campus recruitment operations and supports effective interaction between students, recruiters, and placement administrators.



4.SYSTEM SPECIFICATION:

A software requirements specification (SRS) is a comprehensive description of the intended purpose and environment for software under development. The SRS fully describes what the software will do and how it will be expected to perform.

An SRS minimizes the time and effort required by developers to achieve desired goals and also minimizes the development cost. A good SRS defines how an application will interact with system hardware, other programs, and human users in a wide variety of real-world situations. Parameters such as operating speed, response time, availability, portability, maintainability, footprint, security, and speed of recovery from adverse events are evaluated.

The requirements of this web application (ems) developed by us is very well understood and documented initially when the process starts. Accordingly, the hardware and software requirements are chosen. The same can be referred to in the next section of this report. This web application runs in the Apache Tomcat Server, which can be accessed by web clients through the Internet from anywhere in the world. The SRS parameters are adopted in this system, with the detailed feasibility study. While developing, this system was tested as the localhost, through the Mozilla Firefox browser. Also, few other popular web browsers are tested.



5.SOFTWARE AND HARDWARE CONFIGURATION:

HARDWARE CONFIGURATION

Component	Specification
Processor	AMD Ryzen 5 5000 series
RAM	4 GB Minimum (8 GB Recommended)
Hard Disk (HDD/SSD)	256 GB or Higher
Monitor	Standard LCD/LED Display
Keyboard & Mouse	Standard Input Devices
Network	Internet Connection (Optional for Deployment)

Software Configuration

Component	Specification
Client-SideTechnologies (Frontend)	HTML5, CSS3, JavaScript (ES6), Tailwind CSS
Server-Side Technologies (Backend)	PHP 8.2, Node.js, Express.js
Database Management System	MySQL
IDE	Visual Studio Code (VS Code)
Web Server	XAMPP / WAMP Server
Connection Technology	MySQL Connector, REST APIs
Operating System	Windows 10/11, Linux, macOS
Development Tools	PHP, MySQL, Node.js, Vite, HTML5, CSS3, JavaScript, Tailwind CSS
Daabase Administration Tool	phpMyAdmin



6.1 OVERVIEW OF TOOL AND TECHNOLOGIES USED

The development of the CareerAI Hub: Campus Placement Management System involves the use of various tools and technologies for designing, developing, testing, and managing the application. These technologies ensure efficient performance, secure data management, and a user-friendly experience.

HTML5



Introduction

HTML5 (HyperText Markup Language Version 5) is the latest standard version of HyperText Markup Language used for designing and structuring web pages and web applications. It serves as the foundation of modern web development by providing a standardized method for organizing content, displaying information, and creating interactive user interfaces. HTML5 was developed to overcome the limitations of previous HTML versions and to support the growing demand for multimedia-rich, responsive, and highly interactive web applications.

HTML5 is not only a markup language but also a complete platform for building modern web applications. It introduces new semantic elements, multimedia capabilities, advanced form controls, local storage mechanisms, and APIs that improve the functionality, performance, and accessibility of websites. Due to its flexibility and compatibility, HTML5 has become the preferred technology for developing web-based systems across various domains, including education, healthcare, banking, e-commerce, and recruitment management.

Evolution of HTML5

Before HTML5, web developers relied heavily on external plugins such as Adobe Flash and Silverlight to display multimedia content. Earlier versions of HTML provided limited support for dynamic content and interactive applications. As web technologies evolved, the need for a more powerful and efficient standard became evident.

HTML5 was introduced by the World Wide Web Consortium (W3C) and WHATWG (Web Hypertext Application Technology Working Group) to provide native support for multimedia, graphics, data storage, and advanced web functionalities. It significantly simplified web development and enabled developers to build rich internet applications without depending on third-party technologies.



Features of HTML5

1. Semantic Elements

One of the most important improvements introduced by HTML5 is the use of semantic elements. These tags provide meaningful descriptions of webpage content and improve code readability.

Examples include:

- `<header>` – Defines the header section of a webpage.
- `<nav>` – Represents navigation links.
- `<section>` – Defines a logical section of content.
- `<article>` – Represents independent content.
- `<aside>` – Defines supplementary information.
- `<footer>` – Represents the footer section.

Semantic elements improve website structure, accessibility, and search engine optimization.

2. Native Multimedia Support

HTML5 provides built-in support for audio and video content through dedicated tags.

- `<audio>` tag for embedding audio files.
- `<video>` tag for embedding video files.

This feature eliminates the need for external plugins and improves browser compatibility. Multimedia content can now be integrated directly into web pages, resulting in better performance and user experience.

3. Enhanced Form Controls

HTML5 introduces several advanced form input types and validation features. Examples include:

- Email fields
- Number fields
- Date pickers
- URL fields
- Range sliders
- Search boxes

Built-in validation reduces the amount of JavaScript code required and helps ensure accurate user input.

4. Canvas for Graphics

The `<canvas>` element enables dynamic drawing of graphics using JavaScript.

Applications include:

- Charts and graphs
- Animations
- Interactive visualizations
- Browser-based games
- Data dashboards

Canvas allows developers to create rich graphical interfaces directly within the browser.



5. Local and Session Storage

HTML5 provides client-side storage mechanisms that allow data to be stored locally within the browser.

Local Storage

Stores data permanently until manually removed.

1. Session Storage

- Stores data temporarily during the current browser session.
- These storage mechanisms improve application performance by reducing server requests and enabling offline functionality.

2. Geolocation Support

HTML5 includes a Geolocation API that allows web applications to access a user's geographical location with permission.

Applications include:

- Navigation systems
- Location-based services
- Delivery tracking systems
- Mapping applications

3. Offline Web Applications

HTML5 supports offline capabilities through local storage and caching mechanisms. Users can continue accessing certain application features even when internet connectivity is unavailable.

4. Cross-Platform Compatibility

HTML5 applications can run on various operating systems and devices, including:

- Windows
- Linux
- macOS
- Android
- iOS

This compatibility ensures a consistent user experience across multiple platforms.

5. Improved Accessibility

HTML5 improves accessibility for users with disabilities through semantic tags and support for assistive technologies such as screen readers.

Benefits include:

- Better navigation
- Improved content understanding
- Enhanced usability for all users

6. Search Engine Optimization (SEO)

Semantic HTML5 elements help search engines understand webpage content more effectively. This improves website visibility and ranking in search engine results.



Advantages of HTML5

- Easy to learn and implement.
- Supports multimedia without external plugins.
- Provides faster webpage loading.
- Improves website performance.
- Enhances user experience.
- Supports responsive web design.
- Offers better security and reliability.
- Reduces dependency on third-party software.
- Improves accessibility and SEO.
- Compatible with all modern browsers.

Applications of HTML5

- HTML5 is widely used in:
- Educational portals
- E-commerce websites
- Banking systems
- Social networking platforms
- Healthcare applications
- Online learning systems
- Recruitment and placement management systems
- Content management systems
- Business websites
- Mobile web applications

HTML5 in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, HTML5 is used as the primary technology for designing and structuring web pages. It is responsible for creating student registration forms, login pages, recruiter dashboards, placement drive listings, application forms, profile management pages, and administrative interfaces.

The semantic elements of HTML5 help organize content effectively, while advanced form controls facilitate accurate data entry during student registration and application submission. HTML5 also contributes to responsive design, ensuring that the application functions smoothly across desktops, laptops, tablets, and mobile devices.

By providing a structured and interactive interface, HTML5 enhances the overall usability of the CareerAI Hub system and enables efficient interaction between students, recruiters, and placement administrators.



CSS3



Introduction

CSS3 (Cascading Style Sheets Level 3) is the latest version of Cascading Style Sheets used to control the presentation, formatting, and layout of web pages. It is one of the core technologies of web development, working alongside HTML5 and JavaScript to create modern, interactive, and visually appealing websites. While HTML defines the structure of a webpage, CSS3 determines how that content is displayed to users.

CSS3 was introduced to overcome the limitations of earlier CSS versions and provide advanced styling features for modern web applications. It enables developers to create responsive layouts, attractive designs, animations, and visual effects without relying heavily on external scripts or images. Due to its flexibility and powerful features, CSS3 has become an essential technology in the development of web-based systems.

Features of CSS3

1. Responsive Web Design

CSS3 supports responsive web design through media queries, allowing web pages to automatically adapt to different screen sizes and devices. This ensures that websites function properly on desktops, laptops, tablets, and smartphones.

2. Advanced Styling

CSS3 provides extensive styling capabilities, including control over fonts, colors, backgrounds, margins, padding, borders, and spacing. These features help developers create attractive and professional-looking web pages.

3. Animations and Transitions

CSS3 allows developers to add animations and transition effects directly to web elements. These effects improve user interaction by making the interface more dynamic and visually engaging.

Examples include:

- Hover effects on buttons
- Fade-in and fade-out effects
- Moving and rotating elements
- Loading animations

4. Flexbox Layout

Flexbox is a layout model introduced in CSS3 that makes it easier to arrange elements within a webpage. It provides efficient alignment, spacing, and distribution of content, even when



screen sizes change.

5. Grid Layout

CSS Grid is a powerful two-dimensional layout system that allows developers to create complex webpage structures using rows and columns. It simplifies webpage design and improves layout flexibility.

6. Gradients and Background Effects

CSS3 supports linear and radial gradients, enabling smooth color transitions without the use of image files. It also provides background effects that enhance the visual appeal of web pages.

7. Shadows and Visual Effects

CSS3 introduces Box Shadow and Text Shadow properties, allowing developers to create depth and emphasis in user interface components.

Benefits include:

- Improved readability
- Enhanced visual appearance
- Modern user interface design

8. Rounded Corners

The Border Radius property allows developers to create rounded corners for buttons, images, forms, and containers without using graphics.

9. Custom Fonts

CSS3 supports custom typography through the @font-face rule, enabling the use of unique fonts that enhance website branding and design consistency.

10. Browser Compatibility

CSS3 is supported by all modern web browsers, including Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. This ensures consistent performance across different platforms.

CSS3 in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, CSS3 is used to enhance the visual presentation and usability of the application. It is responsible for designing student registration forms, login pages, recruiter dashboards, placement drive listings, administrative panels, navigation menus, and report pages.

CSS3 ensures that the application is responsive and accessible across different devices and screen sizes. Features such as Flexbox, Grid layouts, animations, and responsive design help create a modern and user-friendly interface. The technology improves navigation, readability, and overall user experience for students, recruiters, and placement administrators.



JavaScript (ES6)



JavaScript

Introduction

JavaScript (ECMAScript 6 or ES6) is a popular programming language used to create interactive and dynamic web applications. It works together with HTML5 and CSS3 to enhance the functionality and responsiveness of web pages. ES6 introduced several advanced features such as arrow functions, classes, modules, template literals, and promises, making JavaScript more efficient and easier to maintain.

Features of JavaScript (ES6)

- Adds interactivity to web pages.
- Supports event handling and form validation.
- Provides object-oriented programming features.
- Introduces arrow functions and template literals.
- Supports asynchronous programming using Promises and Async/Await.
- Enables dynamic content updates without page reloads.

JavaScript in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, JavaScript (ES6) is used for form validation, dynamic content updates, user interactions, placement drive filtering, dashboard functionality, and real-time status updates. It enhances the responsiveness and usability of the application.

MySQL



Introduction

MySQL is an open-source Relational Database Management System (RDBMS) used to store, organize, and manage data efficiently. It uses Structured Query Language (SQL) to perform operations such as inserting, updating, deleting, and retrieving data. MySQL is widely used in web applications because of its reliability, security, and high performance.

Features of MySQL

- Stores data in tables consisting of rows and columns.
- Supports SQL queries for efficient data management.



- Provides high performance and fast data retrieval.
- Ensures data security through user authentication and access control.
- Supports multi-user access and concurrent operations.
- Maintains data integrity using primary keys and foreign keys.
- Offers backup and recovery mechanisms to protect data.

MySQL in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, MySQL is used as the backend database to store student profiles, recruiter information, placement drives, job applications, interview schedules, notifications, and placement records. It ensures secure data storage, quick retrieval of information, and efficient management of placement activities.

Node.js



Introduction

Node.js is an open-source, cross-platform JavaScript runtime environment that allows developers to execute JavaScript code on the server side. It is built on Google's V8 JavaScript engine and is widely used for developing fast, scalable, and high-performance web applications. Node.js uses an event-driven and non-blocking architecture, enabling it to handle multiple user requests efficiently.

Features of Node.js

- Executes JavaScript on the server side.
- Uses an event-driven and non-blocking architecture.
- Supports asynchronous programming for better performance.
- Handles multiple client requests simultaneously.
- Provides fast execution through the V8 JavaScript engine.
- Supports the development of scalable and real-time applications.
- Offers a large collection of reusable packages through NPM (Node Package Manager).

Node.js in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, Node.js is used to develop backend services and REST APIs. It facilitates communication between the frontend and database, processes user requests, manages application data, and supports real-time functionalities such as notifications, dashboard updates, and placement tracking.



PHP 8.2



Introduction

PHP (Hypertext Preprocessor) 8.2 is a popular open-source server-side scripting language used for developing dynamic and interactive web applications. It is widely used for creating websites and database-driven applications due to its simplicity, flexibility, and efficiency. PHP code is executed on the server, and the generated output is sent to the user's web browser.

Features of PHP 8.2

- Open-source and freely available.
- Supports server-side scripting and web development.
- Easily integrates with MySQL and other databases.
- Supports Object-Oriented Programming (OOP).
- Provides improved performance and security features.
- Handles form processing, session management, and user authentication.
- Compatible with multiple operating systems such as Windows, Linux, and macOS.

Advantages of PHP 8.2

- Easy to learn and implement.
- Fast and efficient execution.
- Supports dynamic web page generation.
- Highly secure and reliable.
- Large community support and extensive documentation.
- Platform-independent and flexible.
- Cost-effective for web application development.

PHP 8.2 in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, PHP 8.2 is used to implement server-side functionalities such as student registration, login authentication, recruiter management, placement drive creation, eligibility verification, application processing, and database interactions. It acts as a bridge between the user interface and the MySQL database, ensuring smooth data processing and secure system operations.



Express.js



Introduction

Express.js is a fast, lightweight, and flexible web application framework built on top of Node.js. It simplifies server-side development by providing powerful features for handling routing, middleware, HTTP requests, and API development. Express.js helps developers build scalable and efficient web applications with less code and better organization.

Features of Express.js

- Provides a simple and flexible routing system.
- Supports middleware for request and response processing.
- Simplifies RESTful API development.
- Handles HTTP requests and responses efficiently.
- Supports integration with databases and third-party services.
- Enables secure and scalable web application development.
- Works seamlessly with Node.js.

Express.js in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, Express.js is used to create and manage REST APIs, handle user requests, process application data, and facilitate communication between the frontend and the database. It helps manage student registrations, recruiter information, placement drives, application tracking, and dashboard functionalities efficiently.

6.7 VISUAL STUDIO CODE:



Visual Studio Code is a streamlined code editor with support for development operations like debugging, task running, and version control. It aims to provide just the tools a developer needs



for a quick code-build-debug cycle and leaves more complex workflows to fuller featured IDEs, such as Visual Studio IDE. Visual Studio Code, also commonly referred to as VS Code, is a source-code editor made by Microsoft with the Electron Framework, for Windows, Linux and macOS. Features include support for debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded Git.

XAMPP Server



Introduction

XAMPP is a free and open-source local web server package used for developing, testing, and deploying web applications. The name XAMPP stands for Cross-Platform (X), Apache (A), MySQL (M), PHP (P), and Perl (P). It provides a complete development environment that allows developers to run PHP and MySQL applications on a local computer without requiring an internet connection.

XAMPP is widely used because it simplifies the installation and configuration of web development tools. It enables developers to create, test, and debug web applications before deploying them to a live server.

Features of XAMPP Server

- Provides Apache web server for hosting web applications.
- Supports PHP for server-side scripting.
- Includes MySQL database for data management.
- Offers phpMyAdmin for database administration.
- Easy installation and configuration.
- Supports multiple operating systems such as Windows, Linux, and macOS.
- Provides a local development and testing environment. .



XAMPP Server in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, XAMPP is used as the local server environment for running PHP scripts and managing the MySQL database. It hosts the application during development and testing, allowing developers to verify functionalities such as student registration, recruiter management, placement drive scheduling, and database operations before deployment.

phpMyAdmin



Introduction

phpMyAdmin is a free and open-source web-based database administration tool used for managing MySQL and MariaDB databases. It provides a graphical user interface (GUI) that allows users to perform database operations without writing complex SQL commands. phpMyAdmin simplifies database management tasks such as creating databases, tables, records, queries, and user accounts.

It is commonly included with XAMPP, WAMP, and other web server packages, making it a popular tool for web developers and database administrators.

Features of phpMyAdmin

- Provides a user-friendly graphical interface for database management.
- Allows creation, modification, and deletion of databases and tables.
- Supports SQL query execution and database administration.
- Enables importing and exporting of database files.
- Provides user account and privilege management.

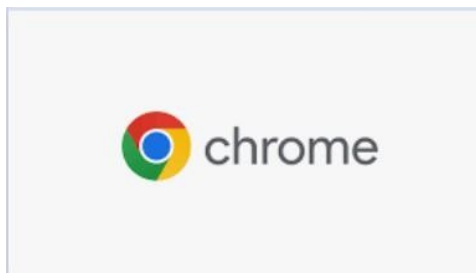


- Supports backup and recovery operations.
- Displays database structure, relationships, and records efficiently.

phpMyAdmin in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, phpMyAdmin is used to create and manage the MySQL database. It helps in designing tables, defining relationships, inserting records, executing queries, and monitoring placement-related data such as student profiles, recruiter details, placement drives, applications, and interview schedules. It also assists in database backup and maintenance activities.

Google Chrome



Introduction

Google Chrome is a widely used web browser developed by Google. It is known for its high speed, security, reliability, and user-friendly interface. Chrome supports modern web standards and provides a powerful platform for accessing, testing, and running web applications. Due to its performance and compatibility, it is one of the most popular browsers used by developers and end users worldwide.

Features of Google Chrome

- Fast and efficient web page loading.
- User-friendly and easy-to-navigate interface.
- Supports modern web technologies such as HTML5, CSS3, and JavaScript.
- Provides built-in security features against malicious websites.
- Includes Developer Tools for debugging and testing web applications.
- Supports extensions and plugins for enhanced functionality.



Google Chrome in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, Google Chrome is used to access, test, and verify the functionality of the web application. It helps developers check the responsiveness, performance, and compatibility of the system across different screen sizes and devices. Chrome's Developer Tools are also used for debugging, inspecting webpage elements, and monitoring application performance during development.

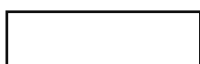


7.SYSTEM DESIGN:

7.1 DATA FLOW DIAGRAM

Data Flow Diagrams is the best and easiest tool to represent the flow of the data in the project. It is otherwise known as a bubble chart. It has the purpose of clarifying system requirements and identifying major transformations that will become programs in System design. It is the major starting point in the design phase that functionally decomposes the requirements specifications down to the lowest level of detail. A DFD consists of a series of bubbles joined by lines. The bubble represents data transformation and lines represent data flow in the system. A data flow diagram (DFD) maps out the flow of information for any process or system. It uses defined symbols like rectangles, circles and arrows, plus short text labels, to show data inputs, outputs, storage points and the routes between each destination. Data flowcharts can range from simple, even hand-drawn process overviews, to in-depth, multi-level DFDs that dig progressively deeper into how the data is handled. They can be used to analyze an existing system or model a new one. Like all the best diagrams and charts, a DFD can often visually “say” things that would be hard to explain in words, and they work for both technical and nontechnical audiences, from developer to CEO. That’s why DFDs remain so popular after all these years. While they work well for data flow software and systems, they are less applicable nowadays to visualizing interactive, real-time or database-oriented software or systems.

Symbols



Square, this defines source or destination of data.

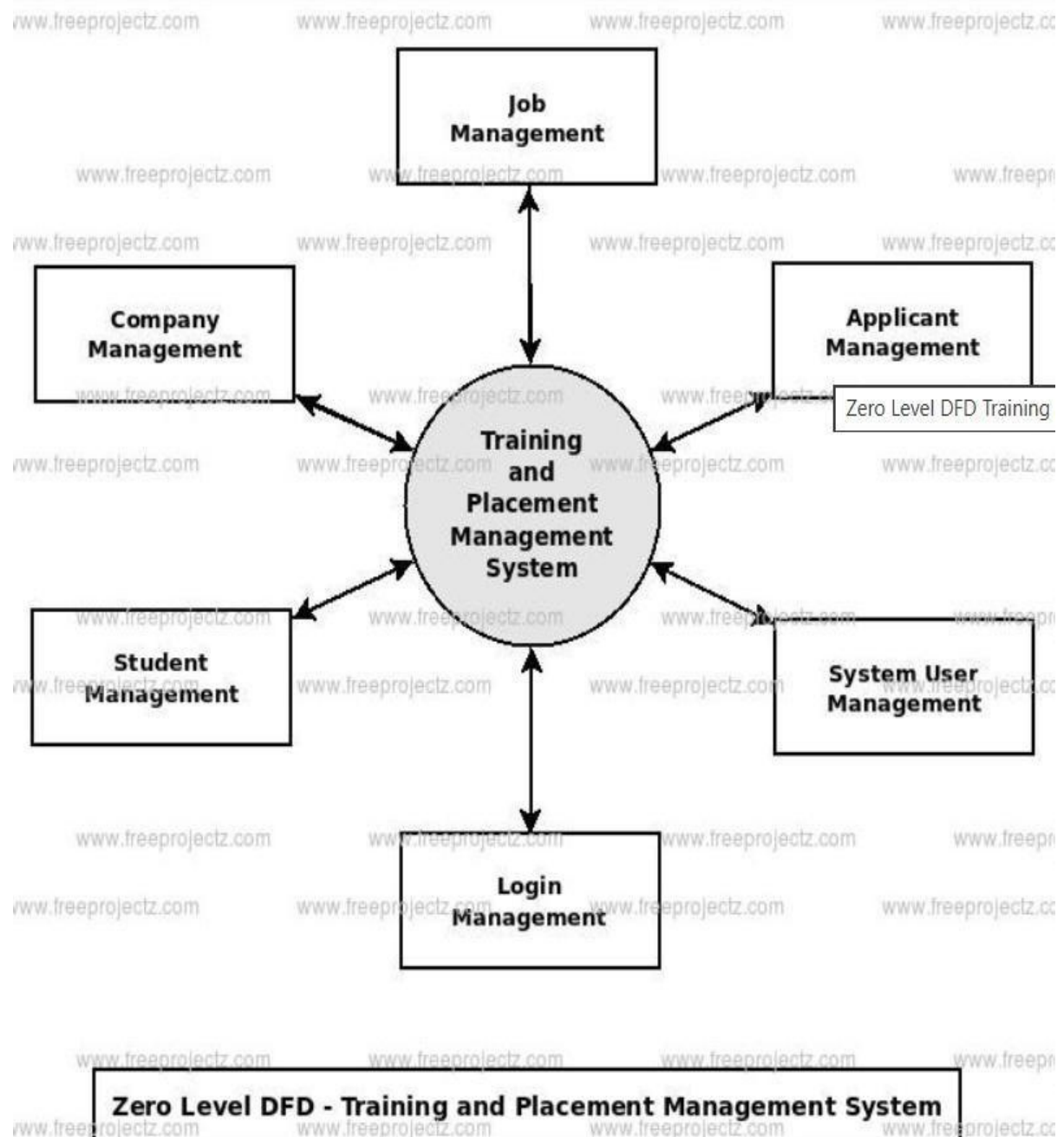


Arrow, which shows data flow

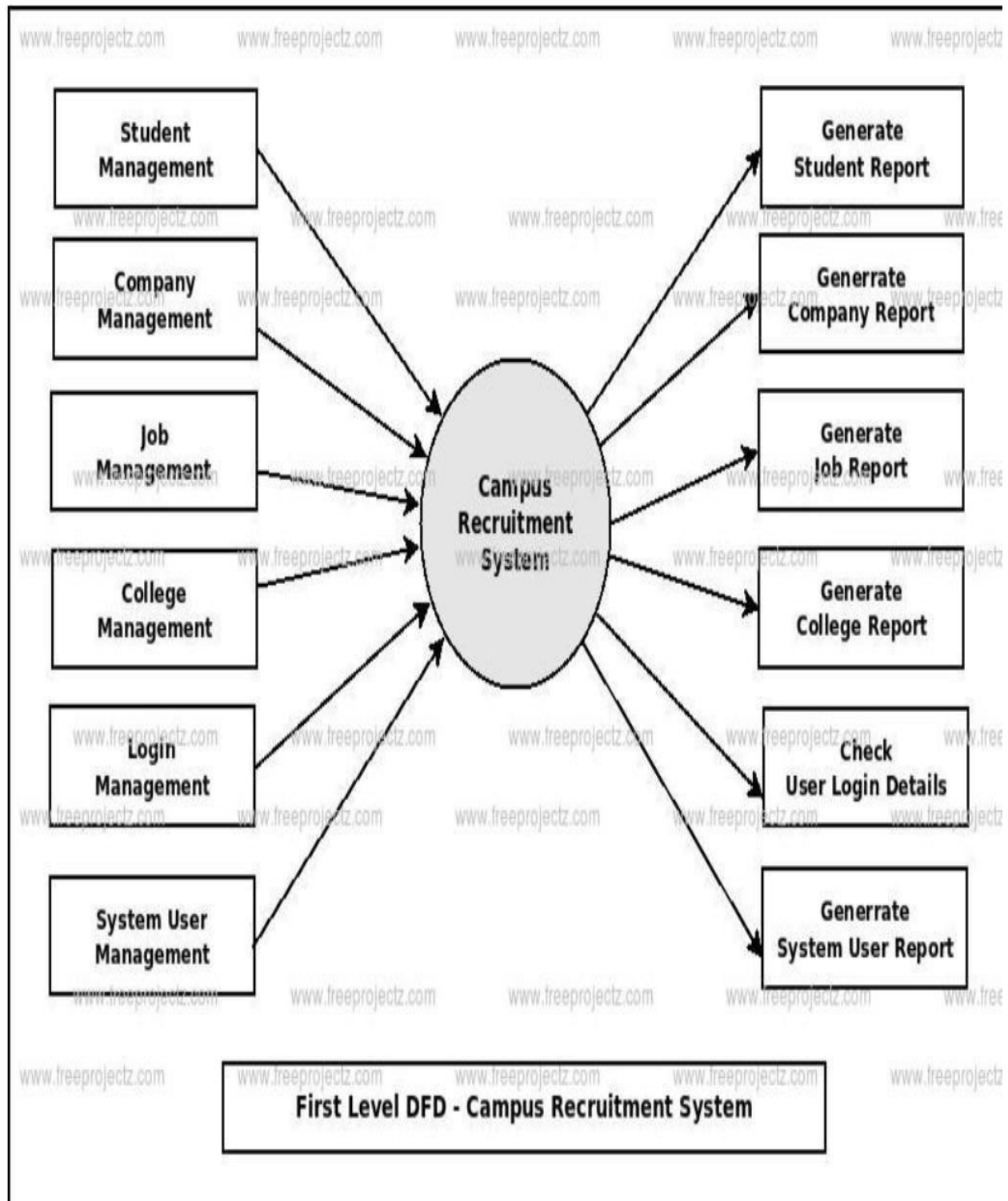


Circle, which represents a process that transforms incoming data into outgoingflow.

DATA FLOW DIAGRAM (LEVEL 0)



DATA FLOW DIAGRAM (LEVEL 1):





7.2 ER DIAGRAM

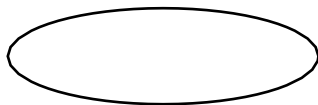
In software engineering an entity–relationship model (ER model) is data model for describing the data or information aspects of a business domain or its process requirements, in an abstract way that lends itself to ultimately being implemented in databases such as relational databases. The main components of ER models are entities (things) and the relationships that can exist among them.

Entity–relationship modelling was developed by Peter Chen and published in a 1976 paper. However, variants of the idea existed previously, and have been devised subsequently such as supertype and subtype data entities and commonality relationships.

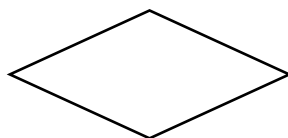
The overall logical structure of a database can be expressed graphically by an ER diagram. While drawing the ER diagram, entity names are represented by a rectangle, relationships are represented by diamond and oval shapes are used for representing attribute



Rectangle: Represents entity set.



Oval: Represents attribute.

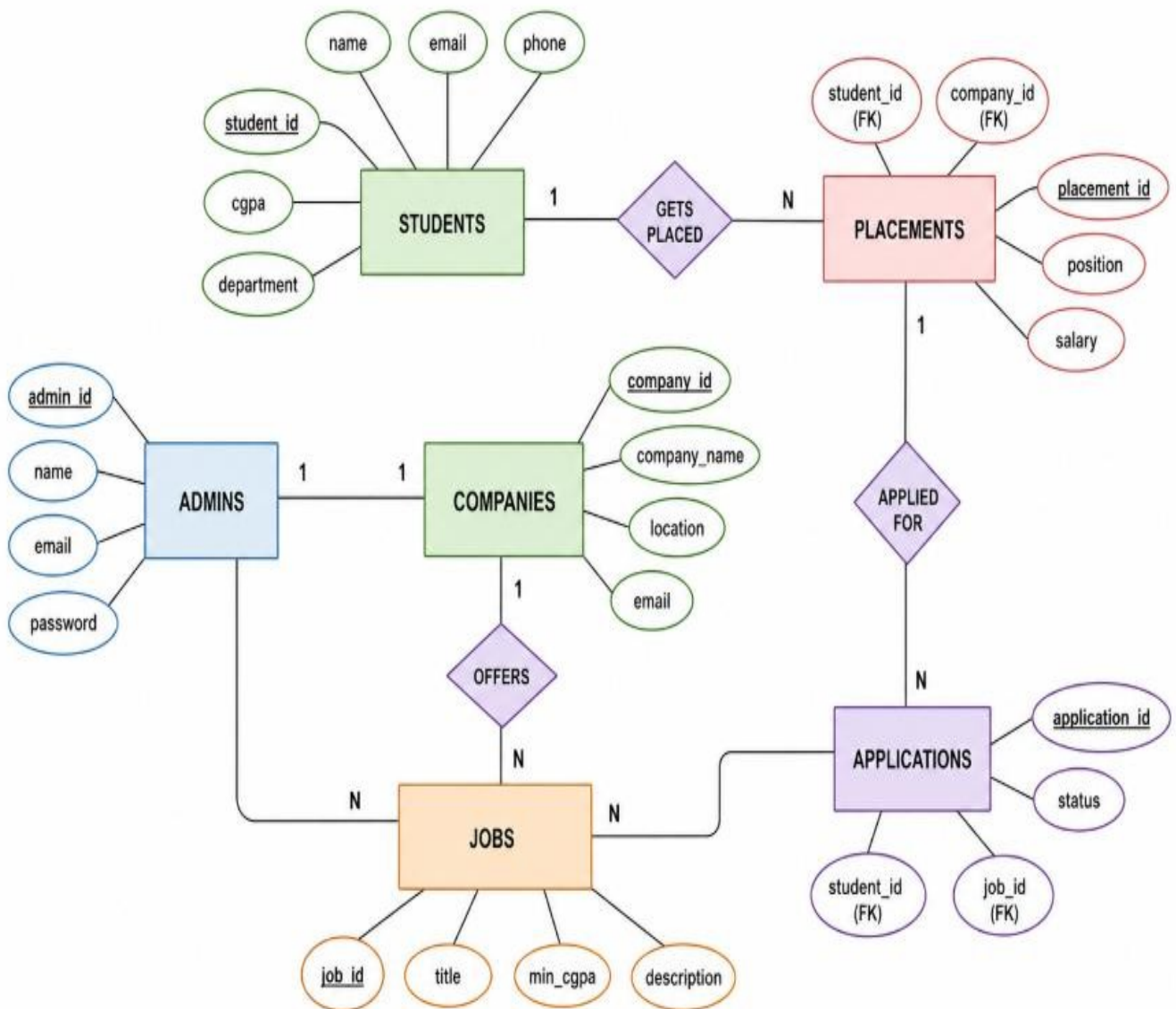


Diamond: Represents relationships entity sets.



Line: links attributes to entity sets and entity set relationships.

A flow chart visually represents the step-by-step process and decision points within the Campus Placement Management System software application.



7.3 UML DIAGRAM

A UML diagram is a diagram based on the UML (Unified Modeling Language) with the purpose of visually representing a system along with its main actors, roles, actions, artifacts or classes, in order to better understand, alter, maintain, or document information about the system.

UML (Unified Modeling Language) diagrams provide a standardized notation for visualizing the structure, behavior, and relationships within a software application. Emotion Sens



application:

7.4 DATABASE DESIGN

Introduction

Database Design is a crucial phase in the development of the CareerAI Hub: Campus Placement Management System. It defines how data is organized, stored, and managed within the system. A well-designed database ensures data consistency, minimizes redundancy, improves performance, and supports efficient retrieval of information. The system uses MySQL as the backend database management system to store and manage placement-related data.

The database is designed using a relational model where data is organized into multiple interconnected tables. Relationships between tables are established using Primary Keys and Foreign Keys to maintain data integrity and consistency.



Objectives of Database Design

- 7.4.1 Store student, recruiter, and placement information efficiently.
- 7.4.2 Eliminate data redundancy and duplication.
- 7.4.3 Maintain data integrity and consistency.
- 7.4.4 Support secure and fast data retrieval.
- 7.4.5 Facilitate relationship management between different entities.
- 7.4.6 Enable efficient placement tracking and reporting.

Main Database Tables Student

Table

Stores information related to registered students.

Field Name	Data Type	Description
student_id	INT (PK)	Unique Student ID
name	VARCHAR(100)	Student Name
email	VARCHAR(100)	Email Address
phone	VARCHAR(15)	Contact Number
department	VARCHAR(50)	Branch/Department
cgpa	DECIMAL(3,2)	Academic CGPA
backlogs	INT	Number of Active Backlogs
password	VARCHAR(255)	Login Password



2.Admin Table

Stores administrator details.

Field Name	Data Type	Description
admin_id	INT (PK)	Unique Admin ID
name	VARCHAR(100)	Administrator Name
email	VARCHAR(100)	Email Address
password	VARCHAR(255)	Login Password

3.Company Table

Stores recruiter/company information.

Field Name	Data Type	Description
company_id	INT (PK)	Unique Company ID
company_name	VARCHAR(100)	Company Name
location	VARCHAR(100)	Company Location
email	VARCHAR(100)	Company Email
industry	VARCHAR(100)	Industry Type



4.Placement Drive Table

Stores information about placement drives.

Field Name	Data Type	Description
drive_id	INT (PK)	Unique Drive ID
company_id	INT (FK)	Company Reference
job_role	VARCHAR(100)	Offered Position
package	DECIMAL(10,2)	Salary Package
min_cgpa	DECIMAL(3,2)	Minimum CGPA Required
max_backlogs	INT	Maximum Allowed Backlogs
drive_date	DATE	Placement Drive Date

5.Application Table

Stores student job applications.

Field Name	Data Type	Description
application_id	INT (PK)	Unique Application ID
student_id	INT (FK)	Student Reference
drive_id	INT (FK)	Drive Reference
application_date	DATE	Date of Application
status	VARCHAR(50)	Applied, Shortlisted, Selected, Rejected



6.Notification Table

Stores placement-related notifications.

Field Name	Data Type	Description
notification_id	INT (PK)	Notification ID
title	VARCHAR(100)	Notification Title
message	TEXT	Notification Content
date_posted	DATE	Date of Notification

Relationship Between Tables

- One Admin can manage multiple companies and placement drives.
- One Company can conduct multiple placement drives.
- One Placement Drive can receive applications from many students.
- One Student can apply for multiple placement drives.
- The Application table acts as a bridge between Students and Placement Drives.
- Notifications are generated and managed by administrators for all users.



8. TESTING

Testing is an important phase in the Software Development Life Cycle (SDLC). It is performed to identify errors, verify system functionality, and ensure that the developed software meets the specified requirements. The CareerAI Hub: Campus Placement Management System was thoroughly tested to ensure reliability, security, accuracy, and efficiency. Testing helps improve software quality by detecting defects before deployment and ensuring smooth interaction between students, recruiters, and placement administrators.

The main objectives of testing are:

- To verify that all system functionalities work correctly.
- To identify and eliminate software defects.
- To ensure accurate data processing and storage.
- To validate system security and access control.
- To evaluate system performance under different workloads.
- To ensure user satisfaction and system reliability.

The testing process included System Testing, Black Box Testing, White Box Testing, Unit Testing, Integration Testing, Validation Testing, and Performance Testing.

8.1 SYSTEM TESTING

System Testing is performed after integrating all modules of the application. The complete system is tested as a whole to verify that it satisfies the specified requirements and performs efficiently.

The major areas covered during system testing include:

Functional Testing

Ensures that all system features operate according to user requirements. Functional testing validates student registration, login, recruiter management, placement drive creation, eligibility verification, job application processing, and report generation.

Browser Compatibility Testing

The application was tested on multiple web browsers including Google Chrome, Mozilla Firefox, and Microsoft Edge to ensure consistent functionality and user experience.

Database Testing

Database operations such as data insertion, retrieval, updating, and deletion were tested to verify data integrity and consistency.



Security Testing

Authentication and authorization mechanisms were tested to ensure that only authorized users can access system resources. Password protection and session management were also verified.

Usability Testing

The user interface was evaluated to ensure easy navigation, readability, responsiveness, and overall user satisfaction.

Performance Testing

The system was tested under different workloads to measure response time, processing speed, and system stability.

8.2 BLACK BOX TESTING

Black Box Testing is a software testing technique that evaluates system functionality without examining the internal program structure or source code. Test cases are designed based on system requirements and expected outputs.

In CareerAI Hub, Black Box Testing was performed on:

- Student Registration and Login
- Admin Authentication
- Recruiter Registration
- Placement Drive Creation
- Student Eligibility Verification
- Job Application Submission
- Interview Scheduling
- Notification Generation
- Dashboard Analytics
- Report Generation

The primary objective of Black Box Testing is to ensure that the system behaves correctly from the user's perspective and produces accurate outputs for valid inputs.

Advantages of Black Box Testings

- Does not require programming knowledge.
- Focuses on user requirements.



- Identifies missing functionalities.
- Detects interface-related errors.
- Ensures user satisfaction.

8.3 TEST ITEMS

The test items represent the major components and functionalities of the CareerAI Hub: Campus Placement Management System that were selected for testing. These components were tested to ensure correct functionality, reliability, security, and smooth interaction between different modules of the system.

1.Student Module

The Student Module was tested to verify that students can successfully access and use all placement-related features.

Test Items:

- Student Registration
- Student Login and Logout
- Profile Creation and Management
- Resume Upload
- Job Application Submission
- Application Status Tracking
- Notification Viewing
- Password Reset

2.Recruiter Module

The Recruiter Module was tested to ensure that companies can efficiently manage recruitment activities.

Test Items:

- Company Registration
- Recruiter Login and Authentication
- Company Profile Management
- Placement Drive Creation
- Placement Drive Management



- Eligibility Criteria Definition
- Applicant Review
- Candidate Shortlisting
- Candidate Selection
- Interview Scheduling

3.Administrator Module

The Administrator Module was tested to verify the overall management and monitoring functionalities of the placement system.

Test Items:

- Admin Login and Authentication
- Student Verification
- Student Record Management
- Company Management
- Placement Drive Approval
- Notification Management
- Eligibility Verification
- Placement Monitoring
- Report Generation
- Analytics Dashboard

4.Database Module

The Database Module was tested to ensure accurate storage, retrieval, and maintenance of placement-related information.

Test Items:

- Data Storage
- Data Retrieval
- Data Updating
- Data Deletion
- Data Validation
- Backup and Recovery



- Query Performance

5.Security Module

The Security Module was tested to verify the protection of user accounts and confidential placement data.

Test Items:

- User Authentication
- Role-Based Authorization
- Session Management
- Password Encryption
- Access Control
- Invalid Login Handling
- Session Timeout
- Protection Against Unauthorized Access

Testing Objective

The objective of testing these items was to ensure that all modules of the CareerAI Hub: Campus Placement Management System function correctly, securely, and efficiently. The testing process verified that the system meets functional requirements, maintains data integrity, provides secure access, and delivers a reliable user experience for students, recruiters, and administrators.

8.4 TEST PLAN

The test plan provides a structured approach for testing the CareerAI Hub system. The testing process included:

- Identification of modules to be tested.
- Preparation of test cases and expected outputs.
- Execution of individual module testing.
- Integration testing of interconnected modules.
- Security and performance testing.
- Defect identification and correction.
- Re-testing after modifications.
- Final system validation.



The testing environment consisted of:

- Operating System: Windows 11
- Web Server: XAMPP
- Programming Language: PHP 8.2
- Database: MySQL
- Browser: Google Chrome

8.5 WHITE BOX TESTING

White Box Testing focuses on the internal structure, logic, and source code of the application. Developers analyze program flow, conditions, loops, and database interactions to ensure that all code paths are executed correctly.

The following aspects were tested:

- Conditional Statements
- Loops and Iterations
- Form Validation Logic
- Session Handling
- Database Queries
- Error Handling Procedures
- Input Validation Mechanisms

Benefits of White Box Testing

- Improves code quality.
- Identifies hidden programming errors.
- Verifies logical correctness.
- Enhances system security.
- Ensures complete code coverage.

Result

White Box Testing conducted on the CareerAI Hub system confirmed that the internal logic, database operations, authentication mechanisms, and application workflows functioned correctly. All critical code paths were executed successfully, validation rules operated as expected, and database interactions produced accurate results. The testing process helped improve code quality, security, and system reliability, ensuring the successful operation of the placement management system.



8.5 Level of Testing

UNIT TESTING

Unit Testing focuses on testing individual modules independently. Each module was tested separately before integration.

Modules tested include:

- Student Registration Module
- Login Module
- Profile Management Module
- Recruiter Module
- Placement Drive Module
- Application Module
- Notification Module
- Report Module

Objectives of Unit Testing

- Verify module functionality.
- Detect coding errors early.
- Ensure correct input-output processing.
- Validate business logic implementation.

INTEGRATION TESTING

Integration Testing verifies the interaction between multiple modules and ensures proper communication among system components.

The following integrations were tested:

- Frontend and Backend Integration
- PHP and MySQL Database Integration
- Student and Application Modules
- Recruiter and Placement Drive Modules
- Dashboard and Analytics Modules
- Notification and User Modules



Integration Strategies

Top-Down Integration

Testing begins from higher-level modules and gradually integrates lower-level modules.

Bottom-Up Integration

Testing begins with lower-level modules and gradually integrates higher-level modules.

Benefits of Integration Testing

- Detects interface defects.
- Verifies data flow between modules.
- Ensures smooth communication among components.

VALIDATION TESTING

Validation Testing ensures that the developed software satisfies user requirements and performs according to expectations.

Alpha Testing

Alpha Testing was conducted internally by developers and project supervisors. The objective was to identify defects before releasing the system to actual users.

Beta Testing

Beta Testing was conducted by selected students and placement coordinators. Their feedback helped improve system functionality, usability, and performance.

Benefits of Validation Testing

- Improves user satisfaction.
- Ensures requirement compliance.
- Enhances software quality.
- Reduces post-deployment issues.

PERFORMANCE TESTING

Performance Testing was conducted to evaluate the speed, responsiveness, stability, and efficiency of the CareerAI Hub: Campus Placement Management System under different operating conditions. The objective of performance testing is to ensure that the system performs effectively when handling multiple users, large volumes of data, and various placement-related operations. Response Time Testing

Measures the time required for the system to process requests and display results.



Load Testing

Load Testing evaluates system performance when multiple users access the application simultaneously. The system was tested with concurrent student, recruiter, and administrator activities to ensure stable operation.

Activities Tested:

- Multiple student logins
- Simultaneous job applications
- Concurrent recruiter access
- Multiple report requests

Objectives:

- Measure system performance under normal workload.
- Ensure consistent response times.
- Verify system reliability during placement drives.

Stress Testing

Stress Testing determines how the system behaves under extreme workload conditions beyond normal operating capacity. The objective is to identify the breaking point of the system and evaluate its recovery capabilities.

Activities Tested:

- Large numbers of simultaneous users
- Bulk job application submissions
- Heavy database transactions
- Continuous report generation requests

Objective:

- Evaluate system stability under peak load.
- Identify performance bottlenecks.
- Ensure graceful recovery from failures.

Database Performance Testing

Database Performance Testing measures the efficiency of database operations, including data insertion, retrieval, updating, and deletion. Since the system handles large amounts of student and recruiter data, database performance is critical.



Activities Tested:

- Student record retrieval
- Recruiter data access
- Placement drive information retrieval
- Application status updates
- Report generation queries

Objective:

- Optimize database query execution.
- Reduce data retrieval time.
- Ensure efficient transaction processing.

Scalability Testing

Ensures that the system can support future growth in the number of users and records.

Results

- Fast retrieval of student and company data.
- Efficient application processing.
- Stable operation under concurrent user access.
- Quick generation of reports and analytics.

8.6 ITEM PASS/FAIL CRITERIA

The pass/fail criteria were established based on the expected behavior and functional requirements of the CareerAI Hub: Campus Placement Management System. Each test item was evaluated by comparing the actual output produced by the system with the expected output defined in the test cases.

Pass Criteria

A test case is considered **Passed** when:

- The system produces the expected output for valid input data.
- Student registration, login, and profile management functions operate correctly.
- Recruiters can successfully create and manage placement drives.
- Administrators can manage students, companies, and placement activities without errors.



- Eligibility verification is performed accurately based on predefined criteria.
- Job applications are submitted and stored successfully in the database.
- Notifications and alerts are generated correctly.
- Reports and analytics are generated accurately.
- Database operations such as insertion, retrieval, updating, and deletion execute successfully.
- Security features including authentication, authorization, and session management function properly.
- The system responds within acceptable performance limits.
- No critical errors, crashes, or data inconsistencies occur during execution.

Fail Criteria

A test case is considered **Failed** when:

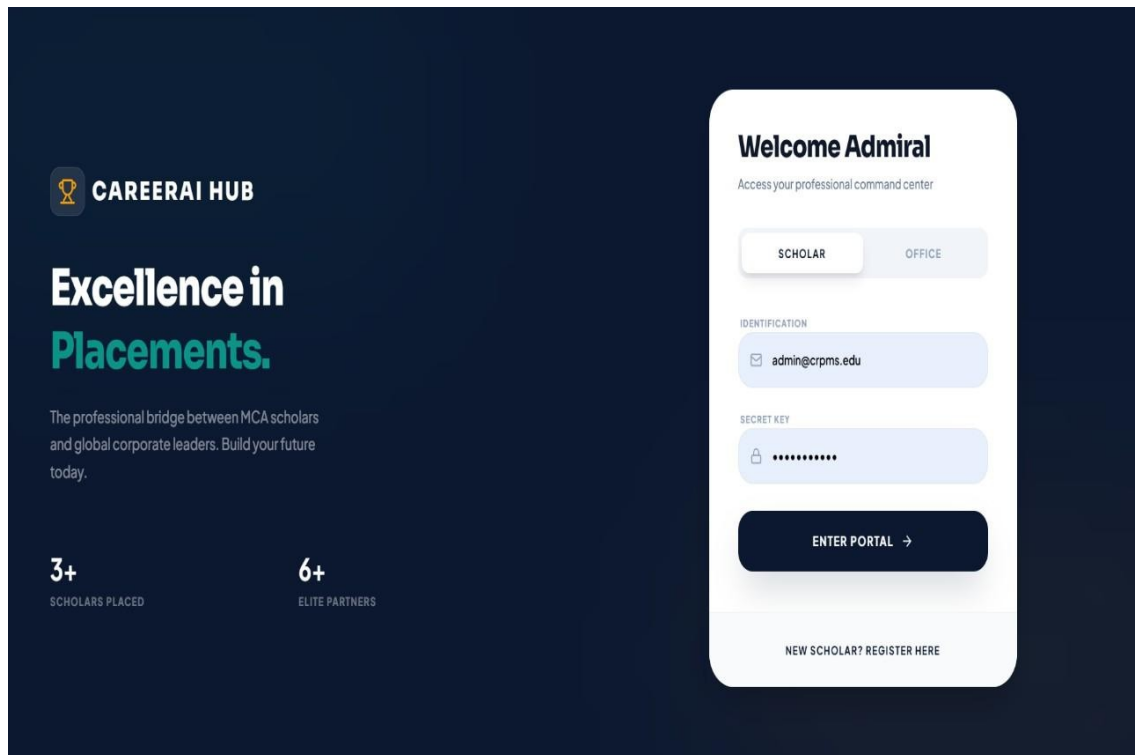
- The actual output differs from the expected output.
- Invalid data is accepted without proper validation.
- User authentication or authorization mechanisms fail.
- Student, recruiter, or administrator operations do not function as intended.
- Database transactions produce incorrect or incomplete results.
- Job applications are not processed correctly.
- Reports or analytics contain inaccurate information.
- Notifications are not delivered or displayed properly.
- System crashes, hangs, or generates unexpected errors.
- Security vulnerabilities allow unauthorized access.
- Response time exceeds acceptable limits.
- Data loss, duplication, or inconsistency is observed.

Evaluation Method

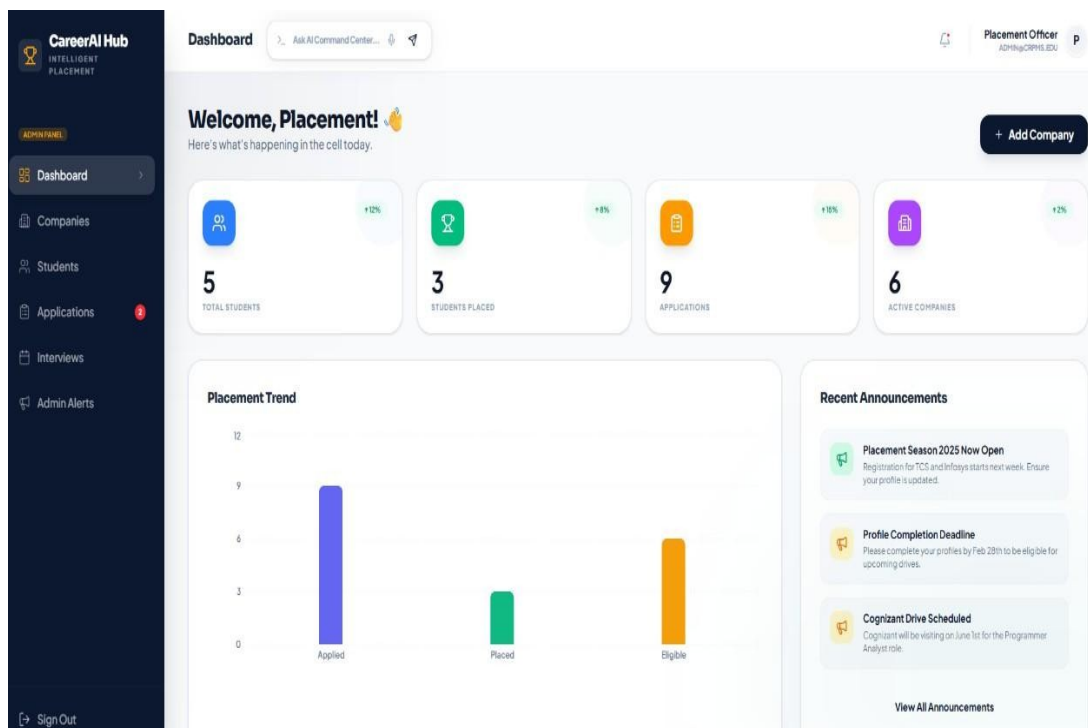
Each test case was executed using predefined input values and expected results. The obtained results were recorded and compared with the expected outcomes. Test cases that met all specified requirements were marked as **Pass**, while those that failed to meet one or more requirements were marked as **Fail** and subjected to corrective actions and re-testing.



Output Screen



Dashboard





Companies List

CareerAI Hub
INTELLIGENT
PLACEMENT

ADMIN PANEL

[Dashboard](#)

[Companies](#)

[Students](#)

[Applications](#)

[Interviews](#)

[Admin Alerts](#)

[Sign Out](#)

Companies

Ask AI Command Center...

Placement Officer
ADMIN@CPSHS.EDU

Corporate Partners

Manage recruitment drives and eligibility

+ Add Company

T

TCS

SYSTEM ENGINEER 3.5 LPA

DRIVE DATE
15 May, 2025

I

Infosys

SYSTEMS ENGINEER 3.6 LPA

DRIVE DATE
20 May, 2025

W

Wipro

PROJECT ENGINEER 3.5 LPA

DRIVE DATE
25 May, 2025

C

Cognizant

PROGRAMMER ANALYST 4.0 LPA

DRIVE DATE
01 Jun, 2025

C

Contrex

DATA ANALYST 5.5 LPA

DRIVE DATE
23 Apr, 2026

G

Google

DRIVE DATE

Students Database

CareerAI Hub
INTELLIGENT
PLACEMENT

ADMIN PANEL

[Dashboard](#)

[Companies](#)

[Students](#)

[Applications](#)

[Interviews](#)

[Admin Alerts](#)

[Sign Out](#)

Students

Ask AI Command Center...

Placement Officer
ADMIN@CPSHS.EDU

Student Database

Total Students: 5

SCHOLAR	PROGRAM	CGPA	STATUS
R Rahul Kumar MCA2301	B.Sc Computer Science MCA	8.2	TAKE INTERVIEW PLACED
A Arjun Singh MCA2303	B.Sc Mathematics MCA	6.9	TAKE INTERVIEW SEARCHING
T Tarun Kumar 24FO1267	BCA MCA	9	TAKE INTERVIEW PLACED
A Adarsh 24FO1269	MCA MCA	92	TAKE INTERVIEW PLACED
T Tarun 24FO127	MCA MCA	90	TAKE INTERVIEW SEARCHING



Manage Application

 CareerAI Hub
INTELLIGENT
PLACEMENT

ADMIN PANEL

Dashboard

Companies

Students

Applications 1

Interviews

Admin Alerts

Sign Out

Applications

Ask AI Command Center...

Placement Officer
ADMIN@CPSYS.EDU

Manage Applications

Total: 9

STUDENT	COMPANY & ROLE	STATUS	ACTIONS
R Rahul Kumar MCA2301	Infosys Systems Engineer	SELECTED	DECISION MADE
R Rahul Kumar MCA2301	Cognizant Programmer Analyst	SELECTED	DECISION MADE
T Tarun Kumar 24PO1267	Wipro Project Engineer	SELECTED	DECISION MADE
A Adarsh 24PO1219	Contrex Data Analyst	SELECTED	DECISION MADE
A Adarsh 24PO1219	Wipro Project Engineer	SELECTED	DECISION MADE
A Adarsh 24PO1219	TCS System Engineer	SELECTED	DECISION MADE
A Adarsh 24PO1219	Infosys Systems Engineer	REJECTED	DECISION MADE
A Adarsh 24PO1219	Google Software Engineer Intern	PENDING REVIEW	

Interviews

 CareerAI Hub
INTELLIGENT
PLACEMENT

ADMIN PANEL

Dashboard

Companies

Students

Applications

Interviews 1

Admin Alerts

Sign Out

Interviews

Ask AI Command Center...

Placement Officer
ADMIN@CPSYS.EDU

Upcoming Interviews

+ Schedule Interview

 Adarsh
TCS - System Engineer

 23 Apr, 2026
10:00 AM



Admin Alerts

CareerAI Hub
INTELLIGENT
PLACEMENT

ADMIN PANEL

Dashboard

Companies

Students

Applications

Interviews

Admin Alerts

Sign Out

Announcements

Ask AI Command Center...

Placement Officer
ADARSH@PHS.EDU

Platform Announcements

New Announcement

SUCCESS

23 Apr, 2024 — 18:40

Placement Season 2025 Now Open

Registration for TCS and Infosys starts next week. Ensure your profile is updated.

WARNING

23 Apr, 2024 — 18:40

Profile Completion Deadline

Please complete your profiles by Feb 28th to be eligible for upcoming drives.

INFO

23 Apr, 2024 — 18:40

Cognizant Drive Scheduled

Cognizant will be visiting on June 1st for the Programmer Analyst role.

User Dashboard

CareerAI Hub
INTELLIGENT
PLACEMENT

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

Dashboard

Ask AI Command Center...

Adarsh
ADARSH@PHS.EDU

Welcome, Adarsh!

Here's what's happening in the cell today.

92

CURRENT CGPA

6

ELIGIBLE COMPANIES

6

APPLICATIONS SENT

Placed

PLACEMENT STATUS

Placement Trend

Category	Value
Applied	9
Placed	3
Eligible	6

Recent Announcements

Placement Season 2025 Now Open

Registration for TCS and Infosys starts next week. Ensure your profile is updated.

Profile Completion Deadline

Please complete your profiles by Feb 28th to be eligible for upcoming drives.

Cognizant Drive Scheduled

Cognizant will be visiting on June 1st for the Programmer Analyst role.

View All Announcements



Apply For Job

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

Jobs

Ask AI Command Center...

Adarsh
ADARSH@GMAIL.COM

Search roles or companies...

Opportunity Hub

T

TCS

SYSTEM ENGINEER

PACKAGE

3.5 LPA

LOCATION

Chennai

DRIVE DATE

15 May

Eligibility Criteria: CGPA 9+

AI Match Score

ANALYZE MATCH

Application Sent

I

Infosys

SYSTEMS ENGINEER

PACKAGE

3.6 LPA

LOCATION

Bangalore

DRIVE DATE

20 May

Eligibility Criteria: CGPA 7.5+

AI Match Score

ANALYZE MATCH

Application Sent

W

Wipro

PROJECT ENGINEER

PACKAGE

3.5 LPA

LOCATION

Hyderabad

DRIVE DATE

25 May

Eligibility Criteria: CGPA 6+

AI Match Score

ANALYZE MATCH

Application Sent

C

Cognizant

PROGRAMMER ANALYST

PACKAGE

4.0 LPA

LOCATION

Chennai

DRIVE DATE

01 Jun

Eligibility Criteria: CGPA 9+

AI Match Score

ANALYZE MATCH

Application Sent

C

Contrex

DATA ANALYST

PACKAGE

5.5 LPA

LOCATION

Mumbai

DRIVE DATE

23 Apr

Eligibility Criteria: CGPA 7.5+

AI Match Score

ANALYZE MATCH

Application Sent

G

Google

SOFTWARE ENGINEER INTERN

PACKAGE

1.5 Lakhs per month

LOCATION

Hyderabad

DRIVE DATE

23 Apr

Eligibility Criteria: CGPA 6+

AI Match Score

ANALYZE MATCH

Application Sent

Interview Mock

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

Mock-Interview

Ask AI Command Center...

Adarsh
ADARSH@GMAIL.COM

AI MOCK INTERVIEW

Practice your technical skills with our high-fidelity AI interviewer.

T

TCS

System Engineer

I

Infosys

Systems Engineer

W

Wipro

Project Engineer

C

Cognizant

Programmer Analyst

ENTER INTERVIEW ROOM



Resume Preparation

INTELLIGENT PLACEMENT

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

Resume

Ask AI Command Center...

Adarsh

ADARSH@GMAIL.COM

A

INTERACTIVE RESUME BUILDER

Build your professional profile for recruiters

Preview Resume

Cancel

Save Professional Profile

Personal & Contact Info

PHONE NUMBER

9220343178

COLLEGE / INSTITUTION

Dept of Computer Applications

DEPARTMENT

MCA

ABOUT YOU (SUMMARY)

My name is adarsh am intrested in Data analyst

Academic Details

UG DEGREE

MCA

UG CGPA

92

10TH %

95

12TH %

94

BACKLOGS

0

BATCH (PASS OUT)

2025

SKILLS (COMMA SEPARATED)

React ,Python,Java,SQL

Placement Inbox

INTELLIGENT PLACEMENT

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

Inbox

Ask AI Command Center...

Adarsh

ADARSH@GMAIL.COM

A

Placement Inbox

Adarsh'S Mailbox

Update on your application for Infosys

From: Placement Cell • 15 Jun, 20:22

Dear Adarsh, Thank you for your interest in the Systems Engineer role at Infosys. Unfortunately, we will not be moving forward with your application at this time. We wish you the best in your future endeavors.

Congratulations! You are selected at TCS

From: Placement Cell • 12 Jun, 13:18

Dear Adarsh, We are pleased to inform you that you have been SELECTED for the System Engineer role at TCS with a package of 3.5 LPA. Please check the portal for next steps.

Interview Call: TCS

From: Placement Cell • 23 Apr, 20:43

Dear Adarsh,

Congratulations! You have been shortlisted for an interview with TCS for the System Engineer position.

Schedule Details:

Date: 2026-04-23

Time: 10:00 AM

Please be prepared and available at the scheduled time.

Best of luck,

Placement Cell



Applications

CareerAI Hub
INTELLIGENT
PLACEMENT

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

My Apps

Ask AI Command Center...

Adarsh
ADARSH@CPAL.COM

Application History

COMPANY	APPLIED DATE	STATUS	NOTES
 Contrex DATA ANALYST	23 Apr, 2026	SELECTED	Awaiting review
 Wipro PROJECT ENGINEER	23 Apr, 2026	SELECTED	Awaiting review
 TCS SYSTEM ENGINEER	12 Jun, 2026	SELECTED	Awaiting review
 Infosys SYSTEMS ENGINEER	12 Jun, 2026	REJECTED	Better Luck Next Time
 Google SOFTWARE ENGINEER INTERN	12 Jun, 2026	UNDER REVIEW	Awaiting review
 Cognizant PROGRAMMER ANALYST	12 Jun, 2026	UNDER REVIEW	Awaiting review

Profile

CareerAI Hub
INTELLIGENT
PLACEMENT

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

Profile

Ask AI Command Center...

Adarsh
ADARSH@CPAL.COM

Adarsh
24PO1219

PLACED

Academic Profile

US CGPA
92

10TH PERCENTAGE
95%

12TH PERCENTAGE
94%

ACTIVE BACKLOGS
0

BATCH
2025

DEGREE
MCA

Contact Details

adarsh@gmail.com

Dept of Computer Applications

Resume Ready?

Keep your professional files updated for faster applications.

UPDATE RESUME

About Me

My name is adarsh am intrested in Data analyst

React Python Java SQL

58



9.1 SYSTEM IMPLEMENTATION CODING

Introduction

System Implementation Coding is the process of translating the system design into executable program code. It is one of the most important phases in software development because the actual functionality of the application is built during this stage. In the CareerAI Hub: Campus Placement Management System, coding was performed using modern web technologies such as HTML5, CSS3, JavaScript (ES6), Tailwind CSS, PHP 8.2, MySQL, Node.js, and Express.js. The coding process focused on developing a secure, efficient, scalable, and user-friendly system that automates placement activities such as student registration, recruiter management, placement drive scheduling, application tracking, eligibility verification, and report generation. A modular programming approach was adopted to improve maintainability and facilitate future enhancements.

Objectives of System Implementation Coding

- The main objectives of coding are:
- To convert system design into a working application.
- To automate campus placement operations.
- To ensure secure data processing and storage.
- To provide a responsive and user-friendly interface.
- To minimize manual work and human errors.
- To maintain data consistency and integrity.
- To support future modifications and upgrades.

Frontend Implementation

The frontend of CareerAI Hub was developed using HTML5, CSS3, JavaScript (ES6), and Tailwind CSS. These technologies were used to create responsive, interactive, and visually appealing user interfaces.

Frontend Functionalities

Student Interface

- Student Registration
- Student Login
- Profile Management
- Resume Upload
- Placement Drive Viewing
- Job Application Submission
- Application Status Tracking
- Notifications Display

Recruiter Interface

- Recruiter Login
- Company Profile Management
- Placement Drive Creation
- Candidate Review
- Selection Status Updates



Administrator Interface

- Student Verification
- Recruiter Management
- Placement Drive Management
- Notification Broadcasting
- Dashboard Analytics
- Report Generation

Benefits of Frontend Coding

- Improved user experience.
- Responsive design for multiple devices.
- Easy navigation and accessibility.
- Faster interaction with the system.

Backend Implementation

The backend was implemented using PHP 8.2, Node.js, and Express.js. The backend handles all server-side operations and business logic.

Backend Functionalities

User Authentication

- Registration Validation
- Login Verification
- Session Management
- Password Security

Placement Management

- Placement Drive Creation
- Eligibility Verification
- Application Processing
- Interview Scheduling

Notification Management

- Placement Announcements
- Schedule Updates
- Selection Notifications

Report Generation

- Student Reports
- Placement Statistics
- Company Reports
- Placement Analytics

Benefits of Backend Coding

- Efficient processing of user requests.
- Improved application security.



- Better system performance.
- Centralized business logic implementation.

Database Implementation

The database was developed using MySQL and managed through phpMyAdmin. The database stores all information related to students, recruiters, placement drives, applications, and notifications.

Major Database Tables Student Table

Stores student details such as:

- Student ID
- Name
- Email
- Department
- CGPA
- Backlogs

Recruiter Table

Stores recruiter information such as:

- Company Name
- Industry Type
- Contact Information
- Eligibility Criteria

Placement Drive Table

Stores:

- Drive Details
- Job Roles
- Package Information
- Interview Dates

Application Table

Stores:

- Student Applications
- Application Status
- Selection Results

Notification Table

Stores:

- Announcements
- Placement Updates
- Schedule Information



Database Operations

The following SQL operations were implemented:

- INSERT
- SELECT
- UPDATE
- DELETE
- JOIN Operations
- Aggregate Queries

Coding Methodology

The following coding practices were adopted:

Modular Programming

The application was divided into separate modules such as:

- Student Module
- Recruiter Module
- Placement Drive Module
- Notification Module
- Report Module

This improves code maintainability and reduces complexity.

Reusability

The CareerAI Hub: Campus Placement Management System was developed with a focus on code reusability to improve efficiency and maintainability. Reusable functions, classes, and components were created to minimize code duplication and simplify future enhancements. Common functionalities such as user authentication, database connectivity, input validation, session management, and notification handling were implemented as reusable modules that can be utilized across different parts of the system. This approach reduces development time, improves consistency, and facilitates easier maintenance and scalability.

Error Handling

Effective error handling mechanisms were implemented throughout the application to ensure system reliability and provide a smooth user experience. The system detects, manages, and responds appropriately to unexpected situations without causing application failure.

Code Documentation

Proper code documentation was maintained throughout the development process to improve readability, maintainability, and future enhancements. Comments were added to explain complex logic, functions, classes, and database operations. Documentation helps developers understand the purpose and functionality of different code sections, making future modifications easier and reducing maintenance effort. Well-documented code also supports collaborative development and knowledge transfer among team members.

Security Implementation

Security is one of the most critical aspects of the CareerAI Hub system because it manages sensitive information related to students, recruiters, placement drives, and recruitment results. Multiple security mechanisms were incorporated to ensure confidentiality, integrity, and



availability of data.

Authentication and Authorization

Authentication mechanisms verify the identity of users before granting access to the system. Authorization controls determine which resources and functionalities users can access based on their assigned roles.

Features:

- Secure login system.
- Role-based user access.
- Restricted access to sensitive modules.
- Protection against unauthorized system usage.

Input Validation

All user inputs are validated before processing to ensure that only valid and properly formatted data is accepted by the system.

Validation Checks Include:

- Mandatory field verification.
- Email format validation.
- Password strength verification.
- Numeric and text field validation.
- Prevention of malicious input submissions.

Benefits:

- Maintains data integrity.
- Prevents application errors.
- Enhances system security.

Session Management

Session management is implemented to maintain secure user authentication during application usage. Once authenticated, users are assigned secure sessions that remain active until logout or session expiration.

Features:

- Secure session creation.
- Automatic session timeout.
- Logout functionality.
- Prevention of unauthorized session access.

Benefits:

- Protects user accounts.
- Improves application security.
- Ensures controlled access to resources.

Database Security

Several database security measures were implemented to protect placement-related information from unauthorized access and attacks.

Security Techniques Used:

- Prepared Statements.
- Parameterized Queries.
- Data Validation.



- Access Restrictions.
- Secure Database Connectivity.

Benefits:

- Prevents SQL Injection attacks.
- Protects sensitive records.
- Maintains data consistency and integrity.

Password Protection

User passwords are protected using modern encryption and hashing techniques before being stored in the database.

Security Measures:

- Password hashing algorithms.
- Secure password storage.
- Password verification mechanisms.
- Protection against password theft.

Benefits:

- Enhances account security.
- Prevents unauthorized access.
- Protects user credentials.

Role-based access control ensures that users can only perform authorized actions.

Coding Challenges Faced

During implementation, several challenges were encountered:

- Managing large volumes of placement data.
- Maintaining database consistency.
- Implementing eligibility verification logic.
- Handling concurrent user access.
- Ensuring secure authentication.
- Optimizing application performance.

These challenges were resolved through efficient coding practices, database normalization, and extensive testing.

Advantages of System Implementation Coding

- Automates placement management activities.
- Reduces manual errors and paperwork.
- Provides accurate and reliable information.
- Improves communication among stakeholders.
- Enhances security and data protection.
- Supports scalability and future enhancements.
- Improves overall system efficiency.



CODING REPORT:

1. server.ts – Backend Core Logic

Important backend implementation including Express server setup, authentication, data initialization, API handling and placement management workflows.

server.ts – Key Implementation (Part A)

```
import express from 'express';
import path from 'path';
import { createServer as createViteServer } from 'vite';
import fs from 'fs';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import AdmZip from 'adm-zip';

const app = express();
const PORT = 3000;
const SECRET_KEY = 'crpms_secret_2024';
const DB_PATH = path.join(process.cwd(), 'database.json');

app.use(express.json());

// Initial Data Structure
const initialData =
{ admins: [
  { id: '1', name: 'Placement Officer', email: 'admin@crpms.edu', password:
bcrypt.hashSync('password123', 10) }
],
students: [
  {
    id: 's1',
    name: 'Rahul Kumar',
    roll_no: 'MCA2301',
    email: 'rahul@student.edu',
    password: bcrypt.hashSync('password123', 10),
    phone: '9876543210',
    college: 'Dept of Computer Applications',
    department: 'MCA',
    pct_10: 88,
    pct_12: 82,
    ug_cgpa: 8.2,
    ug_degree: 'B.Sc Computer Science',
    year_pass: 2025,
```



```
backlogs: 0,
skills: 'React, Node.js, SQL',
about: 'Enthusiastic developer.',
is_placed: false,
is_active: true
},
{
  id: 's2',
  name: 'Arjun Singh',
  roll_no: 'MCA2303',
  email: 'arjun@student.edu',
  password: bcrypt.hashSync('password123', 10),
  phone: '9876543212',
  college: 'Dept of Computer Applications',
  department: 'MCA',
  pct_10: 72,
  pct_12: 68,
  ug_cgpa: 6.9,
  ug_degree: 'B.Sc Mathematics',
  year_pass: 2025,
  backlogs: 1,
  skills: 'Python, Java',
  about: 'Passionate about coding.',
  is_placed: false,
  is_active: true
}
],
companies: [
  { id: 'c1', name: 'TCS', role: 'System Engineer', package: '3.5 LPA', min_cgpa: 6.0,
    min_pct_10: 60, min_pct_12: 60, allow_backlog: false, drive_date: '2025-05-15', location:
    'Chennai', job_type: 'Full Time', status: 'active', color: '#0d1b3e', letter: 'T' },
  { id: 'c2', name: 'Infosys', role: 'Systems Engineer', package: '3.6 LPA', min_cgpa: 6.5,
    min_pct_10: 65, min_pct_12: 65, allow_backlog: false, drive_date: '2025-05-20', location:
    'Bangalore', job_type: 'Full Time', status: 'active', color: '#007cc3', letter: 'I' },
  { id: 'c3', name: 'Wipro', role: 'Project Engineer', package: '3.5 LPA', min_cgpa: 6.0,
    min_pct_10: 60, min_pct_12: 60, allow_backlog: true, drive_date: '2025-05-25', location:
    'Hyderabad', job_type: 'Full Time', status: 'active', color: '#2e3192', letter: 'W' },
  { id: 'c4', name: 'Cognizant', role: 'Programmer Analyst', package: '4.0 LPA', min_cgpa:
    7.0, min_pct_10: 70, min_pct_12: 65, allow_backlog: false, drive_date: '2025-06-01',
    location: 'Chennai', job_type: 'Full Time', status: 'active', color: '#0033a0', letter: 'C' }
],
applications: [],
interviews: [],
announcements: [
```



```
{ id: '1', title: 'Placement Season 2025 Now Open', body: 'Registration for TCS and Infosys
starts next week. Ensure your profile is updated.', type: 'success', is_active: true, created_at:
new Date().toISOString() },
{ id: '2', title: 'Profile Completion Deadline', body: 'Please complete your profiles by Feb
28th to be eligible for upcoming drives.', type: 'warning', is_active: true, created_at: new
Date().toISOString() },
{ id: '3', title: 'Cognizant Drive Scheduled', body: 'Cognizant will be visiting on June 1st
for the Programmer Analyst role.', type: 'info', is_active: true, created_at: new
Date().toISOString() }
],
notifications: [
{ id: 'n1', title: 'Welcome to CRPMS', message: 'Your account has been successfully
created. Explore active jobs now!', type: 'ok', is_read: false, created_at: new
Date().toISOString() }
],
mails: []
};
```

```
// Ensure database exists
if(!fs.existsSync(DB_PATH)) {
  fs.writeFileSync(DB_PATH, JSON.stringify(initialData, null, 2));
}

const getDB = () => JSON.parse(fs.readFileSync(DB_PATH, 'utf-8'));
const saveDB = (data: any) => fs.writeFileSync(DB_PATH, JSON.stringify(data, null, 2));
```

```
// API Routes
app.post('/api/auth/login', (req, res) =>
{ const { email, password, role } =
req.body; const db = getDB();
const users = role === 'admin' ? db.admins : db.students;
const user = users.find((u: any) => u.email === email);

if(user && bcrypt.compareSync(password, user.password))
{ const token = jwt.sign({ id: user.id, role }, SECRET_KEY);
const { password: _, ...userWithoutPassword } = user;
res.json({ token, user: { ...userWithoutPassword, role } });
} else {
res.status(401).json({ error: 'Invalid credentials' });
}
});
```

```
app.post('/api/auth/register', (req, res) =>
{ const db = getDB();
```



```
const userData = req.body;

if (db.students.find((s: any) => s.email === userData.email || s.roll_no ===
userData.roll_no)) {
  return res.status(400).json({ error: 'Email or Roll No already exists' });
}

const newUser = {
  ...userData,
  id: 's' + Date.now(),
  password: bcrypt.hashSync(userData.password, 10),
  is_placed: false,
  is_active: true
};

db.students.push(newUser);
saveDB(db);

const token = jwt.sign({ id: newUser.id, role: 'student' }, SECRET_KEY);
const { password: _, ...userWithoutPassword } = newUser;
res.json({ token, user: { ...userWithoutPassword, role: 'student' } });
});

// Companies
app.get('/api/companies', (req, res) =>
  { const db = getDB();
    res.json(db.companies);
  });

app.post('/api/companies', (req, res) =>
  { const db = getDB();
    const newCo = { id: 'c' + Date.now(), ...req.body, letter: req.body.name.charAt(0) };
    db.companies.push(newCo);
    saveDB(db);
    res.json(newCo);
  });

app.patch('/api/companies/:id', (req, res) =>
  { const { id } = req.params;
    const db = getDB();
    const idx = db.companies.findIndex((c: any) => c.id === id);
    if (idx === -1) return res.status(404).json({ error: 'Not found' });

    db.companies[idx] = { ...db.companies[idx], ...req.body };
  });
```



```
saveDB(db);
res.json(db.companies[idx]);
});

// Applications
app.get('/api/applications', (req, res) =>
{ const db = getDB();
  const apps = db.applications.map((app: any) => ({
    ...app,
    student: db.students.find((s: any) => s.id === app.student_id),
    company: db.companies.find((c: any) => c.id === app.company_id),
  }));
  res.json(apps);
});

app.post('/api/applications', (req, res) =>
{ const { student_id, company_id } =
  req.body; const db = getDB();

  // Check eligibility
  const student = db.students.find((s: any) => s.id === student_id);
  const company = db.companies.find((c: any) => c.id === company_id);

  if (!student || !company) return res.status(404).json({ error: 'Data not found' });

  const reasons = [];
  if (student.ug_cgpa < company.min_cgpa) reasons.push('CGPA too low');
  if (student.pct_10 < company.min_pct_10) reasons.push('10th % too low');
  if (student.pct_12 < company.min_pct_12) reasons.push('12th % too low');
  if (!company.allow_backlog && student.backlogs > 0) reasons.push('Active backlogs');

  if (reasons.length > 0) {
    return res.status(400).json({ error: 'Not eligible', reasons });
  }

  const existing = db.applications.find((a: any) => a.student_id === student_id &&
a.company_id === company_id);
  if (existing) return res.status(400).json({ error: 'Already applied' });

  const newApp =
    { id: 'a' +
      Date.now(),
      student_id,
      company_id,
      status: 'Under Review',
```



```
    applied_at: new Date().toISOString().split('T')[0]
  };

  db.applications.push(newApp);

  // Create notification for student
  db.notifications.unshift({
    id: 'n' + Date.now(),
    student_id,
    title: 'Application Sent',
    message: `You applied for the ${company.role} role at ${company.name}.`,
    type: 'blue',
    is_read: false,
    created_at: new Date().toISOString()
  });

  // Create notification for admin
  db.notifications.unshift({
    id: 'na' + Date.now(),
    role: 'admin',
    title: 'New Application',
    message: `${student.name} applied for ${company.name}.`,
    type: 'info',
    is_read: false,
    created_at: new Date().toISOString()
  });

  saveDB(db);
  res.json(newApp);
});

app.patch('/api/applications/:id', (req, res) =>
  { const { id } = req.params;
    const { status, admin_note } = req.body;
    const db = getDB();
    const appIdx = db.applications.findIndex((a: any) => a.id === id);
    if (appIdx === -1) return res.status(404).json({ error: 'Not found' });

    const application = db.applications[appIdx];
    const student = db.students.find((s: any) => s.id === application.student_id);
    const company = db.companies.find((c: any) => c.id === application.company_id);

    db.applications[appIdx] = { ...application, status, admin_note, decided_at: new
    Date().toISOString() };
```



```
if(status === 'Selected') {
  const sIdx = db.students.findIndex((s: any) => s.id === application.student_id);
  if (sIdx !== -1) db.students[sIdx].is_placed = true;
}

// Create "Email" in internal mail system
const mailSubject = status === 'Selected' ? `Congratulations! You are selected at
${company.name}` : `Update on your application for ${company.name}`;
const mailBody = status === 'Selected'
  ? `Dear ${student.name}, We are pleased to inform you that you have been SELECTED
for the ${company.role} role at ${company.name} with a package of ${company.package}.
Please check the portal for next steps.`
  : `Dear ${student.name}, Thank you for your interest in the ${company.role} role at
${company.name}. Unfortunately, we will not be moving forward with your application at
this time. We wish you the best in your future endeavors.`;

db.mails.unshift({
  id: 'm' + Date.now(),
  student_id: student.id,
  subject: mailSubject,
  body: mailBody,
  from: 'Placement Cell',
  date: new Date().toISOString(),
  is_read: false
});

// Also create a notification for student
db.notifications.unshift({
  id: 'n' + Date.now(),
  student_id: student.id,
  title: status === 'Selected' ? `~a~🎉Offer Received!` : 'Application Update',
  message: mailSubject,
  type: status === 'Selected' ? 'ok' : 'warn',
  is_read: false,
  created_at: new Date().toISOString()
});

saveDB(db);
res.json(db.applications[appIdx]);
});

// Students
app.get('/api/students', (req, res) => {
```



```
const db = getDB();
res.json(db.students.map((s: any) =>
  { const { password: _, ...rest } = s;
  return rest;
}));
});

app.get('/api/students/:id', (req, res) =>
  { const { id } = req.params;
  const db = getDB();
  const student = db.students.find((s: any) => s.id === id);
  if (!student) return res.status(404).json({ error: 'Student not found' });
  const { password: _, ...studentWithoutPassword } = student;
  res.json(studentWithoutPassword);
});

app.patch('/api/students/:id', (req, res) =>
  { const { id } = req.params;
  const db = getDB();
  const idx = db.students.findIndex((s: any) => s.id === id);
  if (idx === -1) return res.status(404).json({ error: 'Student not found' });

  db.students[idx] = { ...db.students[idx], ...req.body };
  saveDB(db);
  const { password: _, ...studentWithoutPassword } = db.students[idx];
  res.json(studentWithoutPassword);
});

// Interviews
app.get('/api/interviews', (req, res) =>
  { const db = getDB();
  const interviews = db.interviews.map((iv: any) => ({
    ...iv,
    student: db.students.find((s: any) => String(s.id) === String(iv.student_id)),
    company: db.companies.find((c: any) => String(c.id) === String(iv.company_id)),
  }));
  res.json(interviews);
});

app.post('/api/interviews', (req, res) =>
  { const db = getDB();
  const { student_id, company_id, date, time } = req.body;
  const newIv = { id: 'iv' + Date.now(), ...req.body };
  db.interviews.unshift(newIv);
```



```
const company = db.companies.find((c: any) => c.id === company_id);
const student = db.students.find((s: any) => s.id === student_id);
```

```
if (company && student) {
  // Add Notification
  db.notifications.unshift({
    id: 'notif' + Date.now(),
    student_id: student_id,
    title: 'Interview Scheduled!',
    message: `You have an
```

src/App.tsx – Important Components

```
6 bg-teal rounded-full flex items-center justify-center mb-6 shadow-xl shadow-teal/20">
  <CheckCircle2 className="w-8 h-8 text-white" />
</div>
<h4 className="text-2xl font-black text-white uppercase tracking-tight mb-
4">Interview Performance</h4>
  <div className="bg-white/5 border border-white/10 rounded-2xl p-6 text-white/90
text-sm font-medium leading-relaxed max-w-xl text-left whitespace-pre-wrap">
    <div className="markdown-body">
      <ReactMarkdown>{feedback}</ReactMarkdown>
    </div>
  </div>
  <button
    onClick={() => setFeedback(null)}
    className="mt-8 px-8 py-4 bg-teal text-white rounded-xl font-black uppercase
tracking-widest hover:bg-teal2 transition-all shadow-xl shadow-teal/20"
  >
    Continue Interview
  </button>
</motion.div>
)}

{messages.map((m, i) => (
  <motion.div
    key={i}
    initial={{ opacity: 0, x: m.role === 'user' ? 20 : -20 }}
    animate={{ opacity: 1, x: 0 }}
    className={cn("flex", m.role === 'user' ? "justify-end" : "justify-start")}
  >
    <div className={cn(
      "max-w-[80%] p-5 rounded-2xl shadow-sm text-sm font-medium leading-relaxed",
      m.role === 'user' ? "bg-teal text-white rounded-tr-none" : "bg-white text-slate-700
```



```

border border-slate-100 rounded-tl-none"
    >>
    {m.text}
  </div>
</motion.div>
))}
</AnimatePresence>
{loading && (
  <div className="flex justify-start">
    <div className="bg-white p-4 rounded-2xl border border-slate-100 flex items-
center gap-3">
      <div className="flex gap-1">
        <div className="w-1.5 h-1.5 bg-teal-500 rounded-full animate-bounce" />
        <div className="w-1.5 h-1.5 bg-teal-500 rounded-full animate-bounce
[animation-delay:0.2s]" />
        <div className="w-1.5 h-1.5 bg-teal-500 rounded-full animate-bounce
[animation-delay:0.4s]" />
      </div>
      <span className="text-xs text-slate-400 font-bold italic">Interviewer is
typing...</span>
    </div>
  </div>
)}
</div>

<div className="p-6 bg-white border-t border-slate-100">
  <div className="flex gap-4">
    <input
      value={input}
      onChange={e => setInput(e.target.value)}
      onKeyDown={e => e.key === 'Enter' && sendMessage()}
      placeholder="Type your response here..."
      className="flex-1 px-6 py-4 bg-slate-50 rounded-2xl border border-slate-200
outline-none focus:border-teal font-medium transition-all"
    />
    <button
      onClick={sendMessage}
      disabled={loading || !input.trim()}
      className="w-14 h-14 bg-teal text-white rounded-2xl flex items-center justify-center
hover:bg-teal2 transition-all shadow-lg shadow-teal/20 disabled:opacity-50"
    >
      <ArrowRight className="w-6 h-6" />
    </button>
  </div>

```



```
    </div>
  </motion.div>
);
};

const AICommandBar = ({ onAction }: { onAction: (action: any) => void }) =>
  { const [query, setQuery] = React.useState("");
    const [loading, setLoading] = React.useState(false);
    const [isListening, setIsListening] = React.useState(false);

    const recognitionRef = React.useRef<any>(null);

    React.useEffect(() => {
      // @ts-ignore
      const SpeechRecognition = window.SpeechRecognition ||
      window.webkitSpeechRecognition;
      if (SpeechRecognition) {
        recognitionRef.current = new SpeechRecognition();
        recognitionRef.current.continuous = false;
        recognitionRef.current.interimResults = false;
        recognitionRef.current.lang = 'en-US';

        recognitionRef.current.onstart = () =>
          { console.log('Speech recognition
            started');
          };

        recognitionRef.current.onresult = (event: any) =>
          { const transcript = event.results[0][0].transcript;
            console.log('Voice detected:', transcript);
            setQuery(transcript);
            setIsListening(false);
            processCommand(transcript);
          };

        recognitionRef.current.onerror = (event: any) =>
          { console.error('Speech recognition error',
            event.error); setIsListening(false);
            if (event.error === 'not-allowed') {
              alert("Microphone access denied. Please click the 'Lock' icon in your browser address
bar and allow Microphone access for this site.");
            } else if (event.error === 'no-speech')
              { console.warn('No speech detected');
            } else if (event.error === 'network') {
              alert("Network error during voice recognition. Please check your connection.");
            }
          };
      }
    }, []);
  };
}
```



```
    } else {
      console.warn(`Voice recognition warning: ${event.error}`);
    }
  };

  recognitionRef.current.onend = () =>
    { console.log('Speech recognition
      ended'); setIsListening(false);
    };
  }
}, []);

const toggleListening = () =>
{ if (!recognitionRef.current)
{
  alert("Voice search is not supported in this browser. Please use Chrome or Edge.");
  return;
}

try {
  if (isListening)
    { recognitionRef.current.stop
      (); setIsListening(false);
    } else
    { setQuery('
      ');
      recognitionRef.current.start();
      setIsListening(true);
    }
} catch (err: any) {
  console.error('Speech recognition error:', err);
  // Handle the "already started" error gracefully
  if (err?.error === 'aborted' || err?.message?.includes('already started'))
    { setIsListening(true);
  } else
    { setIsListening(fals
      e);
    }
}
};

const processCommand = async (text: string) =>
{ if (!text.trim()) return;
  setLoading(true);
  try {
```

```
const response = await
  ai.models.generateContent({ model: "gemini-3-flash-
    preview",
```



contents: `You are an AI Assistant for the CareerAI Hub.
Translate the student/admin request into a structured JSON action.

Command: "\${text}"

Available Actions:

1. navigate: { "type": "navigate", "page": "dashboard" | "companies" | "students" | "applications" | "interviews" | "announcements" | "jobs" | "my_apps" | "profile" | "resume" }
2. filter_students: { "type": "filter_students", "params": { "is_placed": boolean, "department": string, "cgpa_min": number } }
3. open_modal: { "type": "open_modal", "modal": "interview" | "company" | "announcement", "student_name": string }

Examples:

- "Show unplaced students" -> { "type": "filter_students", "params": { "is_placed": false } }
- "Schedule interview for Adarsh" -> { "type": "open_modal", "modal": "interview", "student_name": "Adarsh" }
- "Go to jobs" -> { "type": "navigate", "page": "jobs" }
- "Open profile" -> { "type": "navigate", "page": "profile" }

Return ONLY valid JSON.`,

```
config: { responseType: "application/json" }
});
```

```
const actionText = response.text;
const cleanJson = actionText.replace(/```json/g, "").replace(/```/g, "").trim();
const action = JSON.parse(cleanJson);
if (action)
{ onAction(action); setQuery("");
} else {
console.error('AI returned empty action');
}
} catch (err) {
console.error('AI Command failed', err);
// Fallback: If AI fails, try a simple keyword search
if (text.toLowerCase().includes('student'))
{ onAction({ type: 'navigate', page: 'students' });
} else if (text.toLowerCase().includes('job') || text.toLowerCase().includes('compan'))
{ onAction({ type: 'navigate', page: 'companies' });
}
} finally
{ setLoading(false);
}
```



```

    }
  };

  const handleSubmit = (e: React.FormEvent) =>
  { e.preventDefault();
    processCommand(query);
  };

  return (
    <form onSubmit={handleSubmit} className="relative group max-w-sm w-full">
      {isListening && (
        <div className="absolute -top-12 left-0 right-0 flex justify-center">
          <div className="bg-red-500 text-white text-[10px] font-black uppercase px-4 py-1.5
rounded-full animate-bounce shadow-lg flex items-center gap-2">
            <Sparkles className="w-3 h-3" /> Listening for your voice...
          </div>
        </div>
      )}
      <div className="absolute inset-0 bg-navy/5 rounded-2xl blur-lg transition-all group-
hover:bg-navy/10" />
        <div className="relative flex items-center bg-white border border-slate-200 rounded-2xl
px-4 h-12 shadow-sm focus-within:ring-4 focus-within:ring-navy/5 focus-within:border-navy
transition-all">
          <Terminal className="w-4 h-4 text-navy/40 mr-3" />
          <input
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            placeholder={isListening ? "Listening..." : "Ask AI Command Center..."}
            className="flex-1 bg-transparent border-none outline-none text-xs font-black
uppercase tracking-tight text-slate-700 placeholder:text-slate-400 placeholder:normal-case"
          />
          <div className="flex items-center gap-1">
            <button
              type="button"
              onClick={toggleListening}
              className={cn(
                "p-2 rounded-lg transition-all",
                isListening ? "bg-red-500 text-white animate-pulse" : "text-slate-400 hover:bg-slate-
50 hover:text-navy"
              )}
              title="Voice Search"
            >
              {isListening ? <MicOff className="w-4 h-4" /> : <Mic className="w-4 h-4" />}
            </button>

```



```

        <button type="submit" disabled={loading} className="p-2 hover:bg-slate-50
rounded-lg transition-all text-navy">
          {loading ? <RefreshCw className="w-4 h-4 animate-spin" /> : <Send
className="w-4 h-4" />}
        </button>
      </div>
    </div>
  </form>
);
};

const SidebarItem = ({ icon: Icon, label, active, onClick, badge }: any) => (
  <button
    onClick={onClick}
    className={cn(
      "w-full flex items-center gap-3 px-4 py-3 rounded-xl transition-all duration-200 group
text-left",
      active
        ? "bg-white/15 text-white shadow-lg"
        : "text-white/60 hover:bg-white/5 hover:text-white"
    )}
  >
    <Icon className={cn("w-5 h-5", active ? "text-gold" : "text-white/40 group-hover:text-
white/80")} />
    <span className="font-medium flex-1">{label}</span>
    {badge > 0 && (
      <span className="bg-red-500 text-white text-[10px] font-bold px-1.5 py-0.5 rounded-
full">
        {badge}
      </span>
    )}
    {active && <ChevronRight className="w-4 h-4 text-white/40" />}
  </button>
);

const ResumePreview = ({ user }: { user: User }) =>
{ return (
  <motion.div initial={{ opacity: 0 }} animate={{ opacity: 1 }} className="bg-white p-12
rounded-3xl border border-slate-200 shadow-2xl max-w-4xl mx-auto font-serif" id="resume-
printable">
    <div className="flex justify-between border-b-2 border-navy pb-8 mb-8">
      <div>
        <h1 className="text-4xl font-black text-navy uppercase tracking-tighter mb-
2">{user.name}</h1>

```



```

    <div className="flex items-center gap-4 text-slate-500 font-bold text-sm uppercase
tracking-widest">
      <span className="flex items-center gap-2"><Mail className="w-4 h-4" />
{user.email}</span>
      <span className="flex items-center gap-2"><MapPin className="w-4 h-4" />
{user.location || 'India'}</span>
    </div>
  </div>
  <div className="text-right">
    <p className="text-sm font-black text-teal uppercase tracking-widest mb-
1">{user.ug_degree}</p>
    <p className="text-[10px] text-slate-400 font-bold
uppercase">{user.department}</p>
  </div>
</div>

<div className="grid grid-cols-3 gap-12">
  <div className="col-span-2 space-y-10">
    <section>
      <h2 className="text-xs font-black text-navy uppercase tracking-[4px] mb-4 border-b
border-slate-100 pb-2">Professional Summary</h2>
      <p className="text-slate-600 leading-relaxed text-sm italic">" {user.about || 'A
dedicated student looking for opportunities.'}"</p>
    </section>

    <section>
      <h2 className="text-xs font-black text-navy uppercase tracking-[4px] mb-4 border-b
border-slate-100 pb-2">Technical Skills</h2>
      <div className="flex flex-wrap gap-2">
        {user.skills?.split(',').map((s, i) => (
          <span key={i} className="px-3 py-1 bg-slate-50 border border-slate-200 rounded
text-[10px] font-bold text-slate-700 uppercase tracking-tight">{s.trim()}</span>
        ))}
      </div>
    </section>
  </div>

  <div className="space-y-10">
    <section>
      <h2 className="text-xs font-black text-navy uppercase tracking-[4px] mb-4 border-b
border-slate-100 pb-2">Academic Score</h2>
      <div className="space-y-4">
        <div className="flex justify-between items-center px-4 py-3 bg-navy text-white
rounded-2xl">

```



```

    <span className="text-[10px] font-bold uppercase">CGPA</span>
    <span className="text-xl font-black">{user.ug_cgpa}</span>
  </div>
  <div className="flex justify-between items-center px-4 py-2 bg-slate-50 rounded-
xl border border-slate-100">
    <span className="text-[9px] font-bold text-slate-400 uppercase">10th %</span>
    <span className="text-sm font-bold text-slate-700">{user.pct_10}%</span>
  </div>
  <div className="flex justify-between items-center px-4 py-2 bg-slate-50 rounded-
xl border border-slate-100">
    <span className="text-[9px] font-bold text-slate-400 uppercase">12th %</span>
    <span className="text-sm font-bold text-slate-700">{user.pct_12}%</span>
  </div>
</div>
</section>

<section>
  <h2 className="text-xs font-black text-navy uppercase tracking-[4px] mb-4 border-
b border-slate-100 pb-2">Education</h2>
  <div>
    <p className="text-sm font-extrabold text-slate-900">{user.year_pass} Batch</p>
    <p className="text-xs text-slate-500 mt-1">{user.college}</p>
  </div>
</section>
</div>
</div>

<div className="mt-12 pt-8 border-t border-slate-100 text-center">
  <p className="text-[9px] text-slate-300 font-bold uppercase tracking-
widest">Generated via CareerAI Hub</p>
</div>
</motion.div>
);
};

const ResumeEditor = ({ user, onSave, onCancel }: { user: User, onSave: (data:
Partial<User>) => void, onCancel: () => void }) => {
  const [showPreview, setShowPreview] = React.useState(false);
  const [formData, setFormData] = React.useState({
    abo

```

3. login.php – Authentication Module

Implements user login, session initialization and credential validation.



login.php – Important Code

```
<?php
// login.php
include 'db_connect.php';
session_start();

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $email = $_POST['email'];
    $password = $_POST['password'];

    $sql = "SELECT id, name, role FROM users WHERE email = '$email' AND password = '$password'";
    $result = $conn->query($sql);

    if ($result->num_rows > 0) {
        $user = $result->fetch_assoc();
        $_SESSION['user_id'] = $user['id'];
        $_SESSION['user_name'] = $user['name'];
        $_SESSION['role'] = $user['role'];
        header("Location: index.php");
    } else {
        $error = "Invalid Email or Password!";
    }
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Login - Placement Cell</title>
    <link rel="stylesheet" href="style.css">
    <style>
        .login-box { max-width: 400px; margin: 100px auto; background: white; padding: 40px;
border-radius: 15px; box-shadow: 0 10px 25px rgba(0,0,0,0.1); }
        input { width: 100%; padding: 12px; margin: 10px 0; border: 1px solid #ddd; border-
radius: 8px; box-sizing: border-box; }
        button { width: 100%; padding: 12px; background: #3498db; color: white; border: none;
border-radius: 8px; cursor: pointer; font-weight: bold; }
        .error { color: red; font-size: 14px; margin-bottom: 10px; }
    </style>
</head>
<body style="background: #f0f2f5;">
    <div class="login-box">
        <h2 style="text-align: center;">Sign In</h2>
```



```
<p style="text-align: center; color: #666; font-size: 14px;">Campus Placement  
Management</p>
```

```
<?php if(isset($error)) echo "<p class='error'>$error</p>"; ?>
```

```
<form method="POST">  
  <input type="email" name="email" placeholder="Email (admin@example.com)"  
required>  
  <input type="password" name="password" placeholder="Password (admin123)"  
required>  
  <button type="submit">Login</button>  
</form>
```

```
<div style="margin-top: 20px; text-align: center; font-size: 12px; color: #888;">  
  <p>Admin: admin@example.com / admin123</p>  
  <p>Student: john@student.com / pass123</p>  
</div>  
</div>  
</body>  
</html>
```

4. profile.php – User Profile Module

Displays profile information and academic details for students.

profile.php – Important Code

```
<?php  
include 'db_connect.php';  
include 'header.php';  
  
if (!isset($_SESSION['user_id']))  
  { header("Location: login.php");  
    exit();  
  }  
  
$user_id = $_SESSION['user_id'];  
  
// Get User & Student details  
$sql = "SELECT u.*, s.roll_no, s.cgpa, s.branch, s.skills  
FROM users u  
LEFT JOIN students s ON u.id = s.student_id  
WHERE u.id = $user_id";  
$result = $conn->query($sql);
```



```
$user_data = $result->fetch_assoc();
?>

<div class="container">
  <h2>User Profile Details</h2>
  <hr>
  <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 20px; margin-top: 20px;">
    <div class="card" style="background: #f8f9fa; color: #333; text-align: left; border: 1px
solid #ddd;">
      <h3>Basic Information</h3>
      <p><b>Name:</b> <?php echo $user_data['name']; ?></p>
      <p><b>Email:</b> <?php echo $user_data['email']; ?></p>
      <p><b>Role:</b> <?php echo ucfirst($user_data['role']); ?></p>
      <p><b>Member Since:</b> <?php echo date('d M Y',
strtotime($user_data['created_at'])); ?></p>
    </div>

    <?php if ($user_data['role'] == 'student'): ?>
      <div class="card" style="background: #e3f2fd; color: #333; text-align: left; border: 1px
solid #ddd;">
        <h3>Academic Details</h3>
        <p><b>Roll No:</b> <?php echo $user_data['roll_no'] ?: 'N/A'; ?></p>
        <p><b>Branch:</b> <?php echo $user_data['branch'] ?: 'N/A'; ?></p>
        <p><b>CGPA:</b> <?php echo $user_data['cgpa'] ?: '0.0'; ?></p>
        <p><b>Skills:</b> <?php echo $user_data['skills'] ?: 'Update your skills'; ?></p>
      </div>
    <?php endif; ?>
  </div>

  <div style="margin-top: 30px;">
    <button onclick="window.print()" style="padding: 10px 20px; background: #2c3e50;
color: white; border: none; border-radius: 5px; cursor: pointer;">Print Profile for
Submission</button>
  </div>
</div>

</body>
</html>
```

5. Database SQL Schema

Core database design containing user, student, company and interview tables with relationships and sample data.



database.sql – Schema Definition

-- Database: placement_db

CREATE DATABASE IF NOT EXISTS placement_db;

USE placement_db;

-- 1. Users Table

```
CREATE TABLE IF NOT EXISTS users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    password VARCHAR(255) NOT NULL,  
    role ENUM('admin', 'student') DEFAULT 'student',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- 2. Students Table

```
CREATE TABLE IF NOT EXISTS students (  
    student_id INT PRIMARY KEY,  
    roll_no VARCHAR(20) UNIQUE,  
    cgpa DECIMAL(3,2),  
    branch VARCHAR(50),  
    skills TEXT,  
    FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

-- 3. Companies Table

```
CREATE TABLE IF NOT EXISTS companies  
( id INT AUTO_INCREMENT PRIMARY  
KEY, name VARCHAR(100) NOT NULL,  
website VARCHAR(255),  
role_offered VARCHAR(100),  
ctc VARCHAR(50)  
);
```

-- 4. Interviews Table

```
CREATE TABLE IF NOT EXISTS interviews  
( id INT AUTO_INCREMENT PRIMARY  
KEY,  
student_id INT,  
company_id INT,  
batch_date DATE,  
status ENUM('scheduled', 'completed', 'cancelled') DEFAULT 'scheduled',  
FOREIGN KEY (student_id) REFERENCES users(id),  
FOREIGN KEY (company_id) REFERENCES companies(id)  
);
```



-- Insert Sample Data

```
INSERT INTO users (name, email, password, role) VALUES  
( 'Admin User', 'admin@example.com', 'admin123', 'admin'),  
( 'John Doe', 'john@student.com', 'pass123', 'student');
```

```
INSERT INTO students (student_id, roll_no, cgpa, branch, skills) VALUES  
(2, 'CS101', 8.5, 'Computer Science', 'React, Node, SQL');
```

```
INSERT INTO companies (name, website, role_offered, etc) VALUES  
( 'Google', 'google.com', 'Software Engineer', '30 LPA'),  
( 'Microsoft', 'microsoft.com', 'DevOps Engineer', '25 LPA');
```



11.CONCLUSION AND FUTURE ENHANCEMENT

11.1 CONCLUSION

The CareerAI Hub: Campus Placement Management System was developed to automate and streamline the campus recruitment process. The system successfully replaces traditional manual methods of managing student records, placement drives, recruiter information, and application tracking with a centralized web-based platform.

The application enables students to create profiles, apply for placement drives, and track their application status efficiently. Recruiters can manage job opportunities and review eligible candidates, while administrators can oversee the entire placement process through a dedicated dashboard. The system improves accuracy, reduces paperwork, minimizes human errors, and provides real-time access to placement information.

The use of technologies such as HTML5, CSS3, JavaScript (ES6), PHP 8.2, MySQL, Node.js, and Express.js ensures a secure, reliable, and scalable solution. Overall, the project successfully achieves its objective of enhancing the efficiency, transparency, and effectiveness of campus placement management.

11.2FUTURE ENHANCEMENT

Although the current system meets the essential requirements of campus placement management, several enhancements can be implemented in the future to improve functionality and user experience.

Future Enhancements

- Integration of Artificial Intelligence (AI) for resume screening and candidate recommendations.
- Implementation of Machine Learning algorithms to predict placement trends and student success rates.
- Development of a mobile application for Android and iOS platforms.
- Integration of email and SMS notification services for instant communication.
- Addition of online aptitude tests and coding assessments within the platform.
- Implementation of video interview scheduling and management features.
- Cloud deployment to improve scalability, availability, and data security.
- Integration with professional networking platforms such as LinkedIn for profile verification and recruitment opportunities.
- Advanced analytics and reporting dashboards for placement coordinators.
- Multi-college and multi-campus support for centralized placement management.



12. BIBLIOGRAPHY

1. Pressman, R. S. (2019). Software Engineering: A Practitioner's Approach. McGraw-Hill Education.
2. Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems. Pearson Education.
3. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts. McGraw-Hill Education.
4. Duckett, J. (2021). HTML and CSS: Design and Build Websites. Wiley Publications.
5. Nixon, R. (2023). Learning PHP, MySQL & JavaScript. O'Reilly Media.
6. [PHP Documentation](#)
7. [MySQL Documentation](#)
8. [Node.js Documentation](#)
9. [Express.js Documentation](#)
10. Research papers and articles related to Campus Placement Management Systems, Recruitment Systems, and Web Application Development.



Sl.no	Description	Page no
01.	Introduction	4
	1.1 Modules	5-6
02.	Requirement Analysis	7-12
	2.1 Existing System	11
	2.2 Proposed System	11
	2.3 Feasibility Study	12
03.	Literature Survey	13-14
04.	System Specifications	15
05.	H/w and s/w Configurations	16
06.	Overview of Tools and Technologies used	17-30
	HTML5	17-20
	CSS3	21-22
	JAVASCRIPT	23
	MySQL	23-24
	Node.js	24
	PHP8.2	25
	Express.js	26
	Visual Studio Code	26-27
	XAMPP	27-28
	php MyAdmin	28-29
	Google Chrome	29-30
07.	System Design	31-40
	7.1 Data Flow Diagram	31-33
	7.2 E-R Diagram	34-35
	7.3 Structure of Table	36-40
08.	System Testing	41-46
	8.1 Level of Testing	47-51
09.	Output Screen Pages	52-58
10.	System Implementation – Coding	59-86
11.	Conclusion and Future Enhancements	87
12.	Bibliography	88



ABSTRACT:

The project titled "CareerAI Hub: Campus Placement Management System" is a comprehensive web-based application developed to automate and streamline the campus recruitment and placement process. Traditional placement management systems largely depend on manual record keeping, spreadsheets, and paper-based documentation, which often result in data redundancy, processing delays, communication gaps, and human errors. CareerAI Hub addresses these challenges by providing a centralized platform that efficiently manages student profiles, recruiter information, placement drives, job applications, and placement analytics.

The system offers role-based access control for three primary stakeholders: Students, Recruiters, and Placement Administrators. Students can create and manage their profiles, upload resumes, view eligibility criteria, apply for placement drives, and track application status. Recruiters can register their organizations, publish job opportunities, define eligibility requirements, review candidate applications, and update recruitment outcomes. Placement administrators are provided with advanced tools to manage student records, verify eligibility, schedule placement drives, monitor recruitment activities, generate reports, and communicate important notifications.

The application is developed using modern web technologies including HTML5, CSS3, JavaScript (ES6), Tailwind CSS, PHP 8.2, MySQL, Node.js, and Express.js. MySQL serves as the relational database management system, ensuring secure and efficient storage of placement-related information. The system follows a modular architecture and employs a normalized database design to maintain data integrity, minimize redundancy, and improve retrieval efficiency.

CareerAI Hub incorporates automated eligibility verification mechanisms based on academic criteria such as CGPA, branch, and backlog count. This feature significantly reduces manual effort and ensures that only eligible students are allowed to apply for specific placement drives. The system also provides real-time application tracking, notification management, and dashboard-based analytics that assist placement officers in monitoring recruitment performance and placement statistics.

The software development process follows a structured methodology involving requirement analysis, feasibility study, system design, database design, implementation, testing, and



deployment. Design artifacts such as Data Flow Diagrams (DFD), Entity Relationship Diagrams (ERD), database schemas, and system flowcharts are used to model system functionality and data interactions. Comprehensive testing techniques including Unit Testing, Integration Testing, System Testing, Black Box Testing, White Box Testing, Validation Testing, and Performance Testing were conducted to ensure reliability and accuracy.

The implementation of CareerAI Hub significantly improves operational efficiency, reduces administrative workload, enhances transparency in recruitment activities, and eliminates common errors associated with manual placement management. The system serves as an effective bridge between students, recruiters, and educational institutions, creating a more organized and efficient placement ecosystem.

Future enhancements may include Artificial Intelligence-based resume screening, machine learning-driven placement predictions, cloud deployment, mobile application support, online assessment modules, interview scheduling systems, and advanced analytics dashboards. These enhancements will further strengthen the platform and transform it into a complete placement intelligence and career development solution.



1.INTRODUCTION:

The CareerAI Hub: Campus Placement Management System is an advanced web-based platform developed to streamline and automate the entire campus recruitment process. Traditional placement management systems often rely on manual documentation, spreadsheets, and email-based communication, which can lead to inefficiencies, data redundancy, delays, and human errors. As educational institutions handle increasing numbers of students and recruiting organizations, there is a growing need for a centralized and intelligent placement management solution.

CareerAI Hub provides a comprehensive digital platform that connects students, recruiters, and placement administrators in a single ecosystem. Students can create and manage professional profiles, upload resumes, track application status, receive placement notifications, and apply for eligible job opportunities. Recruiters can register their organizations, publish job openings, define eligibility criteria, review applications, and shortlist candidates efficiently. Placement administrators can manage student records, company details, placement drives, and recruitment schedules through a secure administrative dashboard.

One of the key features of the system is its automated eligibility verification mechanism. The platform evaluates students based on predefined criteria such as CGPA, academic branch, backlog status, and graduation year, ensuring that only qualified candidates can apply for specific recruitment drives. This automation reduces administrative workload and eliminates manual screening errors.

The system is developed using PHP for the backend and MySQL for database management, ensuring reliable performance, secure data storage, and efficient information retrieval. Additional features include user authentication, role-based access control, placement analytics, application tracking, notification management, and report generation.

CareerAI Hub significantly improves transparency, communication, and operational efficiency throughout the placement process. By digitizing recruitment activities and providing real-time access to information, the system reduces processing time, enhances decision-making, and improves the overall placement experience for students, recruiters, and educational institutions



1.1 MAJOR MODULES:

- STUDENT PORTAL MODULE
- PLACEMENT ADMINISTRATION MODULE
- RECRUITER MANAGEMENT MODULE
- DATABASE MANAGEMENT MODULE

FUNCTIONALITIES OF MODULES:

1.Student Portal Module

The Student Portal Module provides students with a centralized platform to manage their placement activities efficiently. Students can register, create and update their profiles, upload resumes and academic documents, view available job opportunities, and apply for recruitment drives. The module also provides real-time notifications regarding placement drives, interview schedules, company announcements, and selection results. Students can track the status of their applications and access detailed information about recruiting companies and job roles. This module enhances student participation and ensures smooth communication between students and the placement cell.

2.Placement Administration Module

The Placement Administration Module enables placement officers to efficiently manage all recruitment-related activities through a secure administrative interface. It allows administrators to maintain student records, verify eligibility criteria, schedule placement drives, manage company information, and monitor recruitment progress. The module supports the publication of notifications, generation of placement reports, and analysis of placement statistics. By automating routine administrative tasks, it reduces manual effort, improves operational efficiency, and ensures transparency throughout the placement process.



3. Recruiter Management Module

The Recruiter Management Module provides organizations with a platform to conduct recruitment activities effectively. Recruiters can create company profiles, post job openings, define eligibility requirements, and manage recruitment drives. The module allows recruiters to view eligible candidates, access student resumes, shortlist applicants, schedule interviews, and update candidate selection status. It facilitates seamless interaction between recruiters and the institution while reducing the time and effort required for candidate screening and recruitment management.

4. Database Management Module

The Database Management Module serves as the core component of the system by securely storing and managing all placement-related information. It maintains student profiles, recruiter details, job postings, application records, interview schedules, and placement results in a structured manner. The module ensures data accuracy, integrity, security, and efficient retrieval of information. It also supports user authentication, data backup, report generation, and smooth communication among different modules. By providing a reliable data management framework, it ensures the overall stability and performance of the placement management system.



2.REQUIREMENT ANALYSIS:

2.1 SOFTWARE REQUIREMENTS:

The CareerAI Hub: Campus Placement Management System requires a combination of modern software technologies and development tools to ensure efficient development, testing, deployment, and maintenance. These software components work together to provide a secure, scalable, and user-friendly platform for managing campus placement activities.

Frontend Technologies

The frontend of the system is developed using HTML5, CSS3, JavaScript (ES6), and Tailwind CSS. HTML5 is used to structure web pages, while CSS3 provides styling and visual presentation. JavaScript enables dynamic and interactive functionalities such as form validation, real-time updates, and user interactions. Tailwind CSS helps create responsive and modern user interfaces that adapt seamlessly to different screen sizes and devices, ensuring an enhanced user experience.

Backend Technology

PHP 8.2 serves as the backend programming language responsible for implementing the business logic of the application. It handles user authentication, session management, form processing, data validation, and communication between the frontend and the database. PHP provides a secure and efficient environment for processing placement-related operations and managing user requests.

Database Management System

MySQL is used as the relational database management system for storing and managing all application data. It maintains records related to students, recruiters, placement drives, job applications, interview schedules, and placement results. MySQL ensures data consistency, efficient query processing, and reliable storage, making it suitable for handling large volumes of placement-related information.



Database Administration Tool

phpMyAdmin is used for database administration and management. It provides a graphical user interface for creating databases, managing tables, executing SQL queries, importing and exporting data, and monitoring database performance. This tool simplifies database maintenance and reduces the complexity of manual database operations.

Application Development and Deployment Stack

Node.js, Express.js, and Vite are utilized to support modern web application development and deployment. Node.js provides a runtime environment for executing JavaScript outside the browser. Express.js facilitates server-side application development and API management, while Vite enhances frontend development through faster build processes, hot module replacement, and optimized production builds. Together, these technologies improve application performance, scalability, and maintainability.

Development Environment

Visual Studio Code (VS Code) and PyCharm Professional are used as integrated development environments (IDEs) for coding, debugging, testing, and project management. These tools offer advanced features such as syntax highlighting, code completion, version control integration, debugging support, and extension ecosystems that improve developer productivity and code quality.

Web Server

XAMPP and WAMP Server are used to create a local server environment for application development and testing. These platforms include Apache Web Server, PHP, and MySQL, allowing developers to run and test the application on local machines before deployment. They provide an easy-to-configure environment for web application development.

Operating System Support

The system is designed to be platform-independent and can operate on multiple operating systems, including Windows 10, Windows 11, macOS Sequoia, and various Linux distributions such as Ubuntu and Fedora. This flexibility enables developers and administrators to choose the operating environment that best suits their requirements.



2.2 HARDWARE REQUIREMENTS

The successful development, testing, deployment, and operation of the CareerAI Hub: Campus Placement Management System require adequate hardware resources to ensure reliable performance and efficient execution of application processes. The following hardware components are recommended for supporting the system and its development environment.

System Configuration

A desktop computer or laptop with a modern hardware configuration is required to develop, test, and run the application. The system should be capable of supporting web development tools, database servers, browsers, and other software components simultaneously without performance issues. A dedicated system with updated hardware specifications improves development efficiency and application responsiveness.

Processor

The application requires an Intel Core i3, Intel Core i5, Intel Core i7, AMD Ryzen 3, Ryzen 5, or equivalent processor. A multi-core processor is recommended to handle database operations, server-side processing, and multitasking efficiently. Higher-performance processors improve system responsiveness during testing, deployment, and concurrent user operations.

Memory (RAM)

A minimum of 4 GB RAM is required to run the application and supporting software tools. However, 8 GB RAM or higher is strongly recommended for smoother performance, faster application execution, efficient database management, and simultaneous use of development environments, browsers, and server applications. Systems with 16 GB RAM provide better performance for large-scale testing and future scalability.

Storage Device

The system requires at least 50 GB of free storage space for installing the operating system, development tools, database software, project files, backups, and documentation. Solid State Drives (SSD) are recommended instead of traditional Hard Disk Drives (HDD) because they offer faster data access, reduced loading times, and improved overall system performance.

Monitor and Display

A monitor with a minimum screen resolution of 1366×768 pixels is required for effective application development and testing. A Full HD (1920×1080) display is recommended to provide better visibility, enhanced user interface design capabilities, and improved productivity during coding and debugging activities.

Input Devices

Standard input devices such as a keyboard and mouse are necessary for data entry, application navigation, system configuration, and software development tasks. Ergonomic input devices may be used to improve user comfort during extended development sessions.



Network Connectivity

A stable internet connection is recommended for downloading software packages, receiving updates, communicating with recruiters and students, accessing cloud services, using version control systems such as GitHub, and deploying the application to web servers. Broadband or high-speed internet connectivity ensures uninterrupted access to online resources and services.

Backup and Storage Devices

External storage devices such as USB drives, external hard disks, or cloud storage services may be used for maintaining project backups, database exports, and important documentation. Regular backups help prevent data loss and support disaster recovery.

Server Hardware (Optional for Deployment)

For institutional deployment, a dedicated server with higher hardware specifications may be used to support multiple concurrent users. Recommended server specifications include:

- Intel Xeon or equivalent processor
- 16 GB RAM or higher
- 250 GB SSD or greater storage capacity
- High-speed network connectivity
- Backup power supply and security mechanisms

Power Supply

A reliable power source is necessary to ensure uninterrupted operation of the system. The use of an Uninterruptible Power Supply (UPS) is recommended to prevent data loss and system shutdown during power failures.

Hardware Compatibility

The hardware should be compatible with modern operating systems such as Windows 10/11, Linux distributions, and macOS. Compatibility ensures smooth installation and operation of the software components required for the placement management system.



2.3 EXISTING SYSTEM:

The existing placement management process mainly relies on spreadsheets, paper records, emails, and Google Forms. This manual approach creates several challenges in managing student and recruiter information.

Limitations of the Existing System

- Student records are stored in multiple files, leading to data duplication and inconsistency.
- Eligibility verification is performed manually, making the process time-consuming and error-prone.
- There is no real-time tracking of application and interview status.
- Important placement notifications may be missed due to reliance on emails and notice boards.
- Generating placement reports and statistics requires significant manual effort.

2.4 PROPOSED SYSTEM:

CareerAI Hub is a centralized web-based placement management system designed to automate and streamline placement activities.

Advantages of the Proposed System

- Maintains all student, recruiter, and placement data in a centralized database.
- Automatically verifies student eligibility based on predefined criteria such as CGPA, branch, and backlogs.
- Provides separate dashboards for administrators, students, and recruiters.
- Enables real-time tracking of applications, interviews, and placement status.
- Generates placement reports, analytics, and statistics instantly.
- Improves accuracy, transparency, and efficiency in the placement process



2.5 FEASIBILITY STUDY:

A feasibility study is conducted to evaluate whether the proposed system can be developed and implemented successfully within the available resources, time, and operational environment. For the CareerAI Hub: Campus Placement Management System, the feasibility study includes technical, economic, operational, and schedule feasibility.

Technical Feasibility

The proposed system is developed using HTML5, CSS3, JavaScript, PHP, MySQL, Node.js, and related web technologies. The required software tools, development environment, and hardware resources are readily available. The technologies used are reliable, widely supported, and suitable for developing a secure web-based placement management system. Therefore, the project is technically feasible.

Economic Feasibility

The project utilizes open-source technologies such as PHP, MySQL, Node.js, and XAMPP, which significantly reduce software licensing costs. Since the system automates manual placement activities, it reduces administrative effort, saves time, and improves productivity. Hence, the project is economically feasible.

Operational Feasibility

The system is designed with a user-friendly interface that can be easily used by students, recruiters, and placement administrators. It provides centralized data management, automated eligibility verification, real-time notifications, and placement tracking. These features improve efficiency and transparency in placement operations, making the system operationally feasible.

Schedule Feasibility

The project can be completed within the timeframe allocated for the MCA academic project. The availability of development tools, technologies, and project resources ensures smooth implementation, testing, and deployment within the planned schedule. Therefore, the project is schedule feasible.



3.LITERATURE SURVEY:

The placement process is one of the most important activities in higher educational institutions, as it serves as a bridge between students and potential employers. With the increasing number of students and recruiting organizations, managing placement activities through traditional methods has become challenging. Educational institutions require efficient systems to handle student information, company details, recruitment drives, eligibility verification, and placement statistics. As a result, researchers and software developers have focused on designing automated placement management systems to improve the efficiency of campus recruitment processes.

Traditionally, placement cells maintained student records, company information, and recruitment details using spreadsheets, paper documents, notice boards, and email communications. Although these methods were effective for smaller student populations, they became difficult to manage as the volume of data increased. Manual processes often resulted in data redundancy, record inconsistencies, delayed communication, and errors in eligibility verification. These limitations highlighted the need for computerized placement management solutions.

Several web-based recruitment and placement systems have been developed to automate various aspects of the recruitment process. These systems allow students to create profiles, upload resumes, and apply for available job opportunities online. Recruiters can post job openings, define eligibility criteria, and review candidate applications. Such systems have significantly reduced paperwork and improved communication between institutions and companies.

Research studies have shown that centralized database systems improve data consistency and accessibility. By storing student records, recruiter details, placement drives, and application information in a single database, institutions can reduce duplication and ensure accurate information management. Database-driven systems also provide better security, faster data retrieval, and efficient record maintenance compared to manual methods.

Modern placement management systems incorporate automated eligibility verification mechanisms. These systems compare student academic information, such as CGPA, branch, and backlog status, against company requirements before allowing applications. This automation reduces the workload of placement

officers and minimizes the chances of human error. It also ensures that only eligible candidates participate in recruitment drives.

Recent advancements in web technologies have further enhanced placement management systems by introducing real-time notifications, dashboard analytics, and application tracking features. Students can instantly receive updates regarding recruitment drives, interview schedules, and selection results. Placement administrators can monitor placement performance



through graphical reports and statistical summaries, enabling better decision-making and planning.

Many existing recruitment platforms focus primarily on job matching and online applications. However, they often lack features specifically required by educational institutions, such as academic eligibility checks, placement drive management, student verification, and institutional reporting. Consequently, there is a growing demand for a dedicated campus placement management solution that addresses the unique requirements of colleges and universities.

The proposed CareerAI Hub: Campus Placement Management System is designed to overcome these limitations by providing a centralized and automated platform for managing placement activities. The system integrates student profile management, recruiter management, eligibility verification, application tracking, drive scheduling, notifications, and placement analytics within a single environment. This approach improves efficiency, transparency, and accuracy throughout the placement process.

In conclusion, the literature survey indicates that automated placement management systems play a vital role in modern educational institutions. Existing studies and systems emphasize the importance of centralized data management, automation, and real-time communication. Building upon these concepts, CareerAI Hub provides a comprehensive solution that enhances campus recruitment operations and supports effective interaction between students, recruiters, and placement administrators.



4.SYSTEM SPECIFICATION:

A software requirements specification (SRS) is a comprehensive description of the intended purpose and environment for software under development. The SRS fully describes what the software will do and how it will be expected to perform.

An SRS minimizes the time and effort required by developers to achieve desired goals and also minimizes the development cost. A good SRS defines how an application will interact with system hardware, other programs, and human users in a wide variety of real-world situations. Parameters such as operating speed, response time, availability, portability, maintainability, footprint, security, and speed of recovery from adverse events are evaluated.

The requirements of this web application (ems) developed by us is very well understood and documented initially when the process starts. Accordingly, the hardware and software requirements are chosen. The same can be referred to in the next section of this report. This web application runs in the Apache Tomcat Server, which can be accessed by web clients through the Internet from anywhere in the world. The SRS parameters are adopted in this system, with the detailed feasibility study. While developing, this system was tested as the localhost, through the Mozilla Firefox browser. Also, few other popular web browsers are tested.



5.SOFTWARE AND HARDWARE CONFIGURATION:

HARDWARE CONFIGURATION

Component	Specification
Processor	AMD Ryzen 5 5000 series
RAM	4 GB Minimum (8 GB Recommended)
Hard Disk (HDD/SSD)	256 GB or Higher
Monitor	Standard LCD/LED Display
Keyboard & Mouse	Standard Input Devices
Network	Internet Connection (Optional for Deployment)

Software Configuration

Component	Specification
Client-SideTechnologies (Frontend)	HTML5, CSS3, JavaScript (ES6), Tailwind CSS
Server-Side Technologies (Backend)	PHP 8.2, Node.js, Express.js
Database Management System	MySQL
IDE	Visual Studio Code (VS Code)
Web Server	XAMPP / WAMP Server
Connection Technology	MySQL Connector, REST APIs
Operating System	Windows 10/11, Linux, macOS
Development Tools	PHP, MySQL, Node.js, Vite, HTML5, CSS3, JavaScript, Tailwind CSS
Daabase Administration Tool	phpMyAdmin



6.1 OVERVIEW OF TOOL AND TECHNOLOGIES USED

The development of the CareerAI Hub: Campus Placement Management System involves the use of various tools and technologies for designing, developing, testing, and managing the application. These technologies ensure efficient performance, secure data management, and a user-friendly experience.

HTML5



Introduction

HTML5 (HyperText Markup Language Version 5) is the latest standard version of HyperText Markup Language used for designing and structuring web pages and web applications. It serves as the foundation of modern web development by providing a standardized method for organizing content, displaying information, and creating interactive user interfaces. HTML5 was developed to overcome the limitations of previous HTML versions and to support the growing demand for multimedia-rich, responsive, and highly interactive web applications.

HTML5 is not only a markup language but also a complete platform for building modern web applications. It introduces new semantic elements, multimedia capabilities, advanced form controls, local storage mechanisms, and APIs that improve the functionality, performance, and accessibility of websites. Due to its flexibility and compatibility, HTML5 has become the preferred technology for developing web-based systems across various domains, including education, healthcare, banking, e-commerce, and recruitment management.

Evolution of HTML5

Before HTML5, web developers relied heavily on external plugins such as Adobe Flash and Silverlight to display multimedia content. Earlier versions of HTML provided limited support for dynamic content and interactive applications. As web technologies evolved, the need for a more powerful and efficient standard became evident.

HTML5 was introduced by the World Wide Web Consortium (W3C) and WHATWG (Web Hypertext Application Technology Working Group) to provide native support for multimedia, graphics, data storage, and advanced web functionalities. It significantly simplified web development and enabled developers to build rich internet applications without depending on third-party technologies.



Features of HTML5

1. Semantic Elements

One of the most important improvements introduced by HTML5 is the use of semantic elements. These tags provide meaningful descriptions of webpage content and improve code readability.

Examples include:

- `<header>` – Defines the header section of a webpage.
- `<nav>` – Represents navigation links.
- `<section>` – Defines a logical section of content.
- `<article>` – Represents independent content.
- `<aside>` – Defines supplementary information.
- `<footer>` – Represents the footer section.

Semantic elements improve website structure, accessibility, and search engine optimization.

2. Native Multimedia Support

HTML5 provides built-in support for audio and video content through dedicated tags.

- `<audio>` tag for embedding audio files.
- `<video>` tag for embedding video files.

This feature eliminates the need for external plugins and improves browser compatibility. Multimedia content can now be integrated directly into web pages, resulting in better performance and user experience.

3. Enhanced Form Controls

HTML5 introduces several advanced form input types and validation features. Examples include:

- Email fields
- Number fields
- Date pickers
- URL fields
- Range sliders
- Search boxes

Built-in validation reduces the amount of JavaScript code required and helps ensure accurate user input.

4. Canvas for Graphics

The `<canvas>` element enables dynamic drawing of graphics using JavaScript.

Applications include:

- Charts and graphs
- Animations
- Interactive visualizations
- Browser-based games
- Data dashboards

Canvas allows developers to create rich graphical interfaces directly within the browser.



5. Local and Session Storage

HTML5 provides client-side storage mechanisms that allow data to be stored locally within the browser.

Local Storage

Stores data permanently until manually removed.

1. Session Storage

- Stores data temporarily during the current browser session.
- These storage mechanisms improve application performance by reducing server requests and enabling offline functionality.

2. Geolocation Support

HTML5 includes a Geolocation API that allows web applications to access a user's geographical location with permission.

Applications include:

- Navigation systems
- Location-based services
- Delivery tracking systems
- Mapping applications

3. Offline Web Applications

HTML5 supports offline capabilities through local storage and caching mechanisms. Users can continue accessing certain application features even when internet connectivity is unavailable.

4. Cross-Platform Compatibility

HTML5 applications can run on various operating systems and devices, including:

- Windows
- Linux
- macOS
- Android
- iOS

This compatibility ensures a consistent user experience across multiple platforms.

5. Improved Accessibility

HTML5 improves accessibility for users with disabilities through semantic tags and support for assistive technologies such as screen readers.

Benefits include:

- Better navigation
- Improved content understanding
- Enhanced usability for all users

6. Search Engine Optimization (SEO)

Semantic HTML5 elements help search engines understand webpage content more effectively. This improves website visibility and ranking in search engine results.



Advantages of HTML5

- Easy to learn and implement.
- Supports multimedia without external plugins.
- Provides faster webpage loading.
- Improves website performance.
- Enhances user experience.
- Supports responsive web design.
- Offers better security and reliability.
- Reduces dependency on third-party software.
- Improves accessibility and SEO.
- Compatible with all modern browsers.

Applications of HTML5

- HTML5 is widely used in:
- Educational portals
- E-commerce websites
- Banking systems
- Social networking platforms
- Healthcare applications
- Online learning systems
- Recruitment and placement management systems
- Content management systems
- Business websites
- Mobile web applications

HTML5 in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, HTML5 is used as the primary technology for designing and structuring web pages. It is responsible for creating student registration forms, login pages, recruiter dashboards, placement drive listings, application forms, profile management pages, and administrative interfaces.

The semantic elements of HTML5 help organize content effectively, while advanced form controls facilitate accurate data entry during student registration and application submission. HTML5 also contributes to responsive design, ensuring that the application functions smoothly across desktops, laptops, tablets, and mobile devices.

By providing a structured and interactive interface, HTML5 enhances the overall usability of the CareerAI Hub system and enables efficient interaction between students, recruiters, and placement administrators.



CSS3



Introduction

CSS3 (Cascading Style Sheets Level 3) is the latest version of Cascading Style Sheets used to control the presentation, formatting, and layout of web pages. It is one of the core technologies of web development, working alongside HTML5 and JavaScript to create modern, interactive, and visually appealing websites. While HTML defines the structure of a webpage, CSS3 determines how that content is displayed to users.

CSS3 was introduced to overcome the limitations of earlier CSS versions and provide advanced styling features for modern web applications. It enables developers to create responsive layouts, attractive designs, animations, and visual effects without relying heavily on external scripts or images. Due to its flexibility and powerful features, CSS3 has become an essential technology in the development of web-based systems.

Features of CSS3

1. Responsive Web Design

CSS3 supports responsive web design through media queries, allowing web pages to automatically adapt to different screen sizes and devices. This ensures that websites function properly on desktops, laptops, tablets, and smartphones.

2. Advanced Styling

CSS3 provides extensive styling capabilities, including control over fonts, colors, backgrounds, margins, padding, borders, and spacing. These features help developers create attractive and professional-looking web pages.

3. Animations and Transitions

CSS3 allows developers to add animations and transition effects directly to web elements. These effects improve user interaction by making the interface more dynamic and visually engaging.

Examples include:

- Hover effects on buttons
- Fade-in and fade-out effects
- Moving and rotating elements
- Loading animations

4. Flexbox Layout

Flexbox is a layout model introduced in CSS3 that makes it easier to arrange elements within a webpage. It provides efficient alignment, spacing, and distribution of content, even when



screen sizes change.

5. Grid Layout

CSS Grid is a powerful two-dimensional layout system that allows developers to create complex webpage structures using rows and columns. It simplifies webpage design and improves layout flexibility.

6. Gradients and Background Effects

CSS3 supports linear and radial gradients, enabling smooth color transitions without the use of image files. It also provides background effects that enhance the visual appeal of web pages.

7. Shadows and Visual Effects

CSS3 introduces Box Shadow and Text Shadow properties, allowing developers to create depth and emphasis in user interface components.

Benefits include:

- Improved readability
- Enhanced visual appearance
- Modern user interface design

8. Rounded Corners

The Border Radius property allows developers to create rounded corners for buttons, images, forms, and containers without using graphics.

9. Custom Fonts

CSS3 supports custom typography through the @font-face rule, enabling the use of unique fonts that enhance website branding and design consistency.

10. Browser Compatibility

CSS3 is supported by all modern web browsers, including Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. This ensures consistent performance across different platforms.

CSS3 in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, CSS3 is used to enhance the visual presentation and usability of the application. It is responsible for designing student registration forms, login pages, recruiter dashboards, placement drive listings, administrative panels, navigation menus, and report pages.

CSS3 ensures that the application is responsive and accessible across different devices and screen sizes. Features such as Flexbox, Grid layouts, animations, and responsive design help create a modern and user-friendly interface. The technology improves navigation, readability, and overall user experience for students, recruiters, and placement administrators.



JavaScript (ES6)



JavaScript

Introduction

JavaScript (ECMAScript 6 or ES6) is a popular programming language used to create interactive and dynamic web applications. It works together with HTML5 and CSS3 to enhance the functionality and responsiveness of web pages. ES6 introduced several advanced features such as arrow functions, classes, modules, template literals, and promises, making JavaScript more efficient and easier to maintain.

Features of JavaScript (ES6)

- Adds interactivity to web pages.
- Supports event handling and form validation.
- Provides object-oriented programming features.
- Introduces arrow functions and template literals.
- Supports asynchronous programming using Promises and Async/Await.
- Enables dynamic content updates without page reloads.

JavaScript in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, JavaScript (ES6) is used for form validation, dynamic content updates, user interactions, placement drive filtering, dashboard functionality, and real-time status updates. It enhances the responsiveness and usability of the application.

MySQL



Introduction

MySQL is an open-source Relational Database Management System (RDBMS) used to store, organize, and manage data efficiently. It uses Structured Query Language (SQL) to perform operations such as inserting, updating, deleting, and retrieving data. MySQL is widely used in web applications because of its reliability, security, and high performance.

Features of MySQL

- Stores data in tables consisting of rows and columns.
- Supports SQL queries for efficient data management.

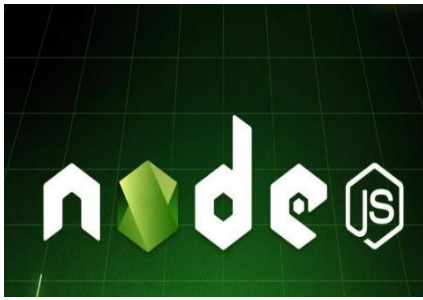


- Provides high performance and fast data retrieval.
- Ensures data security through user authentication and access control.
- Supports multi-user access and concurrent operations.
- Maintains data integrity using primary keys and foreign keys.
- Offers backup and recovery mechanisms to protect data.

MySQL in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, MySQL is used as the backend database to store student profiles, recruiter information, placement drives, job applications, interview schedules, notifications, and placement records. It ensures secure data storage, quick retrieval of information, and efficient management of placement activities.

Node.js



Introduction

Node.js is an open-source, cross-platform JavaScript runtime environment that allows developers to execute JavaScript code on the server side. It is built on Google's V8 JavaScript engine and is widely used for developing fast, scalable, and high-performance web applications. Node.js uses an event-driven and non-blocking architecture, enabling it to handle multiple user requests efficiently.

Features of Node.js

- Executes JavaScript on the server side.
- Uses an event-driven and non-blocking architecture.
- Supports asynchronous programming for better performance.
- Handles multiple client requests simultaneously.
- Provides fast execution through the V8 JavaScript engine.
- Supports the development of scalable and real-time applications.
- Offers a large collection of reusable packages through NPM (Node Package Manager).

Node.js in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, Node.js is used to develop backend services and REST APIs. It facilitates communication between the frontend and database, processes user requests, manages application data, and supports real-time functionalities such as notifications, dashboard updates, and placement tracking.



PHP 8.2



Introduction

PHP (Hypertext Preprocessor) 8.2 is a popular open-source server-side scripting language used for developing dynamic and interactive web applications. It is widely used for creating websites and database-driven applications due to its simplicity, flexibility, and efficiency. PHP code is executed on the server, and the generated output is sent to the user's web browser.

Features of PHP 8.2

- Open-source and freely available.
- Supports server-side scripting and web development.
- Easily integrates with MySQL and other databases.
- Supports Object-Oriented Programming (OOP).
- Provides improved performance and security features.
- Handles form processing, session management, and user authentication.
- Compatible with multiple operating systems such as Windows, Linux, and macOS.

Advantages of PHP 8.2

- Easy to learn and implement.
- Fast and efficient execution.
- Supports dynamic web page generation.
- Highly secure and reliable.
- Large community support and extensive documentation.
- Platform-independent and flexible.
- Cost-effective for web application development.

PHP 8.2 in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, PHP 8.2 is used to implement server-side functionalities such as student registration, login authentication, recruiter management, placement drive creation, eligibility verification, application processing, and database interactions. It acts as a bridge between the user interface and the MySQL database, ensuring smooth data processing and secure system operations.



Express.js



Introduction

Express.js is a fast, lightweight, and flexible web application framework built on top of Node.js. It simplifies server-side development by providing powerful features for handling routing, middleware, HTTP requests, and API development. Express.js helps developers build scalable and efficient web applications with less code and better organization.

Features of Express.js

- Provides a simple and flexible routing system.
- Supports middleware for request and response processing.
- Simplifies RESTful API development.
- Handles HTTP requests and responses efficiently.
- Supports integration with databases and third-party services.
- Enables secure and scalable web application development.
- Works seamlessly with Node.js.

Express.js in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, Express.js is used to create and manage REST APIs, handle user requests, process application data, and facilitate communication between the frontend and the database. It helps manage student registrations, recruiter information, placement drives, application tracking, and dashboard functionalities efficiently.

6.7 VISUAL STUDIO CODE:



Visual Studio Code is a streamlined code editor with support for development operations like debugging, task running, and version control. It aims to provide just the tools a developer needs



for a quick code-build-debug cycle and leaves more complex workflows to fuller featured IDEs, such as Visual Studio IDE. Visual Studio Code, also commonly referred to as VS Code, is a source-code editor made by Microsoft with the Electron Framework, for Windows, Linux and macOS. Features include support for debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded Git.

XAMPP Server



Introduction

XAMPP is a free and open-source local web server package used for developing, testing, and deploying web applications. The name XAMPP stands for Cross-Platform (X), Apache (A), MySQL (M), PHP (P), and Perl (P). It provides a complete development environment that allows developers to run PHP and MySQL applications on a local computer without requiring an internet connection.

XAMPP is widely used because it simplifies the installation and configuration of web development tools. It enables developers to create, test, and debug web applications before deploying them to a live server.

Features of XAMPP Server

- Provides Apache web server for hosting web applications.
- Supports PHP for server-side scripting.
- Includes MySQL database for data management.
- Offers phpMyAdmin for database administration.
- Easy installation and configuration.
- Supports multiple operating systems such as Windows, Linux, and macOS.
- Provides a local development and testing environment. .



XAMPP Server in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, XAMPP is used as the local server environment for running PHP scripts and managing the MySQL database. It hosts the application during development and testing, allowing developers to verify functionalities such as student registration, recruiter management, placement drive scheduling, and database operations before deployment.

phpMyAdmin



Introduction

phpMyAdmin is a free and open-source web-based database administration tool used for managing MySQL and MariaDB databases. It provides a graphical user interface (GUI) that allows users to perform database operations without writing complex SQL commands. phpMyAdmin simplifies database management tasks such as creating databases, tables, records, queries, and user accounts.

It is commonly included with XAMPP, WAMP, and other web server packages, making it a popular tool for web developers and database administrators.

Features of phpMyAdmin

- Provides a user-friendly graphical interface for database management.
- Allows creation, modification, and deletion of databases and tables.
- Supports SQL query execution and database administration.
- Enables importing and exporting of database files.
- Provides user account and privilege management.

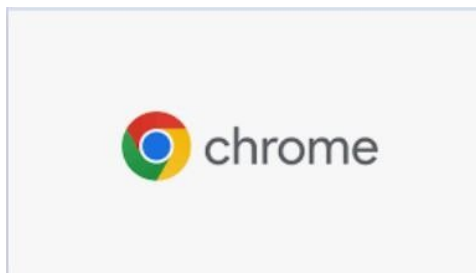


- Supports backup and recovery operations.
- Displays database structure, relationships, and records efficiently.

phpMyAdmin in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, phpMyAdmin is used to create and manage the MySQL database. It helps in designing tables, defining relationships, inserting records, executing queries, and monitoring placement-related data such as student profiles, recruiter details, placement drives, applications, and interview schedules. It also assists in database backup and maintenance activities.

Google Chrome



Introduction

Google Chrome is a widely used web browser developed by Google. It is known for its high speed, security, reliability, and user-friendly interface. Chrome supports modern web standards and provides a powerful platform for accessing, testing, and running web applications. Due to its performance and compatibility, it is one of the most popular browsers used by developers and end users worldwide.

Features of Google Chrome

- Fast and efficient web page loading.
- User-friendly and easy-to-navigate interface.
- Supports modern web technologies such as HTML5, CSS3, and JavaScript.
- Provides built-in security features against malicious websites.
- Includes Developer Tools for debugging and testing web applications.
- Supports extensions and plugins for enhanced functionality.



Google Chrome in CareerAI Hub

In the CareerAI Hub: Campus Placement Management System, Google Chrome is used to access, test, and verify the functionality of the web application. It helps developers check the responsiveness, performance, and compatibility of the system across different screen sizes and devices. Chrome's Developer Tools are also used for debugging, inspecting webpage elements, and monitoring application performance during development.



7.SYSTEM DESIGN:

7.1 DATA FLOW DIAGRAM

Data Flow Diagrams is the best and easiest tool to represent the flow of the data in the project. It is otherwise known as a bubble chart. It has the purpose of clarifying system requirements and identifying major transformations that will become programs in System design. It is the major starting point in the design phase that functionally decomposes the requirements specifications down to the lowest level of detail. A DFD consists of a series of bubbles joined by lines. The bubble represents data transformation and lines represent data flow in the system. A data flow diagram (DFD) maps out the flow of information for any process or system. It uses defined symbols like rectangles, circles and arrows, plus short text labels, to show data inputs, outputs, storage points and the routes between each destination. Data flowcharts can range from simple, even hand-drawn process overviews, to in-depth, multi-level DFDs that dig progressively deeper into how the data is handled. They can be used to analyze an existing system or model a new one. Like all the best diagrams and charts, a DFD can often visually “say” things that would be hard to explain in words, and they work for both technical and nontechnical audiences, from developer to CEO. That’s why DFDs remain so popular after all these years. While they work well for data flow software and systems, they are less applicable nowadays to visualizing interactive, real-time or database-oriented software or systems.

Symbols



Square, this defines source or destination of data.

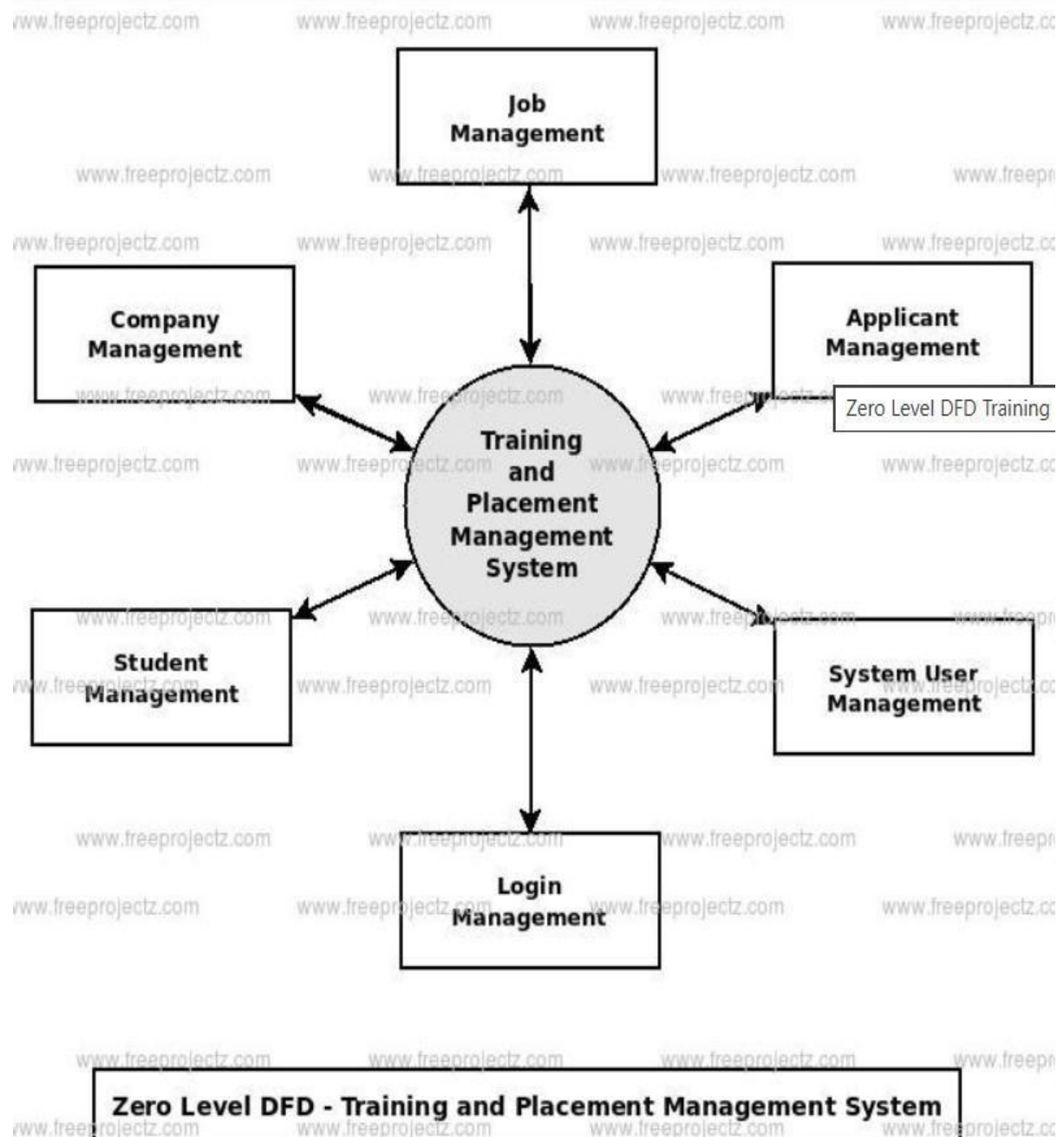


Arrow, which shows data flow

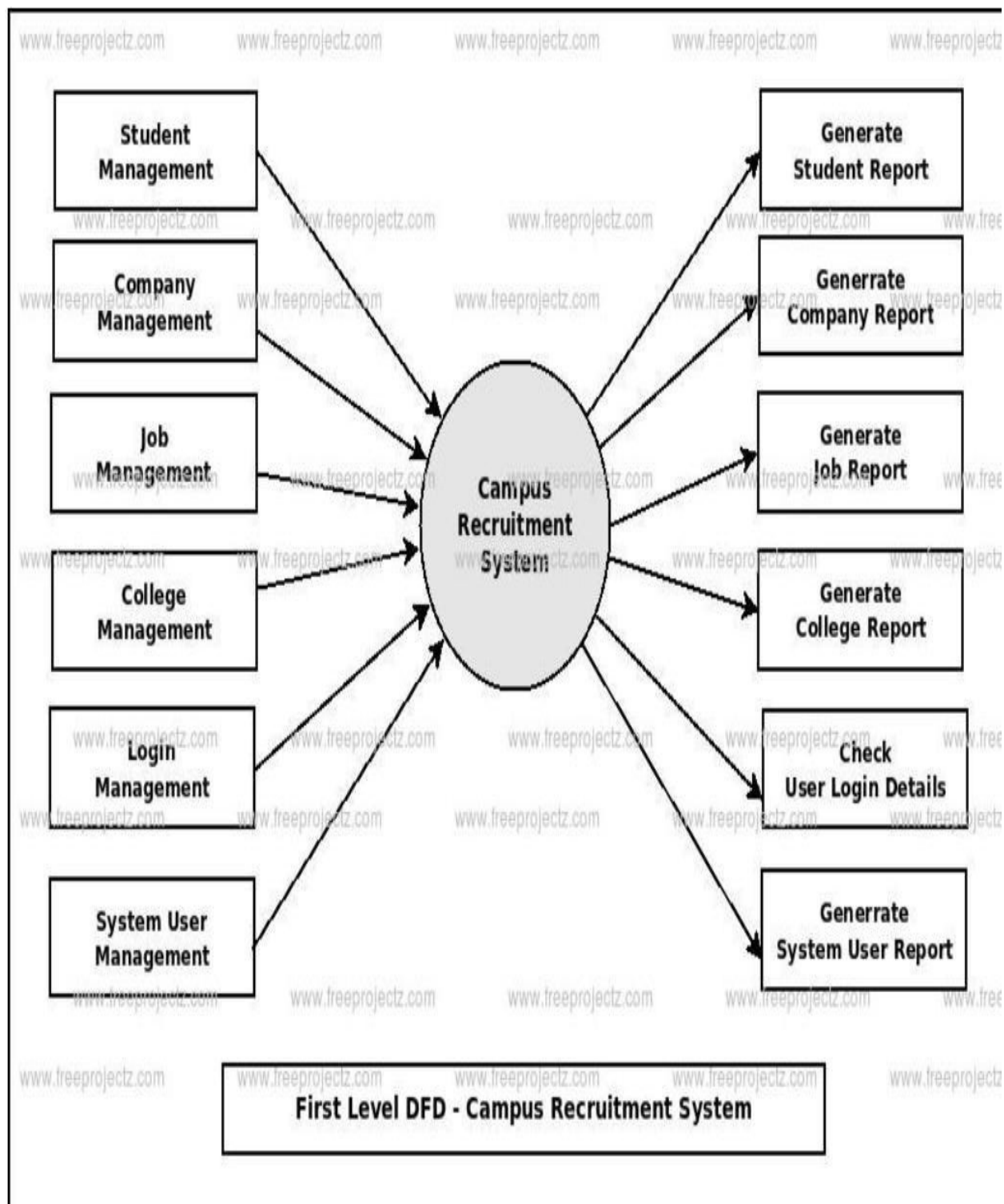


Circle, which represents a process that transforms incoming data into outgoingflow.

DATA FLOW DIAGRAM (LEVEL 0)



DATA FLOW DIAGRAM (LEVEL 1):





7.2 ER DIAGRAM

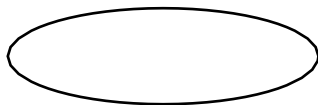
In software engineering an entity–relationship model (ER model) is data model for describing the data or information aspects of a business domain or its process requirements, in an abstract way that lends itself to ultimately being implemented in databases such as relational databases. The main components of ER models are entities (things) and the relationships that can exist among them.

Entity–relationship modelling was developed by Peter Chen and published in a 1976 paper. However, variants of the idea existed previously, and have been devised subsequently such as supertype and subtype data entities and commonality relationships.

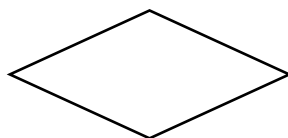
The overall logical structure of a database can be expressed graphically by an ER diagram. While drawing the ER diagram, entity names are represented by a rectangle, relationships are represented by diamond and oval shapes are used for representing attribute



Rectangle: Represents entity set.



Oval: Represents attribute.

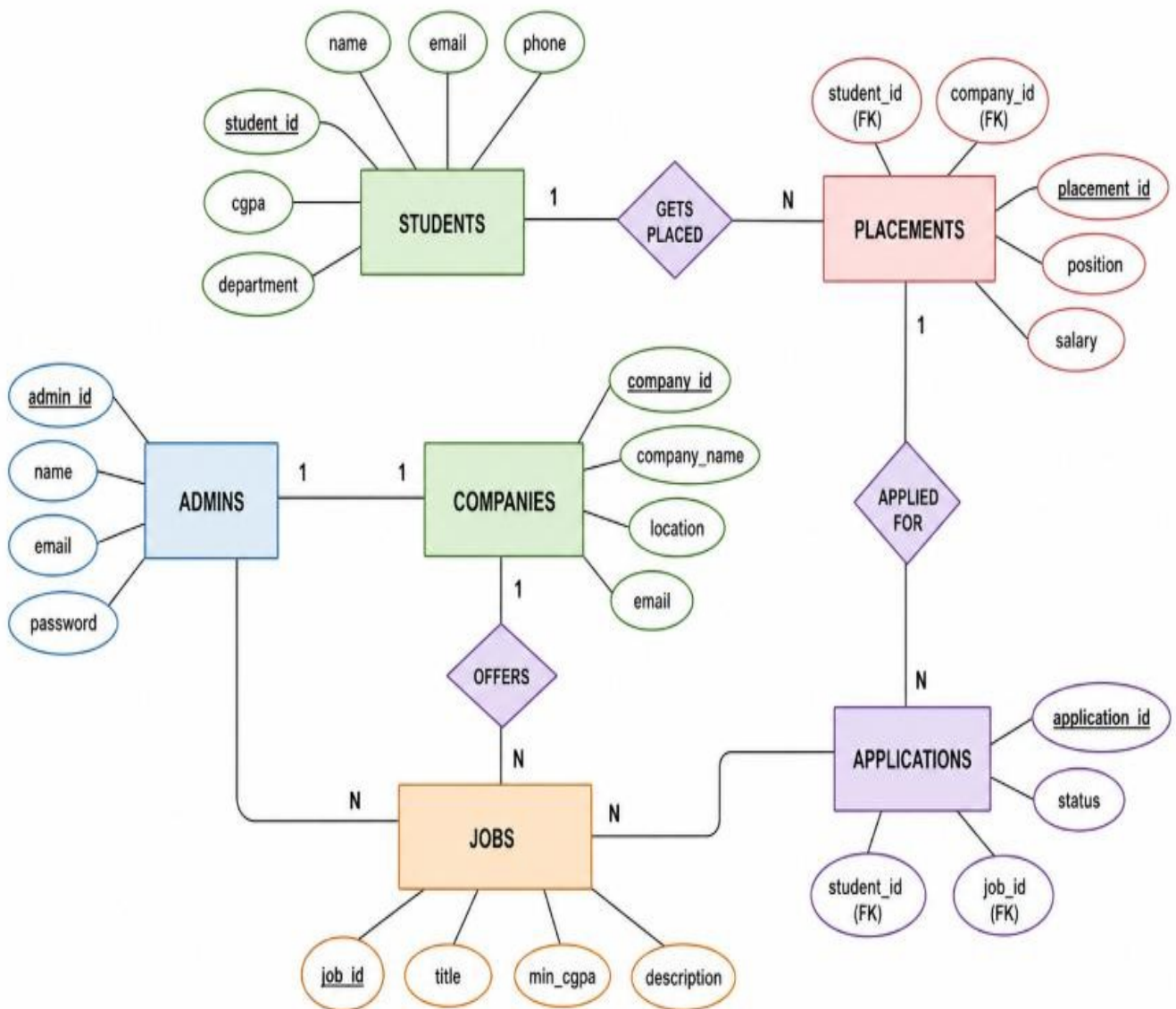


Diamond: Represents relationships entity sets.



Line: links attributes to entity sets and entity set relationships.

A flow chart visually represents the step-by-step process and decision points within the Campus Placement Management System software application.



7.3 UML DIAGRAM

A UML diagram is a diagram based on the UML (Unified Modeling Language) with the purpose of visually representing a system along with its main actors, roles, actions, artifacts or classes, in order to better understand, alter, maintain, or document information about the system.

UML (Unified Modeling Language) diagrams provide a standardized notation for visualizing the structure, behavior, and relationships within a software application. Emotion Sens



application:

7.4 DATABASE DESIGN

Introduction

Database Design is a crucial phase in the development of the CareerAI Hub: Campus Placement Management System. It defines how data is organized, stored, and managed within the system. A well-designed database ensures data consistency, minimizes redundancy, improves performance, and supports efficient retrieval of information. The system uses MySQL as the backend database management system to store and manage placement-related data.

The database is designed using a relational model where data is organized into multiple interconnected tables. Relationships between tables are established using Primary Keys and Foreign Keys to maintain data integrity and consistency.



Objectives of Database Design

- 7.4.1 Store student, recruiter, and placement information efficiently.
- 7.4.2 Eliminate data redundancy and duplication.
- 7.4.3 Maintain data integrity and consistency.
- 7.4.4 Support secure and fast data retrieval.
- 7.4.5 Facilitate relationship management between different entities.
- 7.4.6 Enable efficient placement tracking and reporting.

Main Database Tables Student

Table

Stores information related to registered students.

Field Name	Data Type	Description
student_id	INT (PK)	Unique Student ID
name	VARCHAR(100)	Student Name
email	VARCHAR(100)	Email Address
phone	VARCHAR(15)	Contact Number
department	VARCHAR(50)	Branch/Department
cgpa	DECIMAL(3,2)	Academic CGPA
backlogs	INT	Number of Active Backlogs
password	VARCHAR(255)	Login Password



2.Admin Table

Stores administrator details.

Field Name	Data Type	Description
admin_id	INT (PK)	Unique Admin ID
name	VARCHAR(100)	Administrator Name
email	VARCHAR(100)	Email Address
password	VARCHAR(255)	Login Password

3.Company Table

Stores recruiter/company information.

Field Name	Data Type	Description
company_id	INT (PK)	Unique Company ID
company_name	VARCHAR(100)	Company Name
location	VARCHAR(100)	Company Location
email	VARCHAR(100)	Company Email
industry	VARCHAR(100)	Industry Type



4.Placement Drive Table

Stores information about placement drives.

Field Name	Data Type	Description
drive_id	INT (PK)	Unique Drive ID
company_id	INT (FK)	Company Reference
job_role	VARCHAR(100)	Offered Position
package	DECIMAL(10,2)	Salary Package
min_cgpa	DECIMAL(3,2)	Minimum CGPA Required
max_backlogs	INT	Maximum Allowed Backlogs
drive_date	DATE	Placement Drive Date

5.Application Table

Stores student job applications.

Field Name	Data Type	Description
application_id	INT (PK)	Unique Application ID
student_id	INT (FK)	Student Reference
drive_id	INT (FK)	Drive Reference
application_date	DATE	Date of Application
status	VARCHAR(50)	Applied, Shortlisted, Selected, Rejected



6.Notification Table

Stores placement-related notifications.

Field Name	Data Type	Description
notification_id	INT (PK)	Notification ID
title	VARCHAR(100)	Notification Title
message	TEXT	Notification Content
date_posted	DATE	Date of Notification

Relationship Between Tables

- One Admin can manage multiple companies and placement drives.
- One Company can conduct multiple placement drives.
- One Placement Drive can receive applications from many students.
- One Student can apply for multiple placement drives.
- The Application table acts as a bridge between Students and Placement Drives.
- Notifications are generated and managed by administrators for all users.



8. TESTING

Testing is an important phase in the Software Development Life Cycle (SDLC). It is performed to identify errors, verify system functionality, and ensure that the developed software meets the specified requirements. The CareerAI Hub: Campus Placement Management System was thoroughly tested to ensure reliability, security, accuracy, and efficiency. Testing helps improve software quality by detecting defects before deployment and ensuring smooth interaction between students, recruiters, and placement administrators.

The main objectives of testing are:

- To verify that all system functionalities work correctly.
- To identify and eliminate software defects.
- To ensure accurate data processing and storage.
- To validate system security and access control.
- To evaluate system performance under different workloads.
- To ensure user satisfaction and system reliability.

The testing process included System Testing, Black Box Testing, White Box Testing, Unit Testing, Integration Testing, Validation Testing, and Performance Testing.

8.1 SYSTEM TESTING

System Testing is performed after integrating all modules of the application. The complete system is tested as a whole to verify that it satisfies the specified requirements and performs efficiently.

The major areas covered during system testing include:

Functional Testing

Ensures that all system features operate according to user requirements. Functional testing validates student registration, login, recruiter management, placement drive creation, eligibility verification, job application processing, and report generation.

Browser Compatibility Testing

The application was tested on multiple web browsers including Google Chrome, Mozilla Firefox, and Microsoft Edge to ensure consistent functionality and user experience.

Database Testing

Database operations such as data insertion, retrieval, updating, and deletion were tested to verify data integrity and consistency.



Security Testing

Authentication and authorization mechanisms were tested to ensure that only authorized users can access system resources. Password protection and session management were also verified.

Usability Testing

The user interface was evaluated to ensure easy navigation, readability, responsiveness, and overall user satisfaction.

Performance Testing

The system was tested under different workloads to measure response time, processing speed, and system stability.

8.2 BLACK BOX TESTING

Black Box Testing is a software testing technique that evaluates system functionality without examining the internal program structure or source code. Test cases are designed based on system requirements and expected outputs.

In CareerAI Hub, Black Box Testing was performed on:

- Student Registration and Login
- Admin Authentication
- Recruiter Registration
- Placement Drive Creation
- Student Eligibility Verification
- Job Application Submission
- Interview Scheduling
- Notification Generation
- Dashboard Analytics
- Report Generation

The primary objective of Black Box Testing is to ensure that the system behaves correctly from the user's perspective and produces accurate outputs for valid inputs.

Advantages of Black Box Testings

- Does not require programming knowledge.
- Focuses on user requirements.



- Identifies missing functionalities.
- Detects interface-related errors.
- Ensures user satisfaction.

8.3 TEST ITEMS

The test items represent the major components and functionalities of the CareerAI Hub: Campus Placement Management System that were selected for testing. These components were tested to ensure correct functionality, reliability, security, and smooth interaction between different modules of the system.

1.Student Module

The Student Module was tested to verify that students can successfully access and use all placement-related features.

Test Items:

- Student Registration
- Student Login and Logout
- Profile Creation and Management
- Resume Upload
- Job Application Submission
- Application Status Tracking
- Notification Viewing
- Password Reset

2.Recruiter Module

The Recruiter Module was tested to ensure that companies can efficiently manage recruitment activities.

Test Items:

- Company Registration
- Recruiter Login and Authentication
- Company Profile Management
- Placement Drive Creation
- Placement Drive Management



- Eligibility Criteria Definition
- Applicant Review
- Candidate Shortlisting
- Candidate Selection
- Interview Scheduling

3.Administrator Module

The Administrator Module was tested to verify the overall management and monitoring functionalities of the placement system.

Test Items:

- Admin Login and Authentication
- Student Verification
- Student Record Management
- Company Management
- Placement Drive Approval
- Notification Management
- Eligibility Verification
- Placement Monitoring
- Report Generation
- Analytics Dashboard

4.Database Module

The Database Module was tested to ensure accurate storage, retrieval, and maintenance of placement-related information.

Test Items:

- Data Storage
- Data Retrieval
- Data Updating
- Data Deletion
- Data Validation
- Backup and Recovery



- Query Performance

5.Security Module

The Security Module was tested to verify the protection of user accounts and confidential placement data.

Test Items:

- User Authentication
- Role-Based Authorization
- Session Management
- Password Encryption
- Access Control
- Invalid Login Handling
- Session Timeout
- Protection Against Unauthorized Access

Testing Objective

The objective of testing these items was to ensure that all modules of the CareerAI Hub: Campus Placement Management System function correctly, securely, and efficiently. The testing process verified that the system meets functional requirements, maintains data integrity, provides secure access, and delivers a reliable user experience for students, recruiters, and administrators.

8.4 TEST PLAN

The test plan provides a structured approach for testing the CareerAI Hub system. The testing process included:

- Identification of modules to be tested.
- Preparation of test cases and expected outputs.
- Execution of individual module testing.
- Integration testing of interconnected modules.
- Security and performance testing.
- Defect identification and correction.
- Re-testing after modifications.
- Final system validation.



The testing environment consisted of:

- Operating System: Windows 11
- Web Server: XAMPP
- Programming Language: PHP 8.2
- Database: MySQL
- Browser: Google Chrome

8.5 WHITE BOX TESTING

White Box Testing focuses on the internal structure, logic, and source code of the application. Developers analyze program flow, conditions, loops, and database interactions to ensure that all code paths are executed correctly.

The following aspects were tested:

- Conditional Statements
- Loops and Iterations
- Form Validation Logic
- Session Handling
- Database Queries
- Error Handling Procedures
- Input Validation Mechanisms

Benefits of White Box Testing

- Improves code quality.
- Identifies hidden programming errors.
- Verifies logical correctness.
- Enhances system security.
- Ensures complete code coverage.

Result

White Box Testing conducted on the CareerAI Hub system confirmed that the internal logic, database operations, authentication mechanisms, and application workflows functioned correctly. All critical code paths were executed successfully, validation rules operated as expected, and database interactions produced accurate results. The testing process helped improve code quality, security, and system reliability, ensuring the successful operation of the placement management system.



8.5 Level of Testing

UNIT TESTING

Unit Testing focuses on testing individual modules independently. Each module was tested separately before integration.

Modules tested include:

- Student Registration Module
- Login Module
- Profile Management Module
- Recruiter Module
- Placement Drive Module
- Application Module
- Notification Module
- Report Module

Objectives of Unit Testing

- Verify module functionality.
- Detect coding errors early.
- Ensure correct input-output processing.
- Validate business logic implementation.

INTEGRATION TESTING

Integration Testing verifies the interaction between multiple modules and ensures proper communication among system components.

The following integrations were tested:

- Frontend and Backend Integration
- PHP and MySQL Database Integration
- Student and Application Modules
- Recruiter and Placement Drive Modules
- Dashboard and Analytics Modules
- Notification and User Modules



Integration Strategies

Top-Down Integration

Testing begins from higher-level modules and gradually integrates lower-level modules.

Bottom-Up Integration

Testing begins with lower-level modules and gradually integrates higher-level modules.

Benefits of Integration Testing

- Detects interface defects.
- Verifies data flow between modules.
- Ensures smooth communication among components.

VALIDATION TESTING

Validation Testing ensures that the developed software satisfies user requirements and performs according to expectations.

Alpha Testing

Alpha Testing was conducted internally by developers and project supervisors. The objective was to identify defects before releasing the system to actual users.

Beta Testing

Beta Testing was conducted by selected students and placement coordinators. Their feedback helped improve system functionality, usability, and performance.

Benefits of Validation Testing

- Improves user satisfaction.
- Ensures requirement compliance.
- Enhances software quality.
- Reduces post-deployment issues.

PERFORMANCE TESTING

Performance Testing was conducted to evaluate the speed, responsiveness, stability, and efficiency of the CareerAI Hub: Campus Placement Management System under different operating conditions. The objective of performance testing is to ensure that the system performs effectively when handling multiple users, large volumes of data, and various placement-related operations. Response Time Testing

Measures the time required for the system to process requests and display results.



Load Testing

Load Testing evaluates system performance when multiple users access the application simultaneously. The system was tested with concurrent student, recruiter, and administrator activities to ensure stable operation.

Activities Tested:

- Multiple student logins
- Simultaneous job applications
- Concurrent recruiter access
- Multiple report requests

Objectives:

- Measure system performance under normal workload.
- Ensure consistent response times.
- Verify system reliability during placement drives.

Stress Testing

Stress Testing determines how the system behaves under extreme workload conditions beyond normal operating capacity. The objective is to identify the breaking point of the system and evaluate its recovery capabilities.

Activities Tested:

- Large numbers of simultaneous users
- Bulk job application submissions
- Heavy database transactions
- Continuous report generation requests

Objective:

- Evaluate system stability under peak load.
- Identify performance bottlenecks.
- Ensure graceful recovery from failures.

Database Performance Testing

Database Performance Testing measures the efficiency of database operations, including data insertion, retrieval, updating, and deletion. Since the system handles large amounts of student and recruiter data, database performance is critical.



Activities Tested:

- Student record retrieval
- Recruiter data access
- Placement drive information retrieval
- Application status updates
- Report generation queries

Objective:

- Optimize database query execution.
- Reduce data retrieval time.
- Ensure efficient transaction processing.

Scalability Testing

Ensures that the system can support future growth in the number of users and records.

Results

- Fast retrieval of student and company data.
- Efficient application processing.
- Stable operation under concurrent user access.
- Quick generation of reports and analytics.

8.6 ITEM PASS/FAIL CRITERIA

The pass/fail criteria were established based on the expected behavior and functional requirements of the CareerAI Hub: Campus Placement Management System. Each test item was evaluated by comparing the actual output produced by the system with the expected output defined in the test cases.

Pass Criteria

A test case is considered **Passed** when:

- The system produces the expected output for valid input data.
- Student registration, login, and profile management functions operate correctly.
- Recruiters can successfully create and manage placement drives.
- Administrators can manage students, companies, and placement activities without errors.



- Eligibility verification is performed accurately based on predefined criteria.
- Job applications are submitted and stored successfully in the database.
- Notifications and alerts are generated correctly.
- Reports and analytics are generated accurately.
- Database operations such as insertion, retrieval, updating, and deletion execute successfully.
- Security features including authentication, authorization, and session management function properly.
- The system responds within acceptable performance limits.
- No critical errors, crashes, or data inconsistencies occur during execution.

Fail Criteria

A test case is considered **Failed** when:

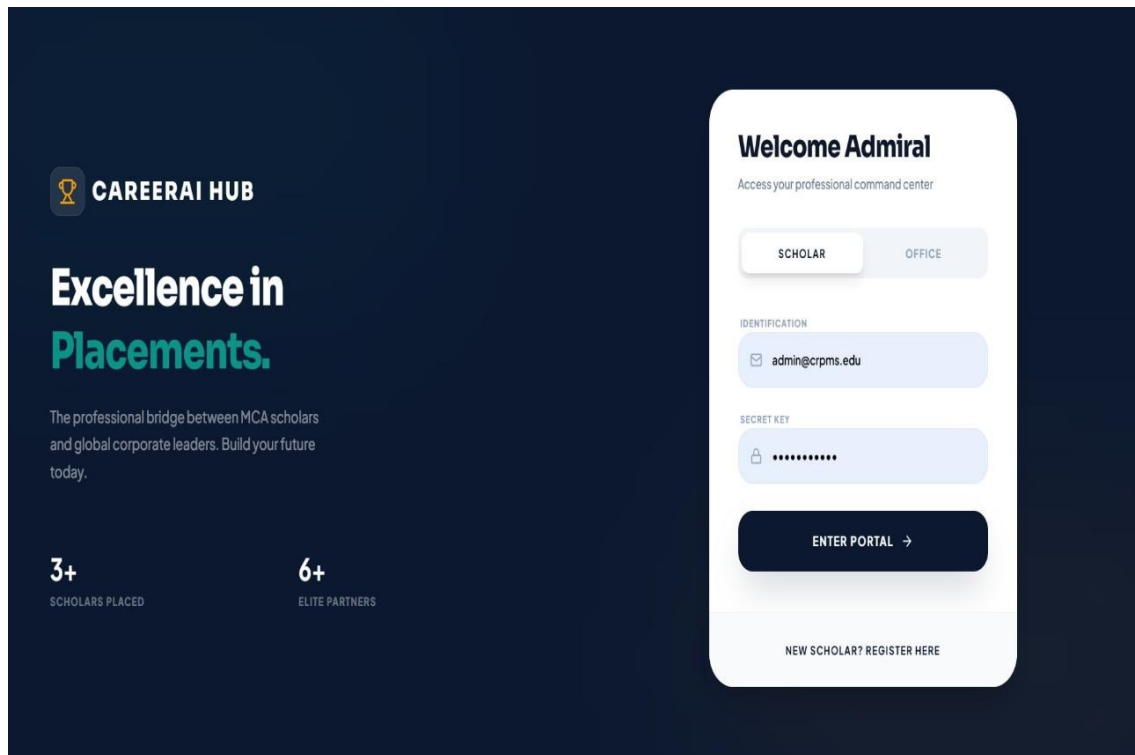
- The actual output differs from the expected output.
- Invalid data is accepted without proper validation.
- User authentication or authorization mechanisms fail.
- Student, recruiter, or administrator operations do not function as intended.
- Database transactions produce incorrect or incomplete results.
- Job applications are not processed correctly.
- Reports or analytics contain inaccurate information.
- Notifications are not delivered or displayed properly.
- System crashes, hangs, or generates unexpected errors.
- Security vulnerabilities allow unauthorized access.
- Response time exceeds acceptable limits.
- Data loss, duplication, or inconsistency is observed.

Evaluation Method

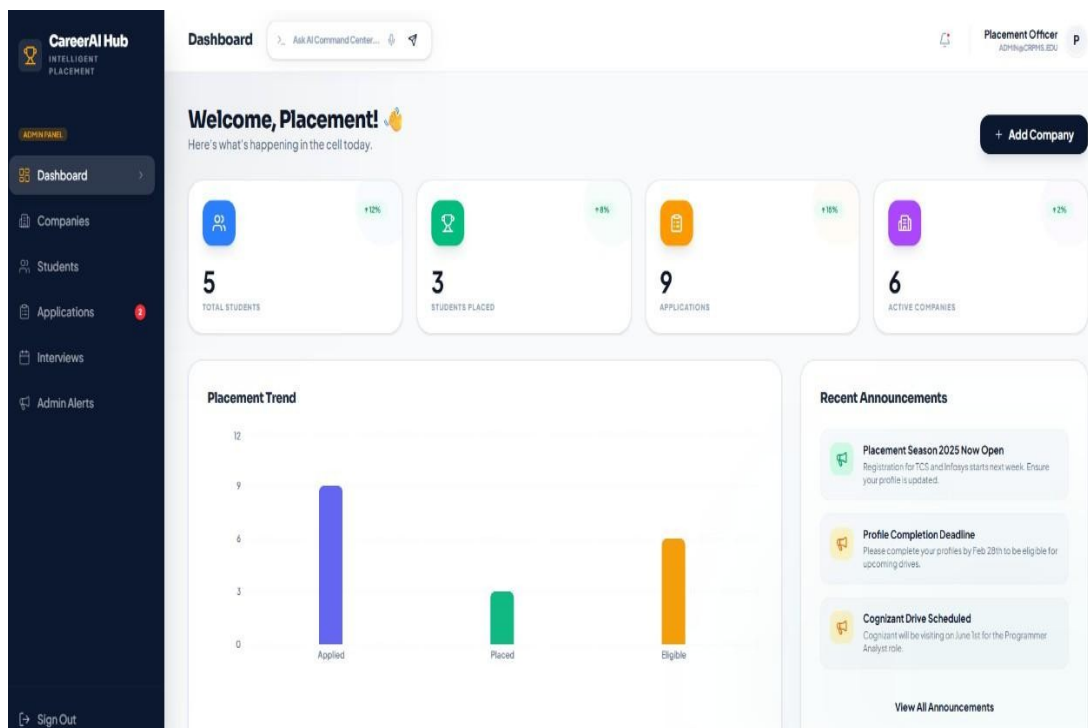
Each test case was executed using predefined input values and expected results. The obtained results were recorded and compared with the expected outcomes. Test cases that met all specified requirements were marked as **Pass**, while those that failed to meet one or more requirements were marked as **Fail** and subjected to corrective actions and re-testing.



Output Screen



Dashboard





Companies List

CareerAI Hub
INTELLIGENT
PLACEMENT

ADMIN PANEL

[Dashboard](#)

[Companies](#)

[Students](#)

[Applications](#)

[Interviews](#)

[Admin Alerts](#)

[Sign Out](#)

Companies

Ask AI Command Center...

Placement Officer
ADMIN@CPSHS.EDU

Corporate Partners

Manage recruitment drives and eligibility

+ Add Company

T

TCS

SYSTEM ENGINEER 3.5 LPA

DRIVE DATE
15 May, 2025

I

Infosys

SYSTEMS ENGINEER 3.6 LPA

DRIVE DATE
20 May, 2025

W

Wipro

PROJECT ENGINEER 3.5 LPA

DRIVE DATE
25 May, 2025

C

Cognizant

PROGRAMMER ANALYST 4.0 LPA

DRIVE DATE
01 Jun, 2025

C

Contrex

DATA ANALYST 5.5 LPA

DRIVE DATE
23 Apr, 2026

G

Google

DRIVE DATE

Students Database

CareerAI Hub
INTELLIGENT
PLACEMENT

ADMIN PANEL

[Dashboard](#)

[Companies](#)

[Students](#)

[Applications](#)

[Interviews](#)

[Admin Alerts](#)

[Sign Out](#)

Students

Ask AI Command Center...

Placement Officer
ADMIN@CPSHS.EDU

Student Database

Total Students: 5

SCHOLAR	PROGRAM	CGPA	STATUS
R Rahul Kumar MCA2301	B.Sc Computer Science MCA	8.2	TAKE INTERVIEW PLACED
A Arjun Singh MCA2303	B.Sc Mathematics MCA	6.9	TAKE INTERVIEW SEARCHING
T Tarun Kumar 24FO1267	BCA MCA	9	TAKE INTERVIEW PLACED
A Adarsh 24FO1269	MCA MCA	92	TAKE INTERVIEW PLACED
T Tarun 24FO127	MCA MCA	90	TAKE INTERVIEW SEARCHING



Manage Application

 CareerAI Hub
INTELLIGENT
PLACEMENT

ADMIN PANEL

Dashboard

Companies

Students

Applications 1

Interviews

Admin Alerts

Sign Out

Applications

Ask AI Command Center...

Placement Officer
ADMIN@CPSYS.EDU

Manage Applications

Total: 9

STUDENT	COMPANY & ROLE	STATUS	ACTIONS
R Rahul Kumar MCA2301	Infosys Systems Engineer	SELECTED	DECISION MADE
R Rahul Kumar MCA2301	Cognizant Programmer Analyst	SELECTED	DECISION MADE
T Tarun Kumar 24PO1267	Wipro Project Engineer	SELECTED	DECISION MADE
A Adarsh 24PO1219	Contrex Data Analyst	SELECTED	DECISION MADE
A Adarsh 24PO1219	Wipro Project Engineer	SELECTED	DECISION MADE
A Adarsh 24PO1219	TCS System Engineer	SELECTED	DECISION MADE
A Adarsh 24PO1219	Infosys Systems Engineer	REJECTED	DECISION MADE
A Adarsh 24PO1219	Google Software Engineer Intern	PENDING REVIEW	

Interviews

 CareerAI Hub
INTELLIGENT
PLACEMENT

ADMIN PANEL

Dashboard

Companies

Students

Applications

Interviews 1

Admin Alerts

Sign Out

Interviews

Ask AI Command Center...

Placement Officer
ADMIN@CPSYS.EDU

Upcoming Interviews

+ Schedule Interview

Adarsh
TCS - System Engineer

CANCEL

23 Apr, 2026
10:00 AM



Admin Alerts

INTELLIGENT
PLACEMENT

ADMIN PANEL

Dashboard

Companies

Students

Applications

Interviews

Admin Alerts

Sign Out

Announcements

Ask AI Command Center...

Placement Officer
ADRESH@PHS.EDU

p

Platform Announcements

+ New Announcement

SUCCESS 23 Apr, 2024 — 18:40

Placement Season 2025 Now Open
Registration for TCS and Infosys starts next week. Ensure your profile is updated.

WARNING 23 Apr, 2024 — 18:40

Profile Completion Deadline
Please complete your profiles by Feb 28th to be eligible for upcoming drives.

INFO 23 Apr, 2024 — 18:40

Cognizant Drive Scheduled
Cognizant will be visiting on June 1st for the Programmer Analyst role.

User Dashboard

INTELLIGENT
PLACEMENT

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

Dashboard

Ask AI Command Center...

Adarsh
ADARSH@PHS.EDU

A

Welcome, Adarsh!

Here's what's happening in the cell today.

92
CURRENT CGPA

6
ELIGIBLE COMPANIES

6
APPLICATIONS SENT

Placed
PLACEMENT STATUS

Placement Trend

Category	Value
Applied	9
Placed	3
Eligible	6

Recent Announcements

Placement Season 2025 Now Open
Registration for TCS and Infosys starts next week. Ensure your profile is updated.

Profile Completion Deadline
Please complete your profiles by Feb 28th to be eligible for upcoming drives.

Cognizant Drive Scheduled
Cognizant will be visiting on June 1st for the Programmer Analyst role.

[View All Announcements](#)



Apply For Job

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

Jobs

Ask AI Command Center...

Adarsh
ADARSH@GMAIL.COM

Search roles or companies...

T

TCS

SYSTEM ENGINEER

PACKAGE

3.5 LPA

LOCATION

Chennai

DRIVE DATE

15 May

Eligibility Criteria: CGPA 9+

AI Match Score

ANALYZE MATCH

Application Sent

I

Infosys

SYSTEMS ENGINEER

PACKAGE

3.6 LPA

LOCATION

Bangalore

DRIVE DATE

20 May

Eligibility Criteria: CGPA 7.5+

AI Match Score

ANALYZE MATCH

Application Sent

W

Wipro

PROJECT ENGINEER

PACKAGE

3.5 LPA

LOCATION

Hyderabad

DRIVE DATE

25 May

Eligibility Criteria: CGPA 6+

AI Match Score

ANALYZE MATCH

Application Sent

C

Cognizant

PROGRAMMER ANALYST

PACKAGE

4.0 LPA

LOCATION

Chennai

DRIVE DATE

01 Jun

Eligibility Criteria: CGPA 9+

AI Match Score

ANALYZE MATCH

Application Sent

C

Contrex

DATA ANALYST

PACKAGE

5.5 LPA

LOCATION

Mumbai

DRIVE DATE

23 Apr

Eligibility Criteria: CGPA 7.5+

AI Match Score

ANALYZE MATCH

Application Sent

G

Google

SOFTWARE ENGINEER INTERN

PACKAGE

1.5 Lakhs per month

LOCATION

Hyderabad

DRIVE DATE

23 Apr

Eligibility Criteria: CGPA 6+

AI Match Score

ANALYZE MATCH

Application Sent

Interview Mock

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

Mock-Interview

Ask AI Command Center...

Adarsh
ADARSH@GMAIL.COM

AI MOCK INTERVIEW

Practice your technical skills with our high-fidelity AI interviewer.

T

TCS

System Engineer

I

Infosys

Systems Engineer

W

Wipro

Project Engineer

C

Cognizant

Programmer Analyst

ENTER INTERVIEW ROOM



Resume Preparation

**CareerAI Hub**
INTELLIGENT
PLACEMENT

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

Resume

Ask AI Command Center...

Adarsh

ADARSH@GMAIL.COM

A

INTERACTIVE RESUME BUILDER
Build your professional profile for recruiters

Preview Resume

Cancel

Save Professional Profile

Personal & Contact Info

PHONE NUMBER
9220343178

COLLEGE / INSTITUTION
Dept of Computer Applications

DEPARTMENT
MCA

ABOUT YOU (SUMMARY)
My name is adarsh am intrested in Data analyst

Academic Details

UG DEGREE
MCA

UG CGPA
92

10TH %
95

12TH %
94

BACKLOGS
0

BATCH (PASS OUT)
2025

SKILLS (COMMA SEPARATED)
React ,Python,Java,SQL

Placement Inbox

**CareerAI Hub**
INTELLIGENT
PLACEMENT

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

Inbox

Ask AI Command Center...

Adarsh

ADARSH@GMAIL.COM

A

Placement Inbox

Adarsh'S Mailbox

**Update on your application for Infosys**
From: Placement Cell • 15 Jun, 20:22

Dear Adarsh, Thank you for your interest in the Systems Engineer role at Infosys. Unfortunately, we will not be moving forward with your application at this time. We wish you the best in your future endeavors.

**Congratulations! You are selected at TCS**
From: Placement Cell • 12 Jun, 13:18

Dear Adarsh, We are pleased to inform you that you have been SELECTED for the System Engineer role at TCS with a package of 3.5 LPA. Please check the portal for next steps.

**Interview Call: TCS**
From: Placement Cell • 23 Apr, 20:43

Dear Adarsh,

Congratulations! You have been shortlisted for an interview with TCS for the System Engineer position.

Schedule Details:
Date: 2026-04-23
Time: 10:00 AM

Please be prepared and available at the scheduled time.

Best of luck,
Placement Cell



Applications

CareerAI Hub
INTELLIGENT
PLACEMENT

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

My Apps

Ask AI Command Center...

Adarsh
ADARSH@CPAL.COM

Application History

COMPANY	APPLIED DATE	STATUS	NOTES
 Contrex DATA ANALYST	23 Apr, 2026	SELECTED	Awaiting review
 Wipro PROJECT ENGINEER	23 Apr, 2026	SELECTED	Awaiting review
 TCS SYSTEM ENGINEER	12 Jun, 2026	SELECTED	Awaiting review
 Infosys SYSTEMS ENGINEER	12 Jun, 2026	REJECTED	Better Luck Next Time
 Google SOFTWARE ENGINEER INTERN	12 Jun, 2026	UNDER REVIEW	Awaiting review
 Cognizant PROGRAMMER ANALYST	12 Jun, 2026	UNDER REVIEW	Awaiting review

Profile

CareerAI Hub
INTELLIGENT
PLACEMENT

STUDENT PANEL

Dashboard

Apply for Jobs

AI Interview Mock

Resume Ready

Placement Inbox

My Applications

My Profile

Sign Out

Profile

Ask AI Command Center...

Adarsh
ADARSH@CPAL.COM

Adarsh
24PO1219

PLACED

Academic Profile

US CGPA
92

10TH PERCENTAGE
95%

12TH PERCENTAGE
94%

ACTIVE BACKLOGS
0

BATCH
2025

DEGREE
MCA

Contact Details

adarsh@gmail.com

Dept of Computer Applications

Resume Ready?

Keep your professional files updated for faster applications.

UPDATE RESUME

About Me

My name is adarsh am intrested in Data analyst

React

Python

Java

SQL

58



9.1 SYSTEM IMPLEMENTATION CODING

Introduction

System Implementation Coding is the process of translating the system design into executable program code. It is one of the most important phases in software development because the actual functionality of the application is built during this stage. In the CareerAI Hub: Campus Placement Management System, coding was performed using modern web technologies such as HTML5, CSS3, JavaScript (ES6), Tailwind CSS, PHP 8.2, MySQL, Node.js, and Express.js. The coding process focused on developing a secure, efficient, scalable, and user-friendly system that automates placement activities such as student registration, recruiter management, placement drive scheduling, application tracking, eligibility verification, and report generation. A modular programming approach was adopted to improve maintainability and facilitate future enhancements.

Objectives of System Implementation Coding

- The main objectives of coding are:
- To convert system design into a working application.
- To automate campus placement operations.
- To ensure secure data processing and storage.
- To provide a responsive and user-friendly interface.
- To minimize manual work and human errors.
- To maintain data consistency and integrity.
- To support future modifications and upgrades.

Frontend Implementation

The frontend of CareerAI Hub was developed using HTML5, CSS3, JavaScript (ES6), and Tailwind CSS. These technologies were used to create responsive, interactive, and visually appealing user interfaces.

Frontend Functionalities

Student Interface

- Student Registration
- Student Login
- Profile Management
- Resume Upload
- Placement Drive Viewing
- Job Application Submission
- Application Status Tracking
- Notifications Display

Recruiter Interface

- Recruiter Login
- Company Profile Management
- Placement Drive Creation
- Candidate Review
- Selection Status Updates



Administrator Interface

- Student Verification
- Recruiter Management
- Placement Drive Management
- Notification Broadcasting
- Dashboard Analytics
- Report Generation

Benefits of Frontend Coding

- Improved user experience.
- Responsive design for multiple devices.
- Easy navigation and accessibility.
- Faster interaction with the system.

Backend Implementation

The backend was implemented using PHP 8.2, Node.js, and Express.js. The backend handles all server-side operations and business logic.

Backend Functionalities

User Authentication

- Registration Validation
- Login Verification
- Session Management
- Password Security

Placement Management

- Placement Drive Creation
- Eligibility Verification
- Application Processing
- Interview Scheduling

Notification Management

- Placement Announcements
- Schedule Updates
- Selection Notifications

Report Generation

- Student Reports
- Placement Statistics
- Company Reports
- Placement Analytics

Benefits of Backend Coding

- Efficient processing of user requests.
- Improved application security.



- Better system performance.
- Centralized business logic implementation.

Database Implementation

The database was developed using MySQL and managed through phpMyAdmin. The database stores all information related to students, recruiters, placement drives, applications, and notifications.

Major Database Tables Student Table

Stores student details such as:

- Student ID
- Name
- Email
- Department
- CGPA
- Backlogs

Recruiter Table

Stores recruiter information such as:

- Company Name
- Industry Type
- Contact Information
- Eligibility Criteria

Placement Drive Table

Stores:

- Drive Details
- Job Roles
- Package Information
- Interview Dates

Application Table

Stores:

- Student Applications
- Application Status
- Selection Results

Notification Table

Stores:

- Announcements
- Placement Updates
- Schedule Information



Database Operations

The following SQL operations were implemented:

- INSERT
- SELECT
- UPDATE
- DELETE
- JOIN Operations
- Aggregate Queries

Coding Methodology

The following coding practices were adopted:

Modular Programming

The application was divided into separate modules such as:

- Student Module
- Recruiter Module
- Placement Drive Module
- Notification Module
- Report Module

This improves code maintainability and reduces complexity.

Reusability

The CareerAI Hub: Campus Placement Management System was developed with a focus on code reusability to improve efficiency and maintainability. Reusable functions, classes, and components were created to minimize code duplication and simplify future enhancements. Common functionalities such as user authentication, database connectivity, input validation, session management, and notification handling were implemented as reusable modules that can be utilized across different parts of the system. This approach reduces development time, improves consistency, and facilitates easier maintenance and scalability.

Error Handling

Effective error handling mechanisms were implemented throughout the application to ensure system reliability and provide a smooth user experience. The system detects, manages, and responds appropriately to unexpected situations without causing application failure.

Code Documentation

Proper code documentation was maintained throughout the development process to improve readability, maintainability, and future enhancements. Comments were added to explain complex logic, functions, classes, and database operations. Documentation helps developers understand the purpose and functionality of different code sections, making future modifications easier and reducing maintenance effort. Well-documented code also supports collaborative development and knowledge transfer among team members.

Security Implementation

Security is one of the most critical aspects of the CareerAI Hub system because it manages sensitive information related to students, recruiters, placement drives, and recruitment results. Multiple security mechanisms were incorporated to ensure confidentiality, integrity, and



availability of data.

Authentication and Authorization

Authentication mechanisms verify the identity of users before granting access to the system. Authorization controls determine which resources and functionalities users can access based on their assigned roles.

Features:

- Secure login system.
- Role-based user access.
- Restricted access to sensitive modules.
- Protection against unauthorized system usage.

Input Validation

All user inputs are validated before processing to ensure that only valid and properly formatted data is accepted by the system.

Validation Checks Include:

- Mandatory field verification.
- Email format validation.
- Password strength verification.
- Numeric and text field validation.
- Prevention of malicious input submissions.

Benefits:

- Maintains data integrity.
- Prevents application errors.
- Enhances system security.

Session Management

Session management is implemented to maintain secure user authentication during application usage. Once authenticated, users are assigned secure sessions that remain active until logout or session expiration.

Features:

- Secure session creation.
- Automatic session timeout.
- Logout functionality.
- Prevention of unauthorized session access.

Benefits:

- Protects user accounts.
- Improves application security.
- Ensures controlled access to resources.

Database Security

Several database security measures were implemented to protect placement-related information from unauthorized access and attacks.

Security Techniques Used:

- Prepared Statements.
- Parameterized Queries.
- Data Validation.



- Access Restrictions.
- Secure Database Connectivity.

Benefits:

- Prevents SQL Injection attacks.
- Protects sensitive records.
- Maintains data consistency and integrity.

Password Protection

User passwords are protected using modern encryption and hashing techniques before being stored in the database.

Security Measures:

- Password hashing algorithms.
- Secure password storage.
- Password verification mechanisms.
- Protection against password theft.

Benefits:

- Enhances account security.
- Prevents unauthorized access.
- Protects user credentials.

Role-based access control ensures that users can only perform authorized actions.

Coding Challenges Faced

During implementation, several challenges were encountered:

- Managing large volumes of placement data.
- Maintaining database consistency.
- Implementing eligibility verification logic.
- Handling concurrent user access.
- Ensuring secure authentication.
- Optimizing application performance.

These challenges were resolved through efficient coding practices, database normalization, and extensive testing.

Advantages of System Implementation Coding

- Automates placement management activities.
- Reduces manual errors and paperwork.
- Provides accurate and reliable information.
- Improves communication among stakeholders.
- Enhances security and data protection.
- Supports scalability and future enhancements.
- Improves overall system efficiency.



CODING REPORT:

1. server.ts – Backend Core Logic

Important backend implementation including Express server setup, authentication, data initialization, API handling and placement management workflows.

server.ts – Key Implementation (Part A)

```
import express from 'express';
import path from 'path';
import { createServer as createViteServer } from 'vite';
import fs from 'fs';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import AdmZip from 'adm-zip';

const app = express();
const PORT = 3000;
const SECRET_KEY = 'crpms_secret_2024';
const DB_PATH = path.join(process.cwd(), 'database.json');

app.use(express.json());

// Initial Data Structure
const initialData =
{ admins: [
  { id: '1', name: 'Placement Officer', email: 'admin@crpms.edu', password:
bcrypt.hashSync('password123', 10) }
],
students: [
  {
    id: 's1',
    name: 'Rahul Kumar',
    roll_no: 'MCA2301',
    email: 'rahul@student.edu',
    password: bcrypt.hashSync('password123', 10),
    phone: '9876543210',
    college: 'Dept of Computer Applications',
    department: 'MCA',
    pct_10: 88,
    pct_12: 82,
    ug_cgpa: 8.2,
    ug_degree: 'B.Sc Computer Science',
    year_pass: 2025,
```



```
backlogs: 0,
skills: 'React, Node.js, SQL',
about: 'Enthusiastic developer.',
is_placed: false,
is_active: true
},
{
  id: 's2',
  name: 'Arjun Singh',
  roll_no: 'MCA2303',
  email: 'arjun@student.edu',
  password: bcrypt.hashSync('password123', 10),
  phone: '9876543212',
  college: 'Dept of Computer Applications',
  department: 'MCA',
  pct_10: 72,
  pct_12: 68,
  ug_cgpa: 6.9,
  ug_degree: 'B.Sc Mathematics',
  year_pass: 2025,
  backlogs: 1,
  skills: 'Python, Java',
  about: 'Passionate about coding.',
  is_placed: false,
  is_active: true
}
],
companies: [
  { id: 'c1', name: 'TCS', role: 'System Engineer', package: '3.5 LPA', min_cgpa: 6.0,
min_pct_10: 60, min_pct_12: 60, allow_backlog: false, drive_date: '2025-05-15', location:
'Chennai', job_type: 'Full Time', status: 'active', color: '#0d1b3e', letter: 'T' },
  { id: 'c2', name: 'Infosys', role: 'Systems Engineer', package: '3.6 LPA', min_cgpa: 6.5,
min_pct_10: 65, min_pct_12: 65, allow_backlog: false, drive_date: '2025-05-20', location:
'Bangalore', job_type: 'Full Time', status: 'active', color: '#007cc3', letter: 'I' },
  { id: 'c3', name: 'Wipro', role: 'Project Engineer', package: '3.5 LPA', min_cgpa: 6.0,
min_pct_10: 60, min_pct_12: 60, allow_backlog: true, drive_date: '2025-05-25', location:
'Hyderabad', job_type: 'Full Time', status: 'active', color: '#2e3192', letter: 'W' },
  { id: 'c4', name: 'Cognizant', role: 'Programmer Analyst', package: '4.0 LPA', min_cgpa:
7.0, min_pct_10: 70, min_pct_12: 65, allow_backlog: false, drive_date: '2025-06-01',
location: 'Chennai', job_type: 'Full Time', status: 'active', color: '#0033a0', letter: 'C' }
],
applications: [],
interviews: [],
announcements: [
```



```
{ id: '1', title: 'Placement Season 2025 Now Open', body: 'Registration for TCS and Infosys
starts next week. Ensure your profile is updated.', type: 'success', is_active: true, created_at:
new Date().toISOString() },
{ id: '2', title: 'Profile Completion Deadline', body: 'Please complete your profiles by Feb
28th to be eligible for upcoming drives.', type: 'warning', is_active: true, created_at: new
Date().toISOString() },
{ id: '3', title: 'Cognizant Drive Scheduled', body: 'Cognizant will be visiting on June 1st
for the Programmer Analyst role.', type: 'info', is_active: true, created_at: new
Date().toISOString() }
],
notifications: [
{ id: 'n1', title: 'Welcome to CRPMS', message: 'Your account has been successfully
created. Explore active jobs now!', type: 'ok', is_read: false, created_at: new
Date().toISOString() }
],
mails: []
};
```

```
// Ensure database exists
if(!fs.existsSync(DB_PATH)) {
  fs.writeFileSync(DB_PATH, JSON.stringify(initialData, null, 2));
}

const getDB = () => JSON.parse(fs.readFileSync(DB_PATH, 'utf-8'));
const saveDB = (data: any) => fs.writeFileSync(DB_PATH, JSON.stringify(data, null, 2));
```

```
// API Routes
app.post('/api/auth/login', (req, res) =>
{ const { email, password, role } =
req.body; const db = getDB();
const users = role === 'admin' ? db.admins : db.students;
const user = users.find((u: any) => u.email === email);

if(user && bcrypt.compareSync(password, user.password))
{ const token = jwt.sign({ id: user.id, role }, SECRET_KEY);
const { password: _, ...userWithoutPassword } = user;
res.json({ token, user: { ...userWithoutPassword, role } });
} else {
res.status(401).json({ error: 'Invalid credentials' });
}
});
```

```
app.post('/api/auth/register', (req, res) =>
{ const db = getDB();
```



```
const userData = req.body;

if (db.students.find((s: any) => s.email === userData.email || s.roll_no ===
userData.roll_no)) {
  return res.status(400).json({ error: 'Email or Roll No already exists' });
}

const newUser = {
  ...userData,
  id: 's' + Date.now(),
  password: bcrypt.hashSync(userData.password, 10),
  is_placed: false,
  is_active: true
};

db.students.push(newUser);
saveDB(db);

const token = jwt.sign({ id: newUser.id, role: 'student' }, SECRET_KEY);
const { password: _, ...userWithoutPassword } = newUser;
res.json({ token, user: { ...userWithoutPassword, role: 'student' } });
});

// Companies
app.get('/api/companies', (req, res) =>
  { const db = getDB();
    res.json(db.companies);
  });

app.post('/api/companies', (req, res) =>
  { const db = getDB();
    const newCo = { id: 'c' + Date.now(), ...req.body, letter: req.body.name.charAt(0) };
    db.companies.push(newCo);
    saveDB(db);
    res.json(newCo);
  });

app.patch('/api/companies/:id', (req, res) =>
  { const { id } = req.params;
    const db = getDB();
    const idx = db.companies.findIndex((c: any) => c.id === id);
    if (idx === -1) return res.status(404).json({ error: 'Not found' });

    db.companies[idx] = { ...db.companies[idx], ...req.body };
  });
```



```
saveDB(db);
res.json(db.companies[idx]);
});

// Applications
app.get('/api/applications', (req, res) =>
{ const db = getDB();
  const apps = db.applications.map((app: any) => ({
    ...app,
    student: db.students.find((s: any) => s.id === app.student_id),
    company: db.companies.find((c: any) => c.id === app.company_id),
  }));
  res.json(apps);
});

app.post('/api/applications', (req, res) =>
{ const { student_id, company_id } =
  req.body; const db = getDB();

  // Check eligibility
  const student = db.students.find((s: any) => s.id === student_id);
  const company = db.companies.find((c: any) => c.id === company_id);

  if (!student || !company) return res.status(404).json({ error: 'Data not found' });

  const reasons = [];
  if (student.ug_cgpa < company.min_cgpa) reasons.push('CGPA too low');
  if (student.pct_10 < company.min_pct_10) reasons.push('10th % too low');
  if (student.pct_12 < company.min_pct_12) reasons.push('12th % too low');
  if (!company.allow_backlog && student.backlogs > 0) reasons.push('Active backlogs');

  if (reasons.length > 0) {
    return res.status(400).json({ error: 'Not eligible', reasons });
  }

  const existing = db.applications.find((a: any) => a.student_id === student_id &&
a.company_id === company_id);
  if (existing) return res.status(400).json({ error: 'Already applied' });

  const newApp =
    { id: 'a' +
      Date.now(),
      student_id,
      company_id,
      status: 'Under Review',
```



```
    applied_at: new Date().toISOString().split('T')[0]
  };

  db.applications.push(newApp);

  // Create notification for student
  db.notifications.unshift({
    id: 'n' + Date.now(),
    student_id,
    title: 'Application Sent',
    message: `You applied for the ${company.role} role at ${company.name}.`,
    type: 'blue',
    is_read: false,
    created_at: new Date().toISOString()
  });

  // Create notification for admin
  db.notifications.unshift({
    id: 'na' + Date.now(),
    role: 'admin',
    title: 'New Application',
    message: `${student.name} applied for ${company.name}.`,
    type: 'info',
    is_read: false,
    created_at: new Date().toISOString()
  });

  saveDB(db);
  res.json(newApp);
});

app.patch('/api/applications/:id', (req, res) =>
  { const { id } = req.params;
    const { status, admin_note } = req.body;
    const db = getDB();
    const appIdx = db.applications.findIndex((a: any) => a.id === id);
    if (appIdx === -1) return res.status(404).json({ error: 'Not found' });

    const application = db.applications[appIdx];
    const student = db.students.find((s: any) => s.id === application.student_id);
    const company = db.companies.find((c: any) => c.id === application.company_id);

    db.applications[appIdx] = { ...application, status, admin_note, decided_at: new
    Date().toISOString() };
```



```
if(status === 'Selected') {
  const sIdx = db.students.findIndex((s: any) => s.id === application.student_id);
  if (sIdx !== -1) db.students[sIdx].is_placed = true;
}

// Create "Email" in internal mail system
const mailSubject = status === 'Selected' ? `Congratulations! You are selected at
${company.name}` : `Update on your application for ${company.name}`;
const mailBody = status === 'Selected'
  ? `Dear ${student.name}, We are pleased to inform you that you have been SELECTED
for the ${company.role} role at ${company.name} with a package of ${company.package}.
Please check the portal for next steps.`
  : `Dear ${student.name}, Thank you for your interest in the ${company.role} role at
${company.name}. Unfortunately, we will not be moving forward with your application at
this time. We wish you the best in your future endeavors.`;

db.mails.unshift({
  id: 'm' + Date.now(),
  student_id: student.id,
  subject: mailSubject,
  body: mailBody,
  from: 'Placement Cell',
  date: new Date().toISOString(),
  is_read: false
});

// Also create a notification for student
db.notifications.unshift({
  id: 'n' + Date.now(),
  student_id: student.id,
  title: status === 'Selected' ? `~a~🎉Offer Received!` : 'Application Update',
  message: mailSubject,
  type: status === 'Selected' ? 'ok' : 'warn',
  is_read: false,
  created_at: new Date().toISOString()
});

saveDB(db);
res.json(db.applications[appIdx]);
});

// Students
app.get('/api/students', (req, res) => {
```



```
const db = getDB();
res.json(db.students.map((s: any) =>
  { const { password: _, ...rest } = s;
  return rest;
}));
});

app.get('/api/students/:id', (req, res) =>
  { const { id } = req.params;
  const db = getDB();
  const student = db.students.find((s: any) => s.id === id);
  if (!student) return res.status(404).json({ error: 'Student not found' });
  const { password: _, ...studentWithoutPassword } = student;
  res.json(studentWithoutPassword);
});

app.patch('/api/students/:id', (req, res) =>
  { const { id } = req.params;
  const db = getDB();
  const idx = db.students.findIndex((s: any) => s.id === id);
  if (idx === -1) return res.status(404).json({ error: 'Student not found' });

  db.students[idx] = { ...db.students[idx], ...req.body };
  saveDB(db);
  const { password: _, ...studentWithoutPassword } = db.students[idx];
  res.json(studentWithoutPassword);
});

// Interviews
app.get('/api/interviews', (req, res) =>
  { const db = getDB();
  const interviews = db.interviews.map((iv: any) => ({
    ...iv,
    student: db.students.find((s: any) => String(s.id) === String(iv.student_id)),
    company: db.companies.find((c: any) => String(c.id) === String(iv.company_id)),
  }));
  res.json(interviews);
});

app.post('/api/interviews', (req, res) =>
  { const db = getDB();
  const { student_id, company_id, date, time } = req.body;
  const newIv = { id: 'iv' + Date.now(), ...req.body };
  db.interviews.unshift(newIv);
```



```
const company = db.companies.find((c: any) => c.id === company_id);
const student = db.students.find((s: any) => s.id === student_id);
```

```
if (company && student) {
  // Add Notification
  db.notifications.unshift({
    id: 'notif' + Date.now(),
    student_id: student_id,
    title: 'Interview Scheduled!',
    message: `You have an
```

src/App.tsx – Important Components

```
6 bg-teal rounded-full flex items-center justify-center mb-6 shadow-xl shadow-teal/20">
  <CheckCircle2 className="w-8 h-8 text-white" />
</div>
<h4 className="text-2xl font-black text-white uppercase tracking-tight mb-
4">Interview Performance</h4>
  <div className="bg-white/5 border border-white/10 rounded-2xl p-6 text-white/90
text-sm font-medium leading-relaxed max-w-xl text-left whitespace-pre-wrap">
    <div className="markdown-body">
      <ReactMarkdown>{feedback}</ReactMarkdown>
    </div>
  </div>
  <button
    onClick={() => setFeedback(null)}
    className="mt-8 px-8 py-4 bg-teal text-white rounded-xl font-black uppercase
tracking-widest hover:bg-teal2 transition-all shadow-xl shadow-teal/20"
  >
    Continue Interview
  </button>
</motion.div>
)}

{messages.map((m, i) => (
  <motion.div
    key={i}
    initial={{ opacity: 0, x: m.role === 'user' ? 20 : -20 }}
    animate={{ opacity: 1, x: 0 }}
    className={cn("flex", m.role === 'user' ? "justify-end" : "justify-start")}
  >
    <div className={cn(
      "max-w-[80%] p-5 rounded-2xl shadow-sm text-sm font-medium leading-relaxed",
      m.role === 'user' ? "bg-teal text-white rounded-tr-none" : "bg-white text-slate-700
```



```

border border-slate-100 rounded-tl-none"
    >>>
    {m.text}
  </div>
</motion.div>
))}
</AnimatePresence>
{loading && (
  <div className="flex justify-start">
    <div className="bg-white p-4 rounded-2xl border border-slate-100 flex items-
center gap-3">
      <div className="flex gap-1">
        <div className="w-1.5 h-1.5 bg-teal-500 rounded-full animate-bounce" />
        <div className="w-1.5 h-1.5 bg-teal-500 rounded-full animate-bounce
[animation-delay:0.2s]" />
        <div className="w-1.5 h-1.5 bg-teal-500 rounded-full animate-bounce
[animation-delay:0.4s]" />
      </div>
      <span className="text-xs text-slate-400 font-bold italic">Interviewer is
typing...</span>
    </div>
  </div>
)}
</div>

<div className="p-6 bg-white border-t border-slate-100">
  <div className="flex gap-4">
    <input
      value={input}
      onChange={e => setInput(e.target.value)}
      onKeyDown={e => e.key === 'Enter' && sendMessage()}
      placeholder="Type your response here..."
      className="flex-1 px-6 py-4 bg-slate-50 rounded-2xl border border-slate-200
outline-none focus:border-teal font-medium transition-all"
    />
    <button
      onClick={sendMessage}
      disabled={loading || !input.trim()}
      className="w-14 h-14 bg-teal text-white rounded-2xl flex items-center justify-center
hover:bg-teal2 transition-all shadow-lg shadow-teal/20 disabled:opacity-50"
    >
      <ArrowRight className="w-6 h-6" />
    </button>
  </div>

```



```
    </div>
  </motion.div>
);
};

const AICommandBar = ({ onAction }: { onAction: (action: any) => void }) =>
  { const [query, setQuery] = React.useState("");
    const [loading, setLoading] = React.useState(false);
    const [isListening, setIsListening] = React.useState(false);

    const recognitionRef = React.useRef<any>(null);

    React.useEffect(() => {
      // @ts-ignore
      const SpeechRecognition = window.SpeechRecognition ||
      window.webkitSpeechRecognition;
      if (SpeechRecognition) {
        recognitionRef.current = new SpeechRecognition();
        recognitionRef.current.continuous = false;
        recognitionRef.current.interimResults = false;
        recognitionRef.current.lang = 'en-US';

        recognitionRef.current.onstart = () =>
          { console.log('Speech recognition
            started');
          };

        recognitionRef.current.onresult = (event: any) =>
          { const transcript = event.results[0][0].transcript;
            console.log('Voice detected:', transcript);
            setQuery(transcript);
            setIsListening(false);
            processCommand(transcript);
          };

        recognitionRef.current.onerror = (event: any) =>
          { console.error('Speech recognition error',
            event.error); setIsListening(false);
            if (event.error === 'not-allowed') {
              alert("Microphone access denied. Please click the 'Lock' icon in your browser address
bar and allow Microphone access for this site.");
            } else if (event.error === 'no-speech')
              { console.warn('No speech detected');
            } else if (event.error === 'network') {
              alert("Network error during voice recognition. Please check your connection.");
            }
          }
        }
    }
  );
};
```



```
    } else {
      console.warn(`Voice recognition warning: ${event.error}`);
    }
  };

  recognitionRef.current.onend = () =>
    { console.log('Speech recognition
      ended'); setIsListening(false);
    };
  }
}, []);

const toggleListening = () =>
{ if (!recognitionRef.current)
{
  alert("Voice search is not supported in this browser. Please use Chrome or Edge.");
  return;
}

try {
  if (isListening)
    { recognitionRef.current.stop
      (); setIsListening(false);
    } else
    { setQuery('
      ');
      recognitionRef.current.start();
      setIsListening(true);
    }
} catch (err: any) {
  console.error('Speech recognition error:', err);
  // Handle the "already started" error gracefully
  if (err?.error === 'aborted' || err?.message?.includes('already started'))
    { setIsListening(true);
  } else
    { setIsListening(fals
      e);
    }
}
};

const processCommand = async (text: string) =>
{ if (!text.trim()) return;
  setLoading(true);
  try {
```

```
const response = await
  ai.models.generateContent({ model: "gemini-3-flash-
    preview",
```



contents: `You are an AI Assistant for the CareerAI Hub.
Translate the student/admin request into a structured JSON action.

Command: "\${text}"

Available Actions:

1. navigate: { "type": "navigate", "page": "dashboard" | "companies" | "students" | "applications" | "interviews" | "announcements" | "jobs" | "my_apps" | "profile" | "resume" }
2. filter_students: { "type": "filter_students", "params": { "is_placed": boolean, "department": string, "cgpa_min": number } }
3. open_modal: { "type": "open_modal", "modal": "interview" | "company" | "announcement", "student_name": string }

Examples:

- "Show unplaced students" -> { "type": "filter_students", "params": { "is_placed": false } }
- "Schedule interview for Adarsh" -> { "type": "open_modal", "modal": "interview", "student_name": "Adarsh" }
- "Go to jobs" -> { "type": "navigate", "page": "jobs" }
- "Open profile" -> { "type": "navigate", "page": "profile" }

Return ONLY valid JSON.`,

```
config: { responseType: "application/json" }
});
```

```
const actionText = response.text;
const cleanJson = actionText.replace(/```json/g, "").replace(/```/g, "").trim();
const action = JSON.parse(cleanJson);
if (action)
{ onAction(action); setQuery("");
} else {
console.error('AI returned empty action');
}
} catch (err) {
console.error('AI Command failed', err);
// Fallback: If AI fails, try a simple keyword search
if (text.toLowerCase().includes('student'))
{ onAction({ type: 'navigate', page: 'students' });
} else if (text.toLowerCase().includes('job') || text.toLowerCase().includes('compan'))
{ onAction({ type: 'navigate', page: 'companies' });
}
} finally
{ setLoading(false);
}
```



```

    }
  };

  const handleSubmit = (e: React.FormEvent) =>
  { e.preventDefault();
    processCommand(query);
  };

  return (
    <form onSubmit={handleSubmit} className="relative group max-w-sm w-full">
      {isListening && (
        <div className="absolute -top-12 left-0 right-0 flex justify-center">
          <div className="bg-red-500 text-white text-[10px] font-black uppercase px-4 py-1.5 rounded-full animate-bounce shadow-lg flex items-center gap-2">
            <Sparkles className="w-3 h-3" /> Listening for your voice...
          </div>
        </div>
      )}
      <div className="absolute inset-0 bg-navy/5 rounded-2xl blur-lg transition-all group-hover:bg-navy/10" />
      <div className="relative flex items-center bg-white border border-slate-200 rounded-2xl px-4 h-12 shadow-sm focus-within:ring-4 focus-within:ring-navy/5 focus-within:border-navy transition-all">
        <Terminal className="w-4 h-4 text-navy/40 mr-3" />
        <input
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          placeholder={isListening ? "Listening..." : "Ask AI Command Center..."}
          className="flex-1 bg-transparent border-none outline-none text-xs font-black uppercase tracking-tight text-slate-700 placeholder:text-slate-400 placeholder:normal-case"
        />
        <div className="flex items-center gap-1">
          <button
            type="button"
            onClick={toggleListening}
            className={cn(
              "p-2 rounded-lg transition-all",
              isListening ? "bg-red-500 text-white animate-pulse" : "text-slate-400 hover:bg-slate-50 hover:text-navy"
            )}
            title="Voice Search"
          />
          {isListening ? <MicOff className="w-4 h-4" /> : <Mic className="w-4 h-4" />}
        </div>
      </div>
    </form>
  );

```



```

        <button type="submit" disabled={loading} className="p-2 hover:bg-slate-50
rounded-lg transition-all text-navy">
          {loading ? <RefreshCw className="w-4 h-4 animate-spin" /> : <Send
className="w-4 h-4" />}
        </button>
      </div>
    </div>
  </form>
);
};

const SidebarItem = ({ icon: Icon, label, active, onClick, badge }: any) => (
  <button
    onClick={onClick}
    className={cn(
      "w-full flex items-center gap-3 px-4 py-3 rounded-xl transition-all duration-200 group
text-left",
      active
        ? "bg-white/15 text-white shadow-lg"
        : "text-white/60 hover:bg-white/5 hover:text-white"
    )}
  >
    <Icon className={cn("w-5 h-5", active ? "text-gold" : "text-white/40 group-hover:text-
white/80")} />
    <span className="font-medium flex-1">{label}</span>
    {badge > 0 && (
      <span className="bg-red-500 text-white text-[10px] font-bold px-1.5 py-0.5 rounded-
full">
        {badge}
      </span>
    )}
    {active && <ChevronRight className="w-4 h-4 text-white/40" />}
  </button>
);

const ResumePreview = ({ user }: { user: User }) =>
  { return (
    <motion.div initial={{ opacity: 0 }} animate={{ opacity: 1 }} className="bg-white p-12
rounded-3xl border border-slate-200 shadow-2xl max-w-4xl mx-auto font-serif" id="resume-
printable">
      <div className="flex justify-between border-b-2 border-navy pb-8 mb-8">
        <div>
          <h1 className="text-4xl font-black text-navy uppercase tracking-tighter mb-
2">{user.name}</h1>

```



```
<div className="flex items-center gap-4 text-slate-500 font-bold text-sm uppercase
tracking-widest">
  <span className="flex items-center gap-2"><Mail className="w-4 h-4" />
{user.email}</span>
  <span className="flex items-center gap-2"><MapPin className="w-4 h-4" />
{user.location || 'India'}</span>
</div>
</div>
<div className="text-right">
  <p className="text-sm font-black text-teal uppercase tracking-widest mb-
1">{user.ug_degree}</p>
  <p className="text-[10px] text-slate-400 font-bold
uppercase">{user.department}</p>
</div>
</div>

<div className="grid grid-cols-3 gap-12">
  <div className="col-span-2 space-y-10">
    <section>
      <h2 className="text-xs font-black text-navy uppercase tracking-[4px] mb-4 border-b
border-slate-100 pb-2">Professional Summary</h2>
      <p className="text-slate-600 leading-relaxed text-sm italic">" {user.about || 'A
dedicated student looking for opportunities.'}"</p>
    </section>

    <section>
      <h2 className="text-xs font-black text-navy uppercase tracking-[4px] mb-4 border-b
border-slate-100 pb-2">Technical Skills</h2>
      <div className="flex flex-wrap gap-2">
        {user.skills?.split(',').map((s, i) => (
          <span key={i} className="px-3 py-1 bg-slate-50 border border-slate-200 rounded
text-[10px] font-bold text-slate-700 uppercase tracking-tight">{s.trim()}</span>
        ))}
      </div>
    </section>
  </div>

  <div className="space-y-10">
    <section>
      <h2 className="text-xs font-black text-navy uppercase tracking-[4px] mb-4 border-b
border-slate-100 pb-2">Academic Score</h2>
      <div className="space-y-4">
        <div className="flex justify-between items-center px-4 py-3 bg-navy text-white
rounded-2xl">
```



```
        <span className="text-[10px] font-bold uppercase">CGPA</span>
        <span className="text-xl font-black">{user.ug_cgpa}</span>
      </div>
      <div className="flex justify-between items-center px-4 py-2 bg-slate-50 rounded-
xl border border-slate-100">
        <span className="text-[9px] font-bold text-slate-400 uppercase">10th %</span>
        <span className="text-sm font-bold text-slate-700">{user.pct_10}%</span>
      </div>
      <div className="flex justify-between items-center px-4 py-2 bg-slate-50 rounded-
xl border border-slate-100">
        <span className="text-[9px] font-bold text-slate-400 uppercase">12th %</span>
        <span className="text-sm font-bold text-slate-700">{user.pct_12}%</span>
      </div>
    </div>
  </section>

  <section>
    <h2 className="text-xs font-black text-navy uppercase tracking-[4px] mb-4 border-
b border-slate-100 pb-2">Education</h2>
    <div>
      <p className="text-sm font-extrabold text-slate-900">{user.year_pass} Batch</p>
      <p className="text-xs text-slate-500 mt-1">{user.college}</p>
    </div>
  </section>
</div>
</div>

<div className="mt-12 pt-8 border-t border-slate-100 text-center">
  <p className="text-[9px] text-slate-300 font-bold uppercase tracking-
widest">Generated via CareerAI Hub</p>
</div>
</motion.div>
);
};

const ResumeEditor = ({ user, onSave, onCancel }: { user: User, onSave: (data:
Partial<User>) => void, onCancel: () => void }) => {
  const [showPreview, setShowPreview] = React.useState(false);
  const [formData, setFormData] = React.useState({
    abo
```

3. login.php – Authentication Module

Implements user login, session initialization and credential validation.



login.php – Important Code

```
<?php
// login.php
include 'db_connect.php';
session_start();

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $email = $_POST['email'];
    $password = $_POST['password'];

    $sql = "SELECT id, name, role FROM users WHERE email = '$email' AND password = '$password'";
    $result = $conn->query($sql);

    if ($result->num_rows > 0) {
        $user = $result->fetch_assoc();
        $_SESSION['user_id'] = $user['id'];
        $_SESSION['user_name'] = $user['name'];
        $_SESSION['role'] = $user['role'];
        header("Location: index.php");
    } else {
        $error = "Invalid Email or Password!";
    }
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Login - Placement Cell</title>
    <link rel="stylesheet" href="style.css">
    <style>
        .login-box { max-width: 400px; margin: 100px auto; background: white; padding: 40px;
border-radius: 15px; box-shadow: 0 10px 25px rgba(0,0,0,0.1); }
        input { width: 100%; padding: 12px; margin: 10px 0; border: 1px solid #ddd; border-
radius: 8px; box-sizing: border-box; }
        button { width: 100%; padding: 12px; background: #3498db; color: white; border: none;
border-radius: 8px; cursor: pointer; font-weight: bold; }
        .error { color: red; font-size: 14px; margin-bottom: 10px; }
    </style>
</head>
<body style="background: #f0f2f5;">
    <div class="login-box">
        <h2 style="text-align: center;">Sign In</h2>
```



```
<p style="text-align: center; color: #666; font-size: 14px;">Campus Placement  
Management</p>
```

```
<?php if(isset($error)) echo "<p class='error'>$error</p>"; ?>
```

```
<form method="POST">  
  <input type="email" name="email" placeholder="Email (admin@example.com)"  
required>  
  <input type="password" name="password" placeholder="Password (admin123)"  
required>  
  <button type="submit">Login</button>  
</form>
```

```
<div style="margin-top: 20px; text-align: center; font-size: 12px; color: #888;">  
  <p>Admin: admin@example.com / admin123</p>  
  <p>Student: john@student.com / pass123</p>  
</div>  
</div>  
</body>  
</html>
```

4. profile.php – User Profile Module

Displays profile information and academic details for students.

profile.php – Important Code

```
<?php  
include 'db_connect.php';  
include 'header.php';  
  
if (!isset($_SESSION['user_id']))  
  { header("Location: login.php");  
    exit();  
  }  
  
$user_id = $_SESSION['user_id'];  
  
// Get User & Student details  
$sql = "SELECT u.*, s.roll_no, s.cgpa, s.branch, s.skills  
FROM users u  
LEFT JOIN students s ON u.id = s.student_id  
WHERE u.id = $user_id";  
$result = $conn->query($sql);
```



```
$user_data = $result->fetch_assoc();
?>

<div class="container">
  <h2>User Profile Details</h2>
  <hr>
  <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 20px; margin-top: 20px;">
    <div class="card" style="background: #f8f9fa; color: #333; text-align: left; border: 1px
solid #ddd;">
      <h3>Basic Information</h3>
      <p><b>Name:</b> <?php echo $user_data['name']; ?></p>
      <p><b>Email:</b> <?php echo $user_data['email']; ?></p>
      <p><b>Role:</b> <?php echo ucfirst($user_data['role']); ?></p>
      <p><b>Member Since:</b> <?php echo date('d M Y',
strtotime($user_data['created_at'])); ?></p>
    </div>

    <?php if ($user_data['role'] == 'student'): ?>
      <div class="card" style="background: #e3f2fd; color: #333; text-align: left; border: 1px
solid #ddd;">
        <h3>Academic Details</h3>
        <p><b>Roll No:</b> <?php echo $user_data['roll_no'] ? : 'N/A'; ?></p>
        <p><b>Branch:</b> <?php echo $user_data['branch'] ? : 'N/A'; ?></p>
        <p><b>CGPA:</b> <?php echo $user_data['cgpa'] ? : '0.0'; ?></p>
        <p><b>Skills:</b> <?php echo $user_data['skills'] ? : 'Update your skills'; ?></p>
      </div>
    <?php endif; ?>
  </div>

  <div style="margin-top: 30px;">
    <button onclick="window.print()" style="padding: 10px 20px; background: #2c3e50;
color: white; border: none; border-radius: 5px; cursor: pointer;">Print Profile for
Submission</button>
  </div>
</div>

</body>
</html>
```

5. Database SQL Schema

Core database design containing user, student, company and interview tables with relationships and sample data.



database.sql – Schema Definition

-- Database: placement_db

CREATE DATABASE IF NOT EXISTS placement_db;

USE placement_db;

-- 1. Users Table

```
CREATE TABLE IF NOT EXISTS users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    password VARCHAR(255) NOT NULL,  
    role ENUM('admin', 'student') DEFAULT 'student',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- 2. Students Table

```
CREATE TABLE IF NOT EXISTS students (  
    student_id INT PRIMARY KEY,  
    roll_no VARCHAR(20) UNIQUE,  
    cgpa DECIMAL(3,2),  
    branch VARCHAR(50),  
    skills TEXT,  
    FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

-- 3. Companies Table

```
CREATE TABLE IF NOT EXISTS companies  
( id INT AUTO_INCREMENT PRIMARY  
KEY, name VARCHAR(100) NOT NULL,  
website VARCHAR(255),  
role_offered VARCHAR(100),  
ctc VARCHAR(50)  
);
```

-- 4. Interviews Table

```
CREATE TABLE IF NOT EXISTS interviews  
( id INT AUTO_INCREMENT PRIMARY  
KEY,  
student_id INT,  
company_id INT,  
batch_date DATE,  
status ENUM('scheduled', 'completed', 'cancelled') DEFAULT 'scheduled',  
FOREIGN KEY (student_id) REFERENCES users(id),  
FOREIGN KEY (company_id) REFERENCES companies(id)  
);
```



-- Insert Sample Data

```
INSERT INTO users (name, email, password, role) VALUES  
( 'Admin User', 'admin@example.com', 'admin123', 'admin'),  
( 'John Doe', 'john@student.com', 'pass123', 'student');
```

```
INSERT INTO students (student_id, roll_no, cgpa, branch, skills) VALUES  
(2, 'CS101', 8.5, 'Computer Science', 'React, Node, SQL');
```

```
INSERT INTO companies (name, website, role_offered, etc) VALUES  
( 'Google', 'google.com', 'Software Engineer', '30 LPA'),  
( 'Microsoft', 'microsoft.com', 'DevOps Engineer', '25 LPA');
```



11.CONCLUSION AND FUTURE ENHANCEMENT

11.1 CONCLUSION

The CareerAI Hub: Campus Placement Management System was developed to automate and streamline the campus recruitment process. The system successfully replaces traditional manual methods of managing student records, placement drives, recruiter information, and application tracking with a centralized web-based platform.

The application enables students to create profiles, apply for placement drives, and track their application status efficiently. Recruiters can manage job opportunities and review eligible candidates, while administrators can oversee the entire placement process through a dedicated dashboard. The system improves accuracy, reduces paperwork, minimizes human errors, and provides real-time access to placement information.

The use of technologies such as HTML5, CSS3, JavaScript (ES6), PHP 8.2, MySQL, Node.js, and Express.js ensures a secure, reliable, and scalable solution. Overall, the project successfully achieves its objective of enhancing the efficiency, transparency, and effectiveness of campus placement management.

11.2FUTURE ENHANCEMENT

Although the current system meets the essential requirements of campus placement management, several enhancements can be implemented in the future to improve functionality and user experience.

Future Enhancements

- Integration of Artificial Intelligence (AI) for resume screening and candidate recommendations.
- Implementation of Machine Learning algorithms to predict placement trends and student success rates.
- Development of a mobile application for Android and iOS platforms.
- Integration of email and SMS notification services for instant communication.
- Addition of online aptitude tests and coding assessments within the platform.
- Implementation of video interview scheduling and management features.
- Cloud deployment to improve scalability, availability, and data security.
- Integration with professional networking platforms such as LinkedIn for profile verification and recruitment opportunities.
- Advanced analytics and reporting dashboards for placement coordinators.
- Multi-college and multi-campus support for centralized placement management.



12. BIBLIOGRAPHY

1. Pressman, R. S. (2019). Software Engineering: A Practitioner's Approach. McGraw-Hill Education.
2. Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems. Pearson Education.
3. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts. McGraw-Hill Education.
4. Duckett, J. (2021). HTML and CSS: Design and Build Websites. Wiley Publications.
5. Nixon, R. (2023). Learning PHP, MySQL & JavaScript. O'Reilly Media.
6. [PHP Documentation](#)
7. [MySQL Documentation](#)
8. [Node.js Documentation](#)
9. [Express.js Documentation](#)
10. Research papers and articles related to Campus Placement Management Systems, Recruitment Systems, and Web Application Development.